

Infraestrutura de Hardware

Ronierison. Maciel

CESAR School

Fevereiro de 2026



Nome: Roni

Formação: Mestre em Ciência da Computação

Ocupação: Pesquisador, Professor e Escovador de bits

Hobbies: Jogar cartas, ficar com a família

Interesses: Carros, DevSec, DS e ML

Email: rsm4@cesar.school

GitHub: <https://github.com/0x03C1>

- 1 Módulo 1 – Circuitos Combinacionais e Aritméticos
- 2 Módulo 2 – Lógica Sequencial
- 3 Módulo 3 – Organização e Arquitetura do Computador
- 4 Módulo 4 – Conjunto de Instruções e Programação Assembly
- 5 Módulo 5 – Processador Monociclo
- 6 Módulo 6 – Processador Multiciclo
- 7 Módulo 7 – Pipeline
- 8 Módulo 8 – Exceções, Interrupções e Entrada/Saída
- 9 Módulo 9 – Hierarquia de Memória e Cache
- 10 Módulo 10 – Memória Principal e Armazenamento
- 11 Módulo 11 – Memória Virtual
- 12 Módulo 12 – Barramento
- 13 Módulo 13 – Arquiteturas Avançadas

O que vamos estudar?

- **Fundamentos Digitais:** Como a eletricidade vira lógica.
- **Aritmética Computacional:** Como o hardware faz contas rápidas.
- **A Unidade de Controle:** O cérebro dentro do cérebro.
- **Hierarquia de Memória:** O segredo da velocidade (Cache, RAM).
- **O Computador Moderno:** Pipeline, Paralelismo e RISC vs CISC.

Módulo	Tópicos Principais
1. Circuitos Combinacionais e Aritméticos	MUX, DEMUX, Somadores
2. Lógica Sequencial	Latches, Flip-Flops, Contadores, FSM
3. Organização e Arquitetura	Von Neumann, Harvard, Flynn
4. Conjunto de Instruções e Assembly	MIPS, Registradores, Desvios
5. Processador Monociclo	Datapath, UC, Caminho Crítico
6. Processador Multiciclo	Datapath, FSM de Controle
7. Pipeline	Estágios, Hazards, Branch Prediction
8. Exceções, Interrupções e E/S	Handler, MMIO, DMA
9. Hierarquia de Memória e Cache	AMAT, Mapeamento, Políticas
10. Memória Principal e Armazenamento	DRAM, DDR, Scheduling
11. Memória Virtual	Paginação, TLB, Page Faults
12. Barramento	PCIe, USB, SATA, Arbitração
13. Arquiteturas Avançadas	Superscalar, Multicore, GPU

Datas Importantes:

- AV1: 16/04
- AV2: 16/06
- Status Reports (Projetos)

Critérios:

- Provas Teóricas (6,0)
- Projetos Práticos (3,0)
- Participação/Labs (1,0)

- O que realmente acontece quando usamos um computador?
- Por que máquinas se comportam de forma diferente?

- **O que vocês usam todos os dias?**

- Notebook
- Smartphone
- Tablet
- Smartwatch

Alguém já teve um celular que travava com o tempo?



Hardware

vs

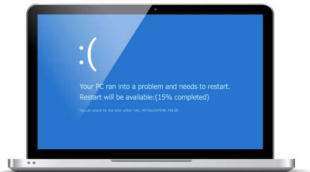
Software

Físico
Tangível

Lógico
Intangível



Aquilo que você bate

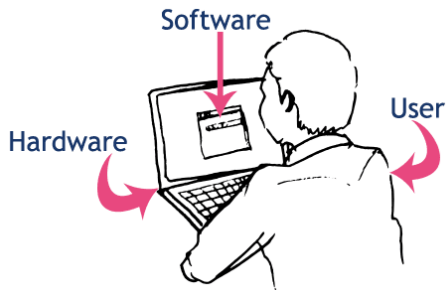


Aquilo que você xinga

Hardware e Software não vivem separados

- **Eles funcionam juntos o tempo todo**

- O usuário interage com o software
- O software solicita recursos ao hardware
- O hardware executa as operações

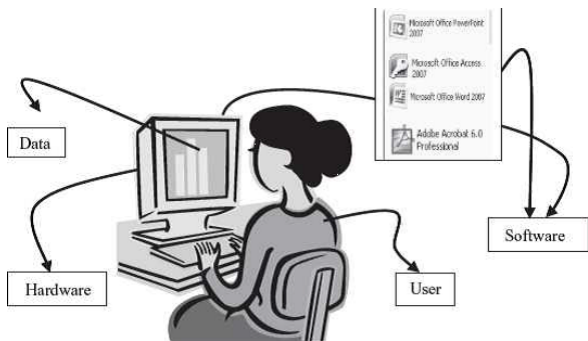


Como hardware e software interagem para executar aplicações do dia a dia?



O que acontece quando você clica em um app?

- ❶ O clique é capturado pelo sistema operacional
- ❷ O SO solicita recursos ao processador
- ❸ O programa é carregado na memória
- ❹ A CPU executa instruções
- ❺ O app aparece na tela



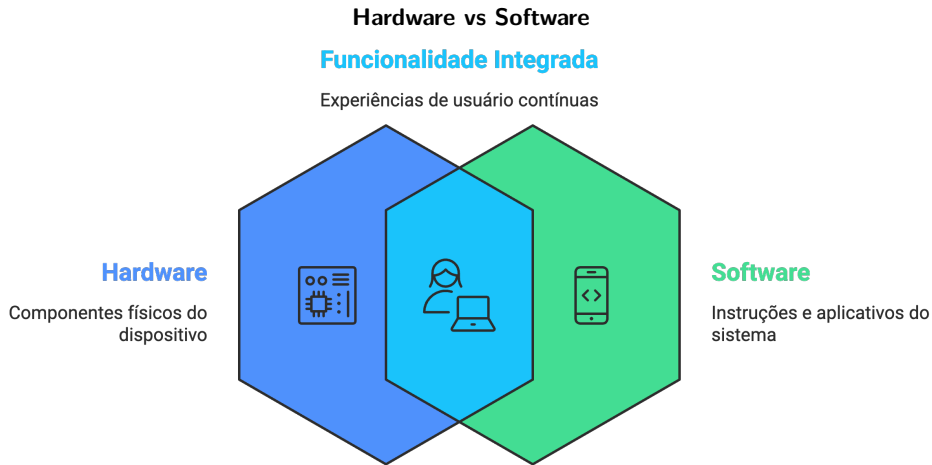
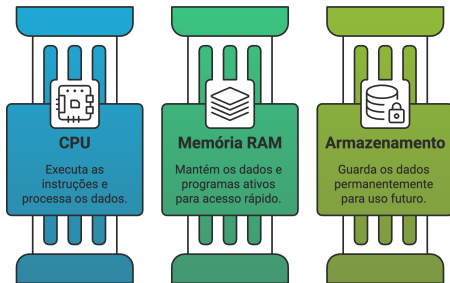


Figura: Onde o Hardware e o Software se encontram para o desempenho?

Quem Faz o trabalho pesado?



- **CPU** = trabalhador
- **RAM** = mesa de trabalho
- **Armazenamento** = arquivo morto

- **Situação comum:**

- Mesmo aplicativo
- Mesma versão
- Dois computadores diferentes
- Experiência diferente

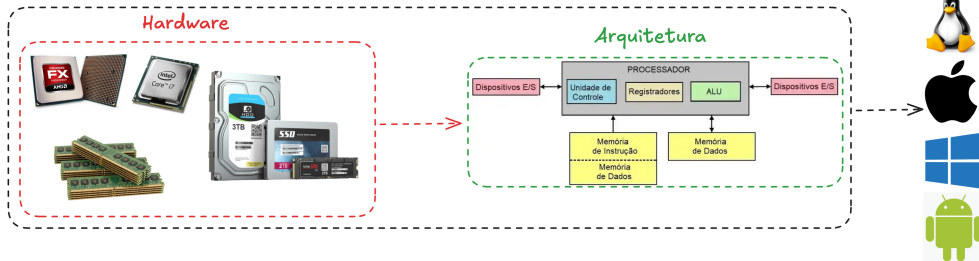


Por que um mesmo software pode ter desempenho diferente em dois computadores?



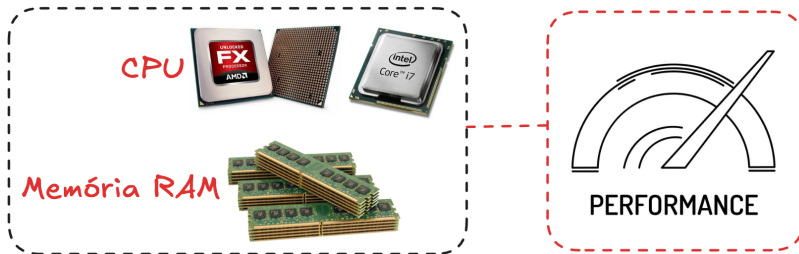
● Possíveis causas:

- Processadores diferentes
- Quantidade de memória RAM
- Tipo de armazenamento (HD vs SSD)
- Arquitetura do hardware
- Sistema operacional



- Hoje vamos olhar principalmente para:

- **CPU** (velocidade, núcleos)
- **Memória RAM** (quantidade e acesso)
- *Impacto direto no desempenho*



Por que isso importa para você?



Por que isso importa para você?

- **Impacto direto na vida profissional:**

- Desenvolvimento de software
- Escolha de equipamentos
- Otimização de sistemas
- Diagnóstico de problemas



HIGH PERFORMANCE

- Diferenciar sistemas analógicos e digitais.
- Identificar exemplos reais de sinais analógicos e digitais.
- Compreender por que computadores são sistemas digitais.
- Relacionar precisão, ruído e confiabilidade aos sistemas digitais.

O que é um Sinal?

Qual tipo de representação de informação devo usar?

Analógico

Adequado para informações contínuas onde a precisão é crucial (ex: áudio, vídeo).



Digital

Ideal para informações discretas onde a eficiência e a precisão são importantes (ex: dados de computador, comunicação).

- **A Era Analógica:** Regras de cálculo, integradores mecânicos e os primeiros computadores eletrônicos analógicos que resolviam equações diferenciais usando voltagens.
- **A Revolução Digital:** A transição para o binário (Shannon e Boole) permitiu a correção de erros e a miniaturização (Lei de Moore).



A Era Analógica

- Na era analógica, a máquina imitava fisicamente o fenômeno real.
- Ela não calculava números, ela simulava comportamentos.

Você queria resolver

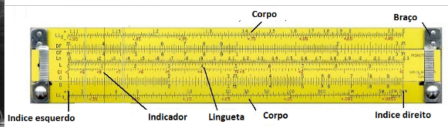
$$y'' + 3y' + 2y = 0$$

- A máquina tinha:
 - Rodas = multiplicação
 - Discos = integração
 - Eixos = soma

Vannevar Bush



Régua de cálculo



A Era Analógica

- Girava uma manivela **entrada**
- Movia um eixo **variável**
- Acionava discos **integração**
- Ajustava engrenagens **coeficientes**
 - O operador girava manivelas
 - E o sistema mecânico obedecia a equação.
 - Resolver cálculo com engrenagem.

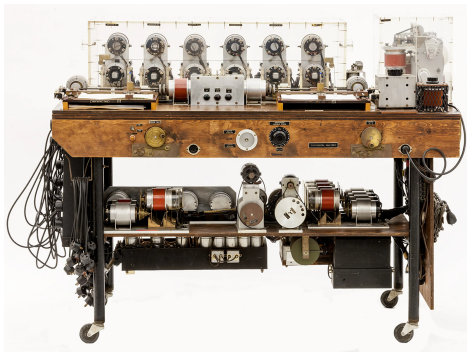
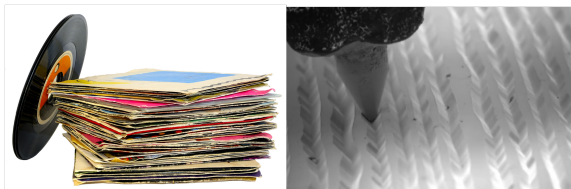


Figura: Analisador diferencial de Arnold

- **Áudio:** As ranhuras de um disco de vinil (analógico) vs. o arquivo FLAC/MP3 (digital).
- **Relógios:** O movimento contínuo dos ponteiros vs. a atualização de quartzo em números.



mp3

- **Analógico:** Qualquer interferência (ruído térmico, EMI) altera o dado permanentemente. Não há "recuperação" perfeita.
- **Digital:** Trabalha com limiares (thresholds). Se um sinal de 5V cair para 4.2V devido ao ruído, o circuito lógico ainda o interpreta como "1". Isso permite a regeneração do sinal.
- Simulador **falstad.com**
- Simulador **everycircuit.com**

- **Hardware Analógico:** Utiliza transistores na região linear (amplificação). Sensível a variações de temperatura.
- **Hardware Digital:** Utiliza transistores como chaves (On/Off). Base da Lógica Booleana.

Tabela: Comparação entre Sistemas Analógicos e Digitais

Característica	Analógico	Digital
Sinal	Contínuo	Discreto (0s e 1s)
Precisão	Limitada pelo ruído	Limitada pelo número de bits
Armazenamento	Difícil e degrada com o tempo	Fácil (SSD, Nuvem) sem perda
Processamento	Em tempo real, mas inflexível	Programável e altamente flexível
Exemplo HW	Sensores de pressão puros	CPUs, Memória RAM

- Com o avanço da IA, pesquisadores estão criando "chips neuromórficos" analógicos que imitam neurônios para economizar energia.
- O digital venceu pela imunidade ao ruído e facilidade de programação.
- O futuro aponta para sistemas híbridos (Computação Quântica e Chips de IA Analógicos).

Multiplexadores e Demultiplexadores

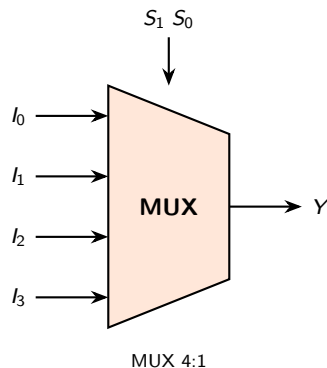
- Conceito e funcionamento de MUX e DEMUX
- Tabelas verdade e expressões booleanas
- MUX como gerador de funções lógicas
- Cascadeamento de MUX
- Atividade prática em duplas

O que é um Multiplexador?

Definição

Um **multiplexador** (MUX) é um circuito combinacional que seleciona **uma** dentre 2^n entradas e a encaminha para uma única saída, usando n linhas de seleção.

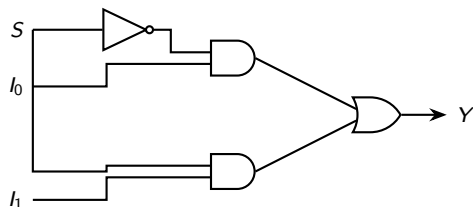
Analogia: Um controle remoto de TV com vários canais – você escolhe *qual canal* assistir (seleção), e apenas aquele sinal chega à tela (saída).



Expressão booleana:

$$Y = \bar{S} \cdot I_0 + S \cdot I_1$$

S	Y	Seleção
0	I_0	Entrada 0
1	I_1	Entrada 1



S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

Expressão:

$$Y = \overline{S_1} \overline{S_0} I_0 + \overline{S_1} S_0 I_1 + S_1 \overline{S_0} I_2 + S_1 S_0 I_3$$

Padrão geral – MUX $2^n:1$

- 2^n entradas de dados
- n linhas de seleção
- 1 saída
- A combinação binária de S seleciona qual I_k passa para Y

Para implementar:

- 2^n portas AND ($n + 1$ entradas cada)
- 1 porta OR (2^n entradas)
- n inversores

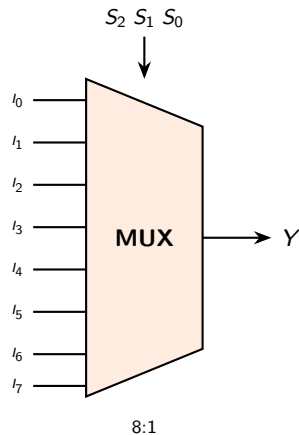
Especificação:

- 8 entradas: $I_0, I_1, \dots, I_7$
- 3 linhas de seleção: S_2, S_1, S_0
- 1 saída: Y

Expressão geral:

$$Y = \sum_{k=0}^7 m_k \cdot I_k$$

onde m_k é o mintermo k de (S_2, S_1, S_0) .



Ideia-chave

Qualquer função booleana de n variáveis pode ser implementada com um MUX $2^n:1$, conectando as variáveis às entradas de seleção e configurando as entradas de dados com 0 ou 1 conforme a tabela verdade.

Exemplo: $F(A, B) = A \oplus B$ com MUX 4:1

A	B	F	Entrada
0	0	0	$I_0 = 0$
0	1	1	$I_1 = 1$
1	0	1	$I_2 = 1$
1	1	0	$I_3 = 0$

Conexão: $S_1 = A$, $S_0 = B$

Configuração: $I_0 = 0$, $I_1 = 1$, $I_2 = 1$, $I_3 = 0$

Resultado

O MUX 4:1 implementa a função XOR sem usar portas XOR! Essa técnica é fundamental para **FPGAs** (LUTs).

Objetivo: Implementar função de n variáveis com MUX $2^{n-1}:1$

Exemplo: $F(A, B, C) = \sum m(1, 2, 4, 7)$ usando MUX 4:1

Usamos A, B como seleção. A variável C vai para as entradas:

A	B	$F(C=0)$	$F(C=1)$	Entrada MUX
0	0	0	1	$I_0 = C$
0	1	1	0	$I_1 = \overline{C}$
1	0	1	0	$I_2 = \overline{C}$
1	1	0	1	$I_3 = C$

Regra prática

Ambos 0 $\rightarrow$ entrada = 0 | Ambos 1 $\rightarrow$ entrada = 1

Cresce (0,1) $\rightarrow C$ | Decresce (1,0) $\rightarrow \overline{C}$

Problema: $F(A, B, C) = \sum m(0, 3, 5, 6)$ usando MUX 4:1

Passo 1: Montar a tabela verdade completa:

A	B	C	F
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

Passo 2: Agrupar por (A, B) e analisar o comportamento de C :

A	B	$F(C=0)$	$F(C=1)$	Entrada MUX
0	0	1	0	$I_0 = \overline{C}$
0	1	0	1	$I_1 = C$
1	0	0	1	$I_2 = C$
1	1	1	0	$I_3 = \overline{C}$

Passo 3: Conectar $S_1 = A$, $S_0 = B$, e as entradas conforme a tabela.

Economia

Usamos 1 MUX 4:1 + 1 inversor, em vez de 1 MUX 8:1!

O que é um Demultiplexador?

Definição

Um **demultiplexador** (DEMUX) faz a operação inversa do MUX: recebe **uma** entrada e a direciona para **uma** dentre 2^n saídas, conforme n linhas de seleção.

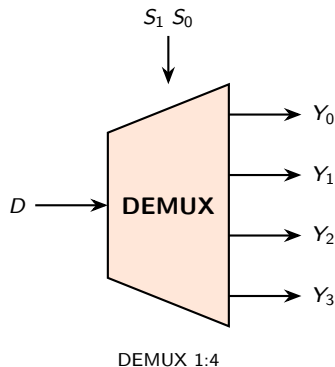
Expressões do DEMUX 1:4:

$$Y_0 = D \cdot \overline{S_1} \cdot \overline{S_0}$$

$$Y_1 = D \cdot \overline{S_1} \cdot S_0$$

$$Y_2 = D \cdot S_1 \cdot \overline{S_0}$$

$$Y_3 = D \cdot S_1 \cdot S_0$$



S_1	S_0	Y_0	Y_1	Y_2	Y_3
0	0	D	0	0	0
0	1	0	D	0	0
1	0	0	0	D	0
1	1	0	0	0	D

Relação com Decodificadores

Um DEMUX $1:2^n$ é estruturalmente idêntico a um **decodificador** $n:2^n$ com entrada de *enable*. Quando $D = 1$, funciona como decodificador puro.

Multiplexadores:

- Seleção de registradores no processador
- Roteamento de dados no datapath
- LUTs em FPGAs
- Multiplexação temporal (comunicações)
- Seleção de operandos na ULA

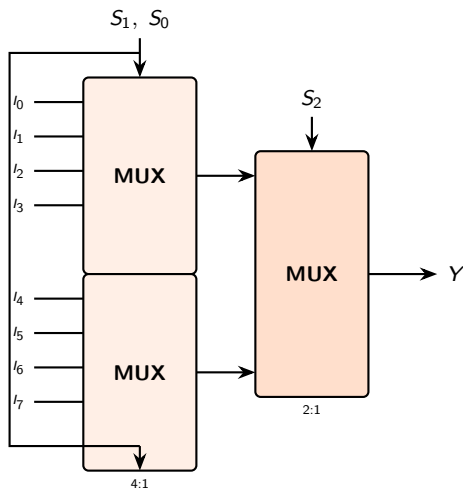
Demultiplexadores:

- Seleção de chip (*chip select*) em memória
- Distribuição de sinais de controle
- Endereçamento de barramentos
- Demultiplexação de canais
- Decodificação de instruções

Conexão com a disciplina

Nas próximos módulos, veremos MUX dentro do **datapath do MIPS**, selecionando entre registradores, imediatos e resultados da ULA.

Cascadeamento – MUX 8:1 a partir de MUX menores



S_1, S_0 selecionam dentro de cada MUX 4:1. S_2 escolhe qual MUX 4:1 passa.

Desafio 1

No Logisim, projete um **MUX 8:1** usando apenas MUX 4:1 e MUX 2:1.

- 1 Conecte 8 entradas, 3 seleções e 1 saída
- 2 Teste com pelo menos 3 combinações de seleção
- 3 Documente: quais sinais vão para qual MUX?

Desafio 2

Implemente $F(A, B, C) = \sum m(0, 2, 5, 6, 7)$ de duas formas:

- 1 Usando MUX 8:1 (método direto)
- 2 Usando MUX 4:1 + inversores (redução de variável)

Compare: qual usa menos componentes?

O que aprendemos:

- MUX: seletor $2^n \rightarrow 1$
- DEMUX: distribuidor $1 \rightarrow 2^n$
- MUX como gerador universal de funções
- Técnica de redução de variáveis
- Cascadeamento para MUX maiores

Para a próxima aula:

- Pesquisar: como um processador soma dois números binários?
- Revisar: operação XOR e soma binária
- Pensar: $1 + 1 = 10$ em binário – como representar o "vai-um"?

Questão para reflexão

Um multiplexador 4:1 possui as seguintes entradas:

$$I_0 = 0, \quad I_1 = 1, \quad I_2 = 0, \quad I_3 = 1$$

$$\text{Linhas de seleção: } S_0 = 0, \quad S_1 = 1$$

Qual é o valor de saída Y ?

Escreva a equação lógica completa do **MUX 4:1** e substituindo os valores fornecidos.

Dica

Lembre-se: os bits $S_1 S_0$ formam o índice binário da entrada selecionada. Escreva a equação:

$$Y = \bar{S}_1 \bar{S}_0 I_0 + \bar{S}_1 S_0 I_1 + S_1 \bar{S}_0 I_2 + S_1 S_0 I_3$$

Questão para reflexão

Um demultiplexador 1:4 recebe $D = 1$ e linhas de seleção $S_0 = 1$, $S_1 = 0$.

- 1 Indique quais saídas Y_0 , Y_1 , Y_2 , Y_3 estarão em nível alto.

Dica

Equações do DEMUX 1:4:

$$Y_0 = D\bar{S}_1\bar{S}_0, \quad Y_1 = D\bar{S}_1S_0, \quad Y_2 = DS_1\bar{S}_0, \quad Y_3 = DS_1S_0$$



Teste seus conhecimentos

Meio Somador e Somador Completo

- Problema motivador: seleção + soma no processador
- Aritmética binária e o circuito XOR
- Meio Somador (Half Adder)
- Somador Completo (Full Adder)
- Construção do FA a partir de HAs

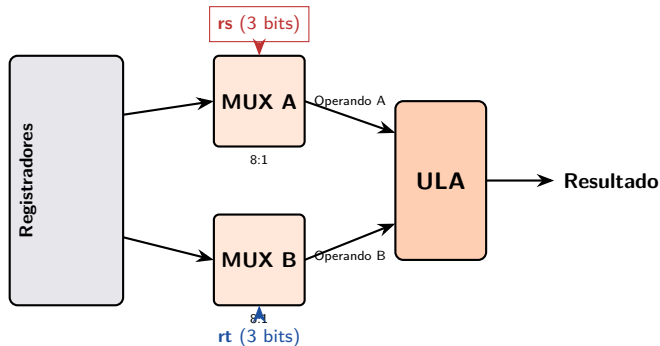
Cenário

Você está projetando um processador com **8 registradores**. A ULA precisa receber dados de **2 registradores quaisquer** e realizar uma soma.

Pergunta: Como selecionar os registradores e somar os valores?

Discussão em grupo:

- 1 Que circuito faz a "seleção"? (*Aula: MUX!*)
- 2 Agora, como construir o circuito que faz a "soma"?
- 3 É possível somar com portas lógicas básicas?



Já temos a seleção (MUX). Agora falta a soma (ULA)!

Soma de bits individuais:

$$\begin{array}{lll} 0 + 0 = 0 & 0 + 1 = 1 & 1 + 0 = 1 \\ 1 + 1 = \mathbf{10} & (\text{resultado} = 0, \text{ carry} = 1) & \end{array}$$

Observação crucial

- A **soma** de 1 bit (sem carry) é idêntica à operação **XOR**
- O **carry** gerado é idêntico à operação **AND**

Isso nos leva diretamente ao **Meio Somador!**

Meio Somador (Half Adder)

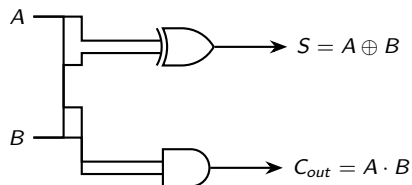
Definição

Soma dois bits (A , B):

$$S = A \oplus B$$

$$C_{out} = A \cdot B$$

A	B	S	C_{out}
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1



Limitação

Não possui entrada de carry-in!

Serve apenas para o bit menos significativo.

Por que o Meio Somador não é suficiente?

Exemplo: Somar $0110_2 + 0111_2$ ($6 + 7 = 13$)

$$\begin{array}{rcccc} & C_3 & C_2 & C_1 & \\ & 0 & 1 & 1 & 0 \\ + & 0 & 1 & 1 & 1 \\ \hline & 1 & 1 & 0 & 1 \end{array}$$

- **Bit 0 (LSB):** $0 + 1 = 1$ ✓ Meio Somador funciona
- **Bit 1:** $1 + 1 + C_1 = ?$ × Precisa de carry-in!
- **Bit 2:** $1 + 1 + C_2 = ?$ × Precisa de carry-in!

Precisamos de um circuito com **3 entradas:** A , B e C_{in} .

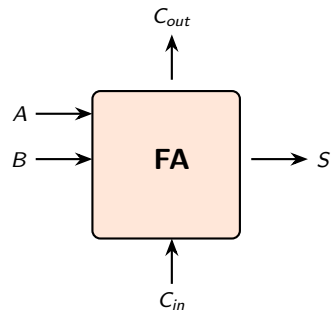
Somador Completo (Full Adder)

Definição

Soma três bits: A , B e C_{in}

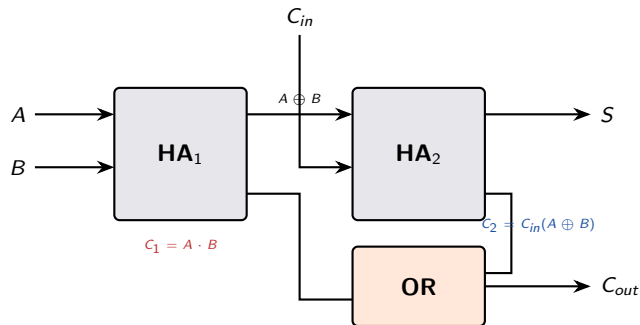
$$S = A \oplus B \oplus C_{in}$$

$$C_{out} = (A \cdot B) + (C_{in} \cdot (A \oplus B))$$



A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Full Adder = 2 Half Adders + 1 OR



$$C_{out} = \underbrace{A \cdot B}_{C_1} + \underbrace{C_{in} \cdot (A \oplus B)}_{C_2}$$

Composição modular

Full Adder = 2 Half Adders + 1 OR. O poder da reutilização em hardware!

Retomando o problema original

Agora sabemos que:

MUX → seleciona operandos dos registradores

Somador → realiza a operação na ULA

Tarefa (em grupo): desenhe o caminho completo de uma instrução `add $t0, $t1, $t2`

- 1 Diagrama de blocos: Registradores → MUX → ULA → Reg. destino
- 2 Identifique os sinais de controle
- 3 Onde entra o DEMUX nesta arquitetura?

Dica: O DEMUX pode selecionar em qual registrador o resultado será escrito!

Somadores de n bits – Laboratório Prático

Metodologia: Laboratório Hands-On (Logisim)

- Construção de HA no Logisim
- Ripple Carry Adder de 4 bits
- Análise de atraso e caminho crítico
- Introdução ao Carry Lookahead Adder

Parte 1 – Construção:

- 1 Construir **Meio Somador** (XOR + AND)
- 2 Construir **Somador Completo** (2 HAs + OR)
- 3 Encapsular FA como subcircuito
- 4 Conectar 4 FAs → **RCA 4 bits**

Parte 2 – Análise:

- 1 Testar com vetores fornecidos
- 2 Medir atraso de propagação
- 3 Documentar com screenshots
- 4 Responder: qual o atraso para n bits?

Material

Logisim Evolution instalado. Arquivo-base no Google Classroom com template do projeto.

Componentes:

- 1 porta XOR (2 entradas) → gera S
- 1 porta AND (2 entradas) → gera C_{out}
- 2 pinos de entrada, 2 pinos de saída

Procedimento:

- 1 Novo subcircuito: HalfAdder
- 2 Posicione portas e conecte
- 3 Teste as 4 combinações: 00, 01, 10, 11
- 4 Confira com a tabela verdade

Dica

Use Projeto → Analisar Circuito para gerar a tabela verdade automaticamente e confirmar suas equações!

Abordagem 1 – Usando subcircuitos HA:

- 1 Novo subcircuito: FullAdder
- 2 Insira 2 instâncias do HalfAdder
- 3 Adicione 1 porta OR
- 4 Conecte: $A, B \rightarrow HA_1$ $S_1, C_{in} \rightarrow HA_2$ $C_1, C_2 \rightarrow OR$

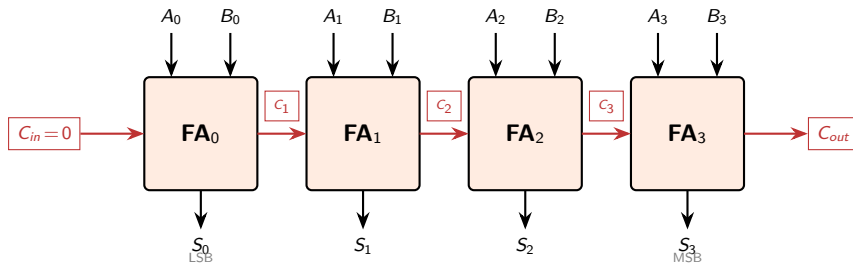
Abordagem 2 – Direto com portas:

- 2 XOR + 2 AND + 1 OR
- Ambas devem produzir resultado idêntico!

Teste obrigatório

Todas as **8 combinações!** Atenção ao caso $A=1, B=1, C_{in}=1 \Rightarrow S=1, C_{out}=1$ (ou seja, $1 + 1 + 1 = 11_2$).

Ripple Carry Adder de 4 bits



O carry "percorre" cada FA da direita para a esquerda (*ripple*).

Teste	A	B	S	C_{out}
1	0000	0000	0000	0
2	0101	0011	1000	0
3	0111	0001	1000	0
4	1111	0001	0000	1
5	1010	0101	1111	0
6	1111	1111	1110	1

Teste 4 ($15 + 1 = 16$): resultado "estoura" 4 bits $\rightarrow C_{out} = 1$

Teste 6 ($15 + 15 = 30$): $11110_2 = 30_{10}$ ✓

Modelo de atraso

Se cada Full Adder tem atraso Δt para propagar o carry:

$$T_{total} = n \cdot \Delta t_{carry}$$

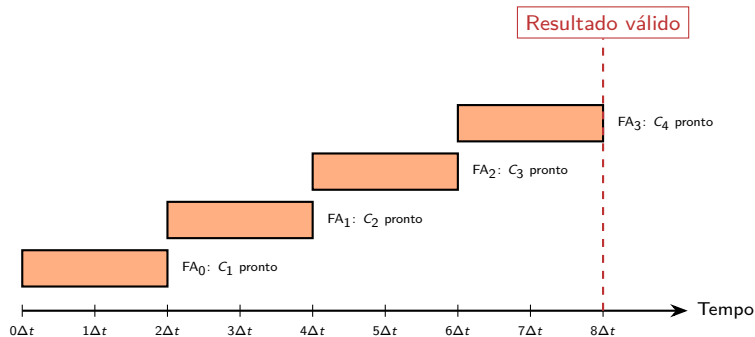
Largura	$n = 4$	$n = 16$	$n = 32$
Atraso ($\Delta t = 2$ ns)	8 ns	32 ns	64 ns
Freq. máxima	125 MHz	31 MHz	15,6 MHz

Problema

Para 32 bits (padrão MIPS), o RCA é **muito lento**!

Processadores modernos operam em GHz – precisamos de algo melhor.

O Problema do Ripple – Visualizando



Cada FA precisa **esperar** o carry do anterior. O atraso é **sequencial**!

Ideia central: Calcular *todos* os carries **simultaneamente**, sem esperar propagação.

Definições para cada bit i :

$$G_i = A_i \cdot B_i \quad (\textbf{Generate:} \text{ gera carry independente de } C_{in})$$

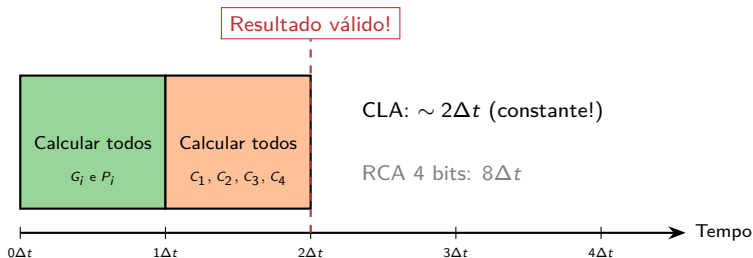
$$P_i = A_i \oplus B_i \quad (\textbf{Propagate:} \text{ propaga carry se } C_{in} = 1)$$

Carries calculados diretamente:

$$C_1 = G_0 + P_0 \cdot C_0$$

$$C_2 = G_1 + P_1 \cdot G_0 + P_1 \cdot P_0 \cdot C_0$$

$$C_3 = G_2 + P_2 \cdot G_1 + P_2 \cdot P_1 \cdot G_0 + P_2 \cdot P_1 \cdot P_0 \cdot C_0$$



Vantagem

O CLA calcula carries em $O(1)$ – apenas 2 níveis de portas.
Na prática (CLA hierárquico): $O(\log n)$.

Característica	RCA	CLA
Atraso de carry	$O(n)$	$O(\log n)$
Complexidade	Baixa	Alta
Número de portas	$\sim 5n$	$\sim n^2$
Facilidade de projeto	Simples	Complexo
Uso em CPUs reais	Raro	Comum
Consumo de energia	Menor	Maior

Na prática

CPUs usam **CLA hierárquico**: grupos de 4 bits com CLA interno e CLA entre grupos. Melhor compromisso velocidade vs complexidade.

Por grupo – enviar até o final da aula

① Arquivo .circ contendo:

- Subcircuito HalfAdder
- Subcircuito FullAdder
- Circuito principal: RCA de 4 bits

② Tabela de testes: 8+ vetores com resultado esperado vs obtido

③ Respostas:

- Atraso máximo do RCA de 4 bits (em atrasos de porta)?
- Se cada porta = 1 ns, qual a frequência máxima?

Revisão Gamificada + Desafio Final

- Fase 1: Quiz no Wayground
- Fase 4: Apresentação + Ranking
- Fase 5: Orientação do projeto avaliativo



Teste seus conhecimentos

Ideia

$$A - B = A + (\overline{B} + 1) = A + \overline{B} \text{ com } C_{in} = 1$$

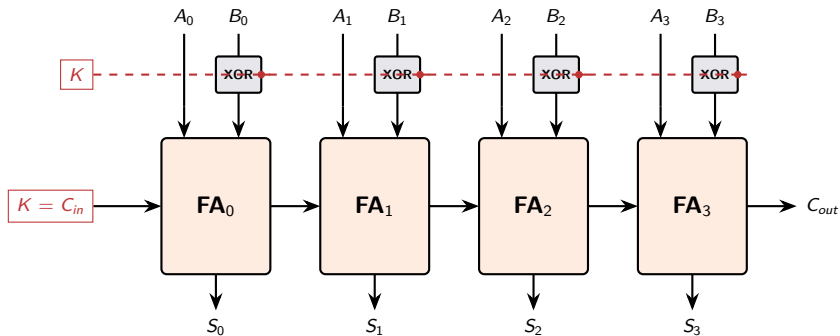
Truque com XOR:

Operação	Controle (K)
Soma ($A + B$)	$K = 0: B_i \oplus 0 = B_i, \quad C_{in} = 0$
Subtração ($A - B$)	$K = 1: B_i \oplus 1 = \overline{B_i}, \quad C_{in} = 1$

Conexão com o MIPS

É exatamente assim que a ULA do MIPS implementa a instrução sub: mesmo hardware do add, invertendo B e setando carry-in!

Somador/Subtrator – Circuito Unificado



$K = 0 \rightarrow$ Soma

$K = 1 \rightarrow$ Subtração

Quando ocorre overflow (complemento de 2)?

- Positivo + Positivo = Negativo $\Rightarrow$ overflow
- Negativo + Negativo = Positivo $\Rightarrow$ overflow

Detecção:

$$\text{Overflow} = C_{n-1} \oplus C_n$$

C_{n-1}	C_n	Overflow?	Situação
0	0	Não	Resultado positivo correto
0	1	Sim	Positivos geraram negativo
1	0	Sim	Negativos geraram positivo
1	1	Não	Resultado negativo correto

Caso 1: Positivo + Positivo (com overflow)

		C_2	C_1		
	0	1	1	1	(= +7)
+	0	0	0	1	(= +1)
	1	0	0	0	(= -8!?)

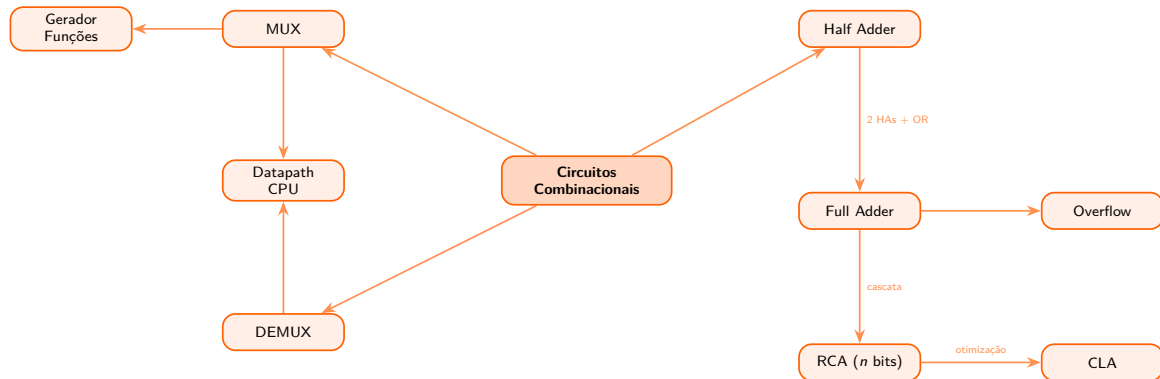
$C_2 = 1, C_3 = 0 \Rightarrow C_2 \oplus C_3 = 1 \Rightarrow$ **Overflow!**

Caso 2: Positivo + Negativo (sem overflow)

		C_2	C_1		
	0	1	0	1	(= +5)
+	1	1	0	1	(= -3)
	0	0	1	0	(= +2 ✓)

$C_2 = 1, C_3 = 1 \Rightarrow C_2 \oplus C_3 = 0 \Rightarrow$ Sem overflow.

Mapa Conceitual do Módulo



Latches e Flip-Flops

- Circuitos combinacionais vs sequenciais
- Latches SR, D e JK
- Flip-flops edge-triggered
- Diagramas de estado e tabelas de excitação
- Simulação no Logisim

Combinacional (Módulo 1):

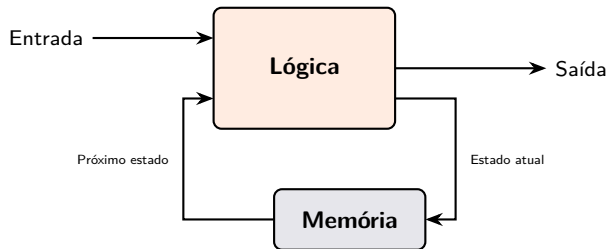
- Saída depende **apenas** das entradas atuais
- Sem memória
- Exemplos: MUX, DEMUX, somadores

Sequencial (este módulo):

- Saída depende das entradas **e do estado atual**
- Possui **memória** (realimentação)
- Exemplos: latches, flip-flops, contadores

Por que isso importa?

Sem circuitos sequenciais, não existem registradores, nem memória, nem unidades de controle. O processador precisa **lembrar** onde está na execução de um programa.



A **realimentação** (feedback) é o que permite ao circuito "lembrar".
O elemento de memória mais simples é o **Latch SR**.

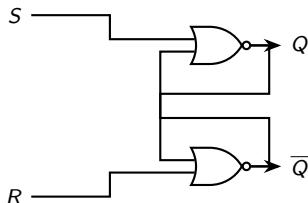
Think – O que é um Latch?

Reflexão individual

Observe o circuito abaixo formado por duas portas NOR com realimentação cruzada.

Perguntas:

- 1 Se $S = 1$ e $R = 0$, qual o valor de Q ?
- 2 Se depois fizermos $S = 0$ e $R = 0$, o que acontece com Q ?
- 3 O circuito "lembra" o estado anterior?



Compartilhem suas respostas com o colega ao lado:

- ❶ Vocês concordam sobre o que acontece quando $S = 1, R = 0$?
- ❷ E quando ambos voltam para 0?
- ❸ O que acontece no caso proibido $S = 1, R = 1$?
- ❹ Conseguem preencher a tabela juntos?

S	R	Q	Ação
0	0	?	
0	1	?	
1	0	?	
1	1	?	

S	R	Q_{next}	Ação
0	0	Q_{ant}	Mantém estado anterior
0	1	0	Reset (limpa)
1	0	1	Set (armazena 1)
1	1	?	Proibido (ambos = 0, indeterminado)

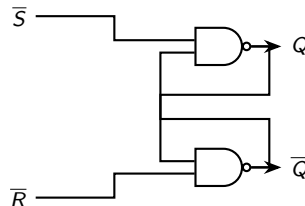
Problema do estado proibido

Quando $S = R = 1$, ambas saídas vão para 0 ($Q = \overline{Q} = 0$).

Ao retornar para $S = R = 0$, o resultado depende de **qual porta é mais rápida** – comportamento **imprevisível** (condição de corrida).

Versão com portas NAND (entradas ativas em nível baixo):

$\bar{S}$	$\bar{R}$	Q_{next}	Ação
1	1	Q_{ant}	Mantém
1	0	0	Reset
0	1	1	Set
0	0	?	Proibido



Nota: A lógica é invertida em relação ao SR-NOR. Entradas ativas em 0.

Ideia

Forçar que $R = \overline{S}$ usando um inversor.

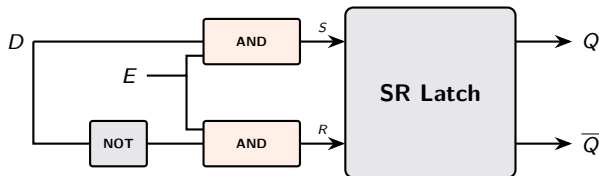
Assim, nunca teremos $S = R$ simultaneamente!

Uma entrada de **Enable** (E) controla *quando* o latch

	E	D	Q_{next}
captura o dado.	0	X	Q_{ant} (mantém)
	1	0	0
	1	1	1

Continua...

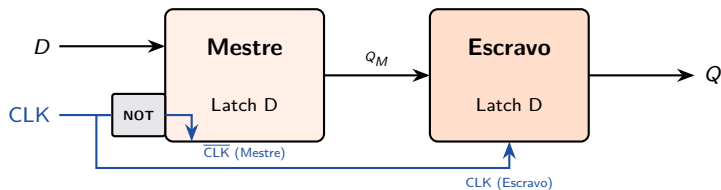
Latch D – Eliminando o Estado Proibido



Característica	Latch	Flip-Flop
Sensibilidade	Nível	Borda
Transparência	Sim ($E=1$)	Não
Captura	Contínua	Na borda do clock
Uso em CPU	Raro	Muito comum
Complexidade	Menor	Maior (2 latches)

Flip-Flop D

Construído com dois Latches D em configuração **mestre-escravo**. Captura o dado apenas na **borda** do clock.



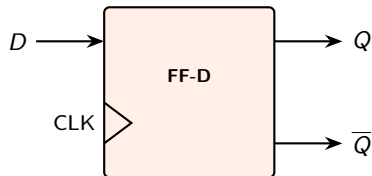
CLK = 0:

- Mestre: transparente (captura D)
- Escravo: travado

CLK = 0 $\rightarrow$ 1 (borda de subida):

- Mestre: trava
- Escravo: transparente (copia Q_M)

Flip-Flop D – Símbolo e Tabela



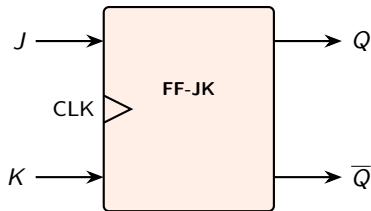
O **triângulo** indica sensibilidade à **borda**.

CLK	D	Q_{next}
↑	0	0
↑	1	1
outro	X	Q_{ant}

Equação característica:

$$Q_{next} = D \quad (\text{na borda } \uparrow)$$

O FF-D simplesmente "fotografa" a entrada na borda do clock.



Equação:

$$Q_{next} = J \cdot \bar{Q} + \bar{K} \cdot Q$$

J	K	Q_{next}	Ação
0	0	Q_{ant}	Mantém
0	1	0	Reset
1	0	1	Set
1	1	$\bar{Q}_{ant}$	Toggle

Vantagem sobre o SR

$J = K = 1$ não é proibido – ele faz **toggle** (inverte o estado)!
Isso é extremamente útil para construir **contadores**.

Tabela de Excitação dos Flip-Flops

A tabela de excitação responde: *que entradas preciso para obter uma transição?*

FF-D:

Q	Q_{next}	D
0	0	0
0	1	1
1	0	0
1	1	1

$D = Q_{next}$
(trivial!)

FF-JK:

Q	Q_{next}	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

Don't-cares facilitam minimização.

FF-T (Toggle):

Q	Q_{next}	T
0	0	0
0	1	1
1	0	1
1	1	0

$T = Q \oplus Q_{next}$

Utilidade

Tabelas de excitação são essenciais para **projetar contadores e FSMs**.

O que aprendemos:

- Combinacional vs Sequencial
- Latch SR (NOR e NAND)
- Latch D (sem estado proibido)
- Transparência vs borda
- Flip-Flop D mestre-escravo
- Flip-Flop JK com toggle
- Tabelas de excitação

Para a próxima aula:

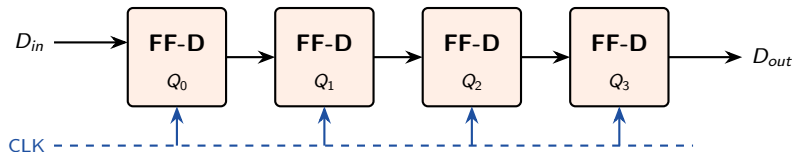
- Pesquisar: o que é um registrador de deslocamento?
- Pesquisar: como funciona um contador binário?
- Pensar: como usar flip-flops para "contar" de 0 a 9?

Registradores de Deslocamento e Contadores

- Registradores de deslocamento (SISO, SIPO, PISO, PIPO)
- Contadores assíncronos e síncronos
- Projeto: contador 0–9 com display 7 segmentos

Definição

Um **registrador de deslocamento** é um conjunto de flip-flops conectados em cadeia, onde o dado "desliza" de um flip-flop para o próximo a cada borda de clock.



A cada borda de clock: $Q_3 \leftarrow Q_2 \leftarrow Q_1 \leftarrow Q_0 \leftarrow D_{in}$

Tipo	Entrada	Saída	Aplicação
SISO	Serial	Serial	Linha de atraso
SIPO	Serial	Paralela	Conversão serial → paralelo
PISO	Paralela	Serial	Conversão paralelo → serial
PIPO	Paralela	Paralela	Buffer, transferência

Aplicações reais

- **SIPO:** Recepção de dados UART (serial → byte)
- **PISO:** Transmissão SPI (byte → serial)
- **PIPO:** Banco de registradores do processador

Exemplo: SIPO em Ação

Entrada serial: $D_{in} = 1, 0, 1, 1$ (bit a bit, a cada clock)

Clock	Q_3	Q_2	Q_1	Q_0	Observação
Inicial	0	0	0	0	Limpo
1	0	0	0	1	Entra 1
2	0	0	1	0	Entra 0, desloca
3	0	1	0	1	Entra 1, desloca
4	1	0	1	1	Entra 1, desloca

Após 4 clocks: saída paralela = 1011_2 ✓

O dado serial foi "montado" em formato paralelo!

Definição

Um **contador** é um circuito sequencial que percorre uma sequência predefinida de estados a cada pulso de clock.

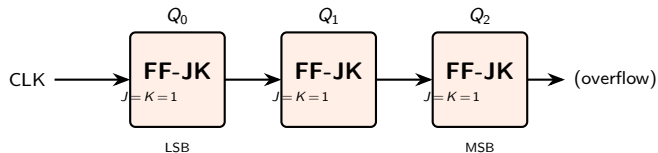
Contador Assíncrono (Ripple):

- Clock aplicado apenas ao primeiro FF
- Cada FF usa a saída do anterior como clock
- Simples, mas com **atraso acumulado**
- Análogo ao Ripple Carry Adder!

Contador Síncrono:

- Todos os FFs recebem o **mesmo clock**
- Transições simultâneas
- Mais rápido e confiável
- Usado em processadores

Contador Assíncrono de 3 bits



Sequência de contagem:

Clock	Q_2	Q_1	Q_0	Decimal
0	0	0	0	0
1	0	0	1	1
2	0	1	0	2
3	0	1	1	3
4	1	0	0	4
5	1	0	1	5
6	1	1	0	6
7	1	1	1	7

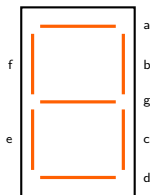
Usando FF-JK: Todos os FFs recebem o mesmo clock.

Precisamos definir J_i e K_i para cada flip-flop.

Q_2	Q_1	Q_0	J_0	K_0	J_1	K_1	J_2	K_2
0	0	0	1	1	0	X	0	X
0	0	1	1	1	1	X	0	X
0	1	0	1	1	X	0	0	X
0	1	1	1	1	X	1	1	X
1	0	0	1	1	0	X	X	0
1	0	1	1	1	1	X	X	0
1	1	0	1	1	X	0	X	0
1	1	1	1	1	X	1	X	1

Simplificando: $J_0 = K_0 = 1$, $J_1 = K_1 = Q_0$, $J_2 = K_2 = Q_1 \cdot Q_0$

Decodificador BCD para 7 Segmentos



Cada segmento ($a-g$) é uma função booleana de $Q_3Q_2Q_1Q_0$.

Q_3	Q_2	Q_1	Q_0	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1

- ❶ **Comece pelo contador:** 4 FF-JK com lógica combinacional para J_i, K_i
- ❷ **Deteção do 10:** Use uma porta AND com Q_3 e Q_1
- ❸ **Decodificador:** Logisim já tem componente pronto de display 7 segmentos (Input/Output → Hex Digit Display), mas tente construir o seu!
- ❹ **Teste passo a passo:** Use botão (não clock automático) primeiro
- ❺ **Depois:** Conecte clock automático e observe a contagem fluida

Subcircuitos sugeridos

- ContadorBCD – o contador síncrono 0–9
- Decodificador7Seg – BCD para segmentos
- Main – integração com display

O que aprendemos:

- Registradores de deslocamento (SISO/SIPO/PISO/PIPO)
- Contadores assíncronos vs síncronos
- Projeto de contador síncrono com FF-JK
- Reset automático (BCD 0–9)
- Decodificador BCD para 7 segmentos

Para a próxima aula:

- Pesquisar: como funciona um semáforo de trânsito?
- Quais estados um semáforo tem?
- O que determina a mudança de estado?
- Pensar: isso é um contador ou algo mais geral?

Questão para reflexão

Analise as opções abaixo e identifique qual **NÃO** é uma aplicação típica de registradores de deslocamento:

- ❶ Conversão de dados paralelos para seriais (ex.: interface UART)
- ❷ Multiplicação de inteiros por potências de 2
- ❸ **Decodificação de endereços de memória em sistemas com múltiplos bancos**
- ❹ Implementação de buffers de pipeline (FIFO simples)

Dica

Revise: quais operações são realizadas **por deslocamento de bits**? Qual dessas funções pertence a outro tipo de circuito já estudado nesta aula?

Máquinas de Estados Finitos (FSM)

- Conceito de FSM: estados, transições, saídas
- Máquinas de Moore vs Mealy
- Estudo de caso: controlador de semáforo

O que é uma Máquina de Estados Finitos?

Definição

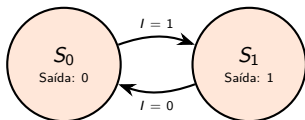
Uma Finite State Machine (**FSM**) é um modelo matemático de computação com um número finito de estados. A cada pulso de clock, a máquina pode transitar de um estado para outro, dependendo das entradas.

Componentes de uma FSM:

- **Estados (S):** conjunto finito de situações possíveis
- **Entradas (I):** sinais externos que influenciam transições
- **Saídas (O):** sinais produzidos pela máquina
- **Função de transição (δ):** $S \times I \rightarrow S$
- **Função de saída (λ):** depende do tipo (Moore ou Mealy)
- **Estado inicial (s_0):** ponto de partida

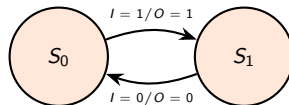
Moore:

- Saída depende **apenas do estado atual**
- $\lambda : S \rightarrow O$
- Saídas mudam **com** a transição de estado
- Mais previsível e segura



Mealy:

- Saída depende do **estado e da entrada**
- $\lambda : S \times I \rightarrow O$
- Saídas podem mudar **entre** transições
- Resposta mais rápida, menos estados



Na prática

A unidade de controle do processador MIPS (multiciclo) é uma FSM de Moore!

Cenário

Você precisa projetar o controlador de um cruzamento com 2 semáforos (Norte-Sul e Leste-Oeste). Cada semáforo tem 3 estados: Verde, Amarelo, Vermelho.

Pergunta socrática: Quantos estados a FSM precisa ter?

Discussão guiada:

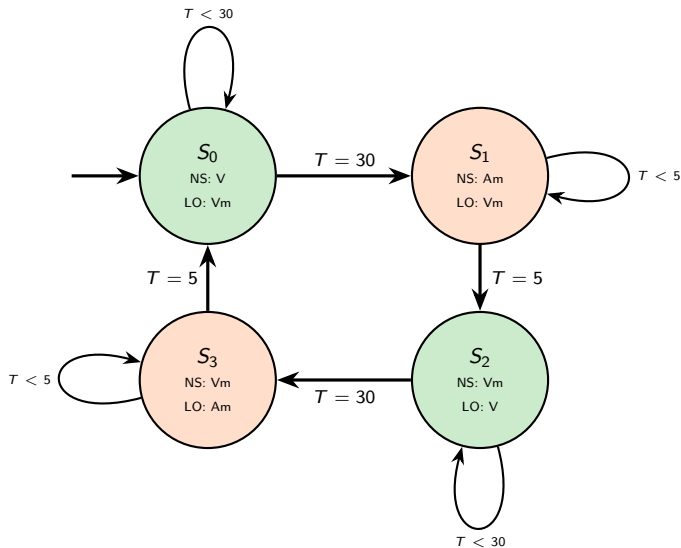
- 1 Os dois semáforos podem estar verdes ao mesmo tempo?
- 2 Qual a sequência segura de transição?
- 3 É necessário um estado de "todos vermelhos" por segurança?
- 4 Como representar a temporização (ex.: verde = 30s)?

Solução com 4 estados (FSM de Moore):

Estado	N-S	L-O	Duração
S_0	Verde	Vermelho	30 clocks
S_1	Amarelo	Vermelho	5 clocks
S_2	Vermelho	Verde	30 clocks
S_3	Vermelho	Amarelo	5 clocks

Codificação: $S_0 = 00$, $S_1 = 01$, $S_2 = 10$, $S_3 = 11$

Precisamos de **2 flip-flops** (Q_1, Q_0) + **1 contador** de tempo.



T = contador de tempo (decrementa a cada clock). Transição quando T atinge o valor alvo.

Q_1	Q_0	Timeout?	Q'_1	Q'_0	N-S			L-O		
					V	Am	Vm	V	Am	Vm
0	0	Não	0	0	1	0	0	0	0	1
0	0	Sim	0	1	1	0	0	0	0	1
0	1	Não	0	1	0	1	0	0	0	1
0	1	Sim	1	0	0	1	0	0	0	1
1	0	Não	1	0	0	0	1	1	0	0
1	0	Sim	1	1	0	0	1	1	0	0
1	1	Não	1	1	0	0	1	0	1	0
1	1	Sim	0	0	0	0	1	0	1	0

Próximo estado: $Q'_1 Q'_0 = Q_1 Q_0 + 1$ (contador de 2 bits!) quando timeout.

Saídas: Decodificação combinacional direta de Q_1, Q_0 .

Transição (com entrada $T = \text{timeout}$):

$$Q'_0 = Q_0 \oplus T$$

$$Q'_1 = Q_1 \oplus (Q_0 \cdot T)$$

Saídas (Moore – dependem apenas do estado):

$$\text{NS_Verde} = \overline{Q_1} \cdot \overline{Q_0}$$

$$\text{NS_Amarelo} = \overline{Q_1} \cdot Q_0$$

$$\text{NS_Vermelho} = Q_1$$

$$\text{LO_Verde} = Q_1 \cdot \overline{Q_0}$$

$$\text{LO_Amarelo} = Q_1 \cdot Q_0$$

$$\text{LO_Vermelho} = \overline{Q_1}$$

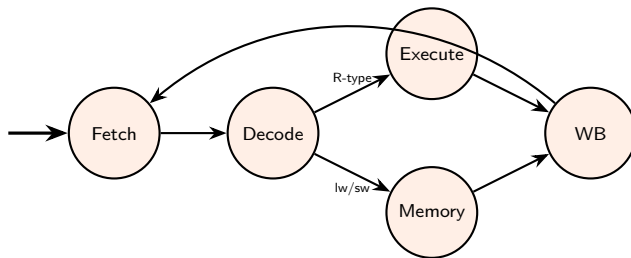
Após a implementação, discutam:

- ❶ **Segurança:** O que acontece se houver um erro no circuito e dois semáforos ficarem verdes ao mesmo tempo? Como prevenir isso em hardware?
- ❷ **Escalabilidade:** Se adicionarmos um terceiro semáforo (pedestre), quantos estados a FSM precisa ter?
- ❸ **Moore vs Mealy:** Seria vantajoso usar Mealy neste caso? Quando Mealy seria melhor?
- ❹ **Conexão com processadores:** A unidade de controle do MIPS multiciclo é uma FSM. O que são os "estados" nesse contexto?
- ❺ **Generalização:** Todo circuito sequencial pode ser modelado como uma FSM?

Prévia do Módulo 6

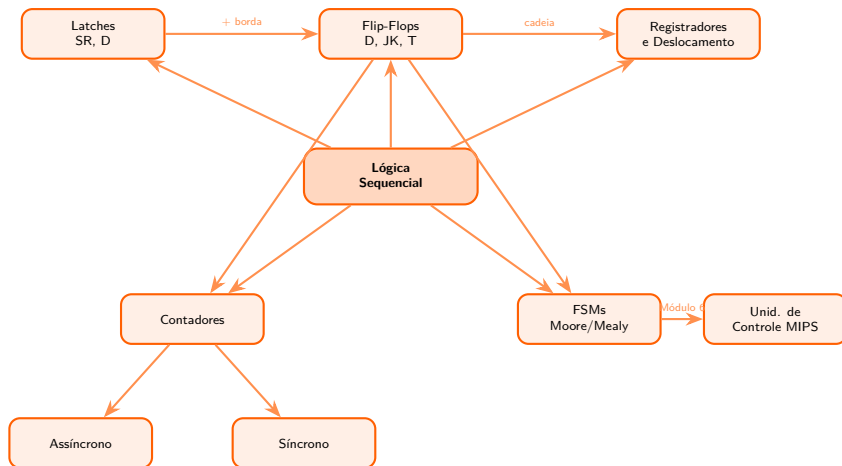
A unidade de controle do processador MIPS multiciclo é uma FSM de Moore com:

- **Estados:** Fetch, Decode, Execute, Memory, WriteBack
- **Entrada:** Opcode da instrução (6 bits)
- **Saídas:** Sinais de controle (RegWrite, MemRead, ALUSrc, etc.)



Tudo o que vimos neste módulo é a base para entender o controle do processador!

Mapa Conceitual do Módulo



Questão para reflexão

Uma FSM do tipo **Moore** possui 3 estados com as seguintes saídas:

Estado	Saída (λ)
A	0
B	1
C	0

A sequência de estados percorrida é: $A \rightarrow B \rightarrow B \rightarrow C \rightarrow A$

Qual é a **sequência de saídas** gerada?

- 1 0, 1, 1, 0, 0 b. 1, 1, 0, 0, 1
2 0, 1, 0, 1, 0 d. 0, 0, 1, 1, 0

Dica

Em Moore, a saída depende **apenas do estado atual**, não da entrada.

Componentes e Organização do Computador

- O conceito de programa armazenado
- Arquiteturas Von Neumann e Harvard
- Componentes: ULA, UC, Registradores, Memória, Barramento
- Ciclo de instrução: Fetch, Decode, Execute
- Atividade Jigsaw: cada aluno vira especialista

Como um computador executa um programa?

- Você escreve código em Python ou C
- O compilador traduz para linguagem de máquina
- O processador executa instrução por instrução

Mas **como** o processador sabe o que fazer?
Onde ficam as instruções? Quem as decodifica? Quem faz as contas?

Objetivo desta aula

Entender a **organização interna** de um computador e como seus componentes colaboram para executar programas.

Máquinas de propósito fixo:

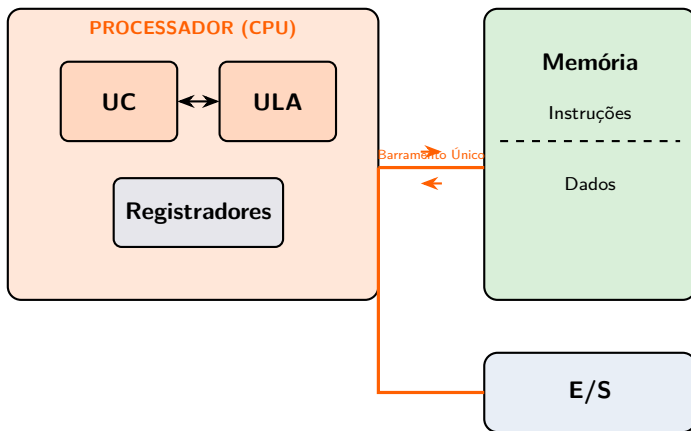
- ENIAC (1945): programado por **recabeamento físico**
- Para mudar o programa, era necessário reconectar fios manualmente
- Processo levava **dias** para reconfigurar

A revolução de Von Neumann:

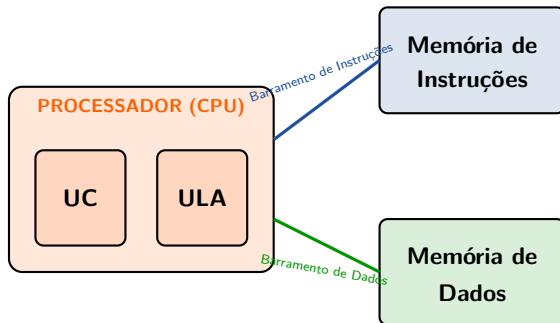
- E se as instruções ficassem **na mesma memória** que os dados?
- O programa seria **carregado** na memória como dados
- Para mudar o programa, basta **reescrever a memória**

Conceito de Programa Armazenado (1945)

Instruções e dados compartilham a mesma memória. O computador busca instruções da memória, decodifica e executa – automaticamente, sem intervenção humana.



Característica central: memória única para instruções e dados, acessada por um **barramento compartilhado**.



Diferença: memórias **separadas** para instruções e dados, com barramentos independentes $\Rightarrow$ acesso **simultâneo**.

Característica	Von Neumann	Harvard
Memória	Única	Separadas
Barramento	Compartilhado	Independentes
Acesso simultâneo	Não	Sim
Gargalo	Barramento	Menor
Flexibilidade	Maior	Menor
Complexidade	Menor	Maior
Uso típico	PCs/Servidores	DSPs/MCUs

Harvard Modificada

CPUs modernas usam modelo híbrido:

L1 separadas (instr./dados) + **RAM unificada**.



O problema:

- A CPU processa dados muito mais rápido do que a memória os fornece
- Instruções e dados competem pelo **mesmo barramento**
- A CPU fica **ociosa** esperando a memória

Soluções (veremos nos próximos módulos): Cache, pipeline, prefetching.

Questão para reflexão

Qual característica da Arquitetura de Von Neumann origina o chamado "**gargalo de Von Neumann**" e qual técnica moderna o atenua?

- ❶ Uso de registradores de propósito geral, atenuado por mais registradores físicos.
- ❷ **Barramento único compartilhado entre instruções e dados, atenuado por hierarquia de cache.**
- ❸ Execução sequencial de instruções, atenuada por múltiplos núcleos.
- ❹ Tamanho fixo de palavra, atenuado por arquiteturas de 64 bits.

Dica

Observe o diagrama acima: CPU e memória compartilham o **mesmo barramento**. O que acontece quando instruções e dados precisam trafegar simultaneamente?

Pergunta

Se um processador tem barramento de endereços de **32 bits**, qual a quantidade máxima de memória que ele pode endereçar?

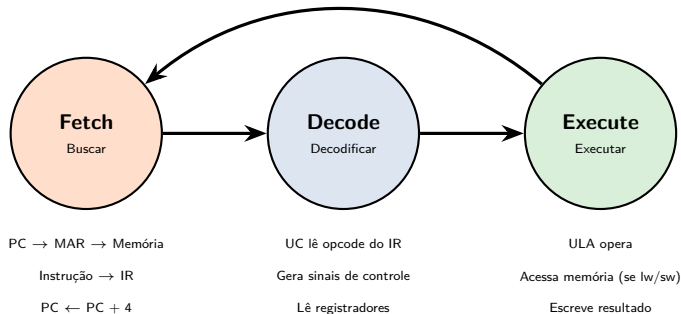
Resposta:

$$2^{32} = 4.294.967.296 \text{ bytes} = 4 \text{ GB}$$

É por isso que...

Sistemas operacionais de **32 bits** não conseguem usar mais de 4 GB de RAM!
Para superar esse limite: processadores e SOs de **64 bits** ($2^{64} = 16$ exabytes).

Ciclo de Instrução – Fetch, Decode, Execute



O processador repete este ciclo **bilhões de vezes por segundo**.

Exemplo: Execução de `add $t0, $t1, $t2` no MIPS

1. Fetch:

- PC envia endereço para a memória de instruções
- Memória retorna a instrução codificada em 32 bits
- Instrução é armazenada no IR; PC incrementa (+4)

2. Decode:

- UC extrai opcode (000000 = R-type) e funct (100000 = add)
- Banco de registradores lê \$t1 e \$t2
- UC gera sinais: `RegWrite=1`, `ALUSrc=0`, `ALUOp=add`

3. Execute:

- ULA soma os valores de \$t1 e \$t2
- Resultado é escrito em \$t0

```
1 .text
2 .globl main
3
4 main:
5     addi $t1, $zero, 5      # busca, decodifica e executa
6     addi $t2, $zero, 3      # busca, decodifica e executa
7     add  $t0, $t1, $t2      # soma os registradores
8
9     move $a0, $t0           # copia valor para argumento
10    li $v0, 1                # prepara impressao
11    syscall                  # executa a impressao
12
13    li $v0, 10                # prepara encerramento
14    syscall                  # finaliza programa
```

O que aprendemos:

- Programa armazenado
- Von Neumann vs Harvard
- Gargalo de Von Neumann
- 5 componentes: ULA, UC, Registradores, Memória, Barramento
- Ciclo Fetch-Decode-Execute

Para a próxima aula:

- Pesquisar a história de um marco: ENIAC, UNIVAC, IBM 360, Intel 4004, Intel 8086, ARM, RISC-V
- Pesquisar: o que é CISC e RISC?
- Preparar argumento para o debate

Questão para reflexão

Descreva o ciclo **Fetch-Decode-Execute** para a instrução MIPS:

```
add $t0, $t1, $t2
```

Para **cada etapa**, indique:

- Quais componentes do datapath são ativados (PC, Memória de Instruções, Banco de Registradores, ALU)
- Quais sinais de controle são relevantes (RegDst, ALUSrc, ALUOp, RegWrite, etc.)

Dica

add é instrução tipo R (opcode = 000000). No estágio **Fetch**: PC é lido e incrementado. No **Decode**: banco lê rs e rt. No **Execute**: ALU soma, RegWrite=1 salva em rd.

Questão para reflexão

Considere dois sistemas:

- (A) Microprocessador de propósito geral executando um sistema operacional
- (B) DSP embarcado que processa áudio em tempo real

Justifique qual arquitetura (**Von Neumann** ou **Harvard**) é mais adequada para cada caso, citando as **limitações da outra**.

Dica

Pense: qual sistema precisa de **flexibilidade** para carregar programas dinamicamente? Qual precisa buscar instrução e dado **ao mesmo tempo** para cumprir prazos rígidos de tempo real?

Evolução Histórica e Taxonomia de Flynn

- Evolução: do ENIAC aos processadores modernos
- Gerações de computadores
- Taxonomia de Flynn: SISD, SIMD, MISD, MIMD
- Debate: "Von Neumann ainda é relevante?"

Geração	Período	Tecnologia	Exemplo
1 ^a	1940–1956	Válvulas	ENIAC, UNIVAC
2 ^a	1956–1963	Transistores	IBM 7090, PDP-1
3 ^a	1964–1971	Circuitos Integrados	IBM 360, PDP-11
4 ^a	1971–hoje	Microprocessadores	Intel 4004, 8086, ARM
5 ^a	Futuro	IA, Quântica	GPUs, TPUs, QPUs

Tendência geral

A cada geração: **menor, mais rápido, mais barato, menor consumo de energia** ⇒ Lei de Moore.

Dinâmica (30 min)

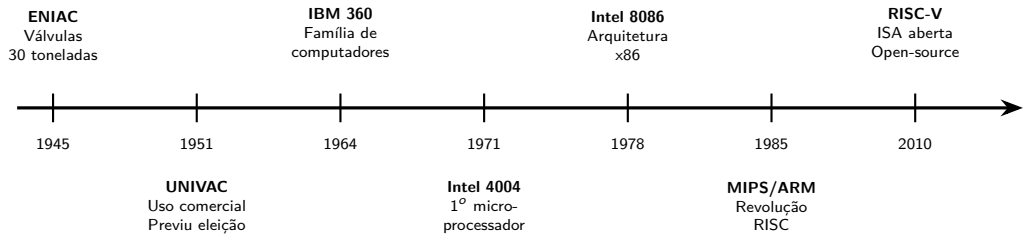
Cada grupo recebeu previamente um marco histórico para pesquisar.

Agora, apresentem em **3 minutos**:

- ❶ Que máquina/processador é?
- ❷ Que problema resolveu?
- ❸ Qual a inovação arquitetural?
- ❹ Impacto na computação moderna

Marcos sugeridos:

- ENIAC (1945)
- UNIVAC I (1951)
- IBM System/360 (1964)
- Intel 4004 (1971)
- Intel 8086 (1978)
- MIPS R2000 (1985)
- ARM (1985)
- RISC-V (2010)



Gordon Moore (1965)

"O número de transistores em um chip dobra aproximadamente a cada 2 anos, com custo aproximadamente constante."

Implicações:

- Intel 4004 (1971): **2.300** transistores
- Intel Pentium (1993): **3,1 milhões** de transistores
- Apple M2 (2022): **20 bilhões** de transistores

Desaceleração

A Lei de Moore está chegando a limites físicos (átomos!). A indústria agora busca ganhos por **paralelismo** (multicore), **especialização** (GPUs, TPUs) e novos materiais.

Característica	CISC	RISC
Instruções	Muitas, complexas	Poucas, simples
Formato	Variável	Fixo (32 bits)
Ciclos por instrução	Múltiplos	Idealmente 1
Acesso à memória	Qualquer instrução	Apenas load/store
Registradores	Poucos	Muitos
Pipeline	Difícil	Facilita
Exemplo	x86 (Intel, AMD)	MIPS, ARM, RISC-V

Na prática hoje

Processadores x86 modernos **traduzem** instruções CISC para micro-operações RISC internamente. O debate CISC vs RISC se diluiu.

Por que estudamos MIPS?

- Arquitetura **RISC limpa e didática**
- Formato de instrução fixo (32 bits) – fácil de decodificar
- Apenas 3 formatos: R, I, J
- Pipeline clássico de 5 estágios
- Documentação excelente (Patterson & Hennessy)
- Simulador gratuito: **MARS**

Transferência de conhecimento

Os conceitos aprendidos com MIPS se aplicam diretamente a **ARM** (smartphones), **RISC-V** (IoT, open-source) e ajudam a entender **x86** por contraste.

Michael Flynn (1966)

Classificação de arquiteturas de computadores baseada em **fluxos de instruções** e **fluxos de dados**.

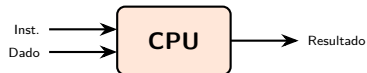
	1 Fluxo de Dados	Múltiplos Fluxos de Dados
1 Fluxo de Inst. Múlt. Fluxos de Inst.	SISD MISD	SIMD MIMD

Descrição

Um processador executa **uma instrução** sobre **um dado** por vez. É o modelo clássico de Von Neumann sequencial.

Exemplos:

- Processadores moncore tradicionais
- Microcontroladores simples (PIC, AVR)
- O MIPS que estudaremos neste curso

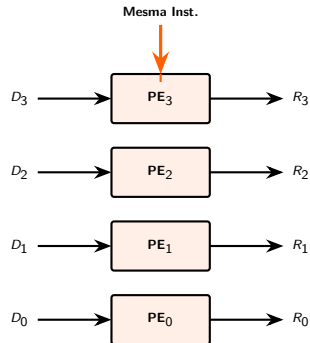


Descrição

Uma **mesma instrução** é aplicada simultaneamente a **múltiplos dados** diferentes.

Exemplos:

- GPUs (milhares de núcleos SIMD)
- Instruções SSE/AVX do x86
- NEON do ARM
- Processamento de imagens, IA, jogos



MISD: Multiple Instruction, Single Data

- Múltiplas instruções sobre o **mesmo dado**
- Raro na prática
- Exemplo teórico: arrays sistólicos, tolerância a falhas (votação)

MIMD: Multiple Instruction, Multiple Data

- Múltiplos processadores executando instruções **diferentes** sobre dados **diferentes**
- Mais geral e poderoso
- **Exemplos:** CPUs multicore, clusters, supercomputadores

Resumo visual

	1 Dado	Vários Dados
1 Instrução	SISD (monocore)	SIMD (GPU, SSE)
Várias Instruções	MISD (raro)	MIMD (multicore)



Teste seus conhecimentos

Questão para reflexão

Classifique cada sistema segundo a taxonomia de Flynn (**SISD**, **SIMD**, **MISD**, **MIMD**) e justifique:

- 1 Um processador escalar clássico executando uma única thread.
- 2 Uma GPU executando o mesmo shader em 1024 pixels simultaneamente.
- 3 Um cluster com 8 nós, cada um com seu próprio programa e dados.
- 4 Um pipeline de processamento de sinais com múltiplos filtros em série sobre o mesmo fluxo de dados.

Dica

Foque em **dois fluxos**: instrução (S/M) e dado (S/M). GPU: mesma instrução, dados diferentes (pixels). Cluster: programas independentes.

Debate: "Von Neumann Ainda é Relevante?"

Regras do Debate (30 min)

- Turma dividida em 2 grupos (posição sorteada)
- **Grupo A:** "Sim, Von Neumann é a base de tudo"
- **Grupo B:** "Não, precisamos de novos paradigmas"
- 10 min de preparação, 15 min de debate, 5 min de síntese

Argumentos para explorar:

A favor de Von Neumann:

- Flexibilidade do programa armazenado
- Base de 99% dos computadores
- Harvard Modificado preserva a ideia

Contra Von Neumann:

- Gargalo de memória
- IA exige paralelismo massivo
- Computação neuromórfica
- Computação quântica

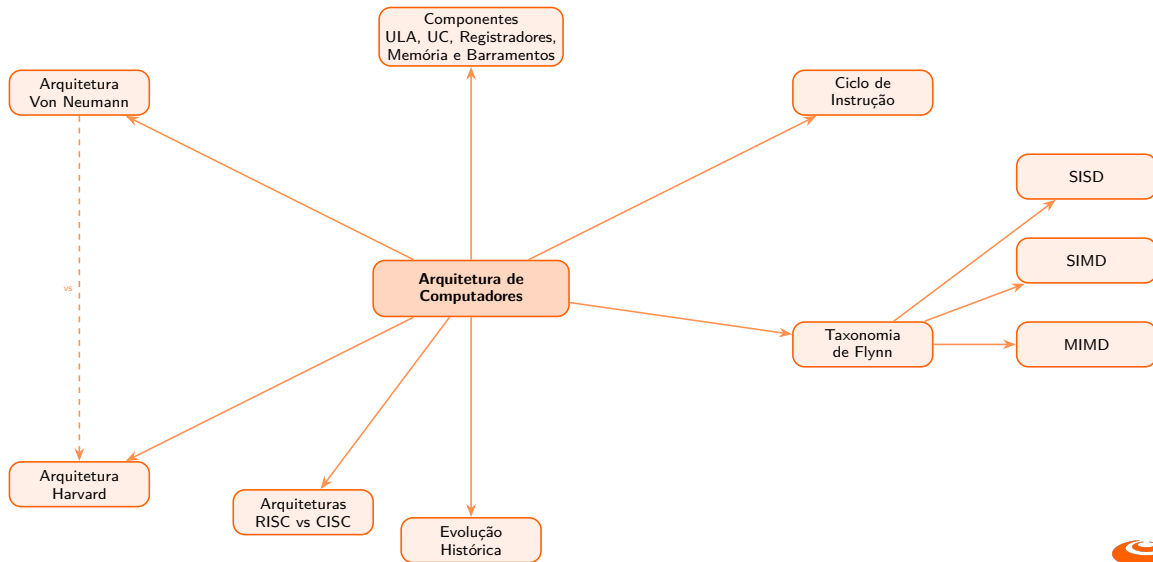
Para além de Von Neumann:

- **Multicore/Manycore:** mais núcleos em vez de mais velocidade
- **GPUs e Aceleradores:** paralelismo massivo para IA e gráficos
- **Processing-in-Memory (PIM):** computação dentro da memória (reduz gargalo)
- **Computação Neuromórfica:** chips que imitam neurônios (Intel Loihi)
- **Computação Quântica:** qubits e superposição (IBM, Google)
- **RISC-V:** ISA aberta e livre, crescimento exponencial
- **Chipelets:** múltiplos chips em um pacote (AMD, Apple)

Reflexão

Mesmo com todas essas inovações, os **fundamentos** que estamos estudando (ciclo de instrução, ULA, registradores, memória) continuam sendo a base de praticamente toda a computação.

Mapa Conceitual do Módulo



Registradores e Instruções Básicas do MIPS

Metodologia: Sala de Aula Invertida + Coding Dojo

- Arquitetura MIPS: visão geral e filosofia RISC
- Os 32 registradores e suas convenções
- Formatos de instrução: R, I, J
- Instruções aritméticas e lógicas
- Primeiro programa no MARS (Coding Dojo)

O que você escreve:

```
1 x = a + b;  
2
```

Uma linha. Simples, certo?

O que o processador executa:

```
1 lw $t0, 0($s0) # carrega a  
2 lw $t1, 4($s0) # carrega b  
3 add $t2, $t0, $t1 # soma  
4 sw $t2, 8($s0) # salva x  
5
```

4 instruções. 2 acessos à memória.

💡 Cada linha de código de alto nível esconde **decisões do hardware**: busca em memória, uso de registradores, ciclos de clock. Assembly revela essas decisões.

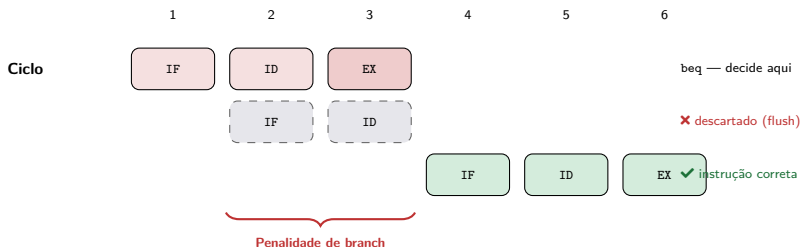
O custo invisível do if

Alto nível — parece inofensivo:

```
1  if (a == b)
2      x = 1;
3  else
4      x = 2;
5
```

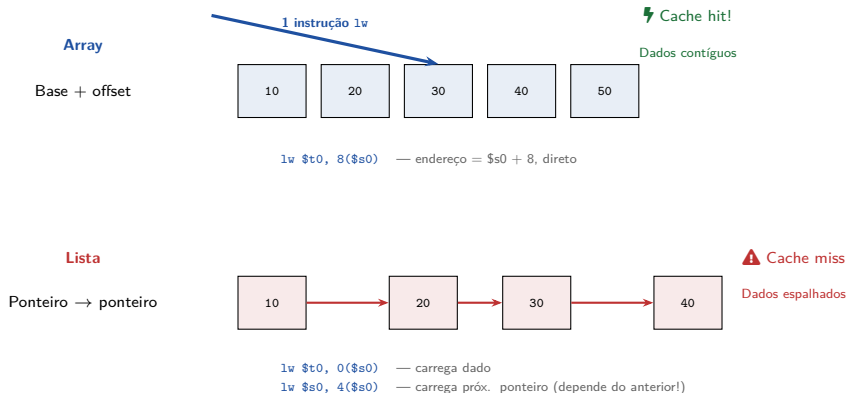
Assembly — revela a complexidade:

```
1  beq  $s0, $s1, EQUAL
2  addi $t0, $zero, 2
3  j     END
4  EQUAL:
5  addi $t0, $zero, 1
6  END:
7  sw   $t0, 0($s2)
8
```



Um if pode custar **ciclos extras** por causa de flush no pipeline.
Em alto nível, isso é **completamente invisível**.

Por que array é rápido e lista encadeada é lenta?



💡 Em assembly fica **evidente**: array = 1 cálculo de endereço; lista = cadeia de lw com **dependência de dados**. Isso impacta diretamente cache e pipeline.

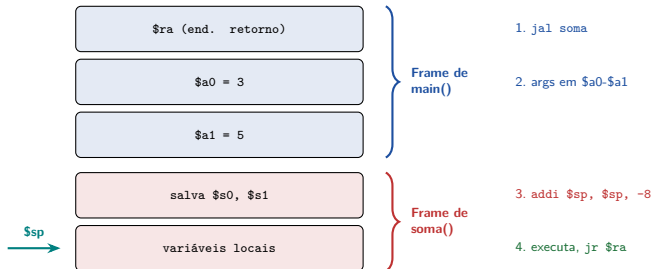
O que realmente acontece quando você chama uma função?

Alto nível:

```
1  int soma(int a, int b) {  
2      return a + b;  
3  }  
4  
5  int main() {  
6      int r = soma(3, 5);  
7  }  
8
```

Parece simples:
uma função, um retorno.

Stack (pilha)



Cada chamada de função = alocar stack frame, salvar registradores, passar argumentos, retornar. **Recursão** é isso $\times N$ vezes.

Onde isso aparece na sua carreira?

Debugging avançado

gdb, objdump, crash dumps
exigem leitura de assembly

Otimização

Profilers mostram hot paths
em instruções de máquina

Segurança

Buffer overflow, stack smashing,
ROP chains — tudo em assembly

Sistemas e embarcados

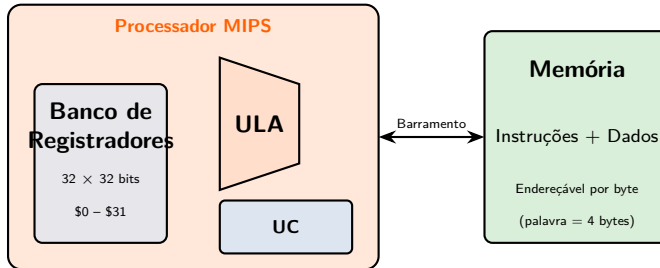
Drivers, SO, firmware, IoT
— assembly é o dia a dia



Vocês aprenderam a **falar com o computador**
na linguagem de vocês.

Agora vão aprender a **linguagem nativa dele**
e descobrir que ele não pensa nada como
vocês imaginavam.





Características RISC: instruções de tamanho fixo (32 bits), formato regular, operações apenas entre registradores.

#	Nome	Uso	#	Nome	Uso
0	\$zero	Constante 0	16–23	\$s0–\$s7	Salvos (preserved)
1	\$at	Reservado assembler	24–25	\$t8–\$t9	Temporários
2–3	\$v0–\$v1	Retorno de função	26–27	\$k0–\$k1	Kernel (SO)
4–7	\$a0–\$a3	Argumentos	28	\$gp	Global pointer
8–15	\$t0–\$t7	Temporários	29	\$sp	Stack pointer
			30	\$fp	Frame pointer
			31	\$ra	Return address

Convenção importante

Caller-saved: \$t0–\$t9, \$a0–\$a3, \$v0–\$v1 (a função chamada pode alterar)

Callee-saved: \$s0–\$s7, \$sp, \$ra (a função chamada deve preservar)

Por que Registradores Importam?

Local	Tempo de Acesso	Relação
Registrador	$\sim 0,5 \text{ ns}$	$1\times$
Cache L1	$\sim 1\text{--}2 \text{ ns}$	$2\text{--}4\times$
Memória RAM	$\sim 50\text{--}100 \text{ ns}$	$100\text{--}200\times$

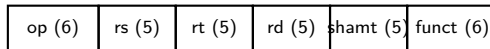
- Registradores são a memória **mais rápida** do processador
- O compilador tenta manter variáveis em registradores o máximo possível
- MIPS tem 32 registradores: ótimo balanço entre velocidade e custo de codificação (5 bits para endereçar)

Analogia

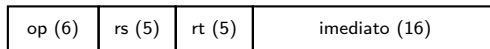
Registradores são como a **mesa de trabalho**. RAM é o **armário**. Quanto mais cabe na mesa, menos vezes você precisa ir ao armário.

Todas as instruções MIPS têm exatamente 32 bits.

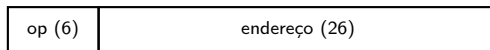
Tipo R:



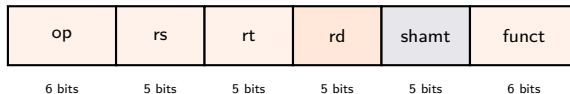
Tipo I:



Tipo J:



- **R-type:** operações entre registradores (add, sub, and, or, slt)
- **I-type:** operações com imediato, acesso à memória, desvios (addi, lw, sw, beq)
- **J-type:** saltos incondicionais (j, jal)



Campo	Significado
op (opcode)	Código da operação (sempre 000000 para R-type)
rs	Registrador fonte 1 (source)
rt	Registrador fonte 2 (source)
rd	Registrador destino (destination)
shamt	Shift amount (deslocamento, 0 para add/sub)
funct	Código da função (diferencia add, sub, and, etc.)

Sintaxe assembly: op rd, rs, rt Exemplo: add \$t0, \$t1, \$t2

Instrução	Exemplo	Significado	funct
add	add \$t0,\$t1,\$t2	$t0 = t1 + t2$	100000
sub	sub \$t0,\$t1,\$t2	$t0 = t1 - t2$	100010
addu	addu \$t0,\$t1,\$t2	Soma sem overflow	100001
subu	subu \$t0,\$t1,\$t2	Sub. sem overflow	100011
mult	mult \$t1,\$t2	$Hi:Lo = t1 \times t2$	011000
div	div \$t1,\$t2	Lo=quoc., Hi=resto	011010

add vs addu

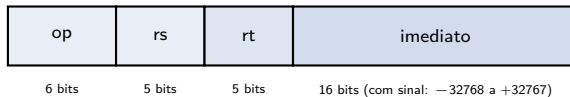
add gera exceção em overflow. addu ignora overflow.

Em C: variáveis int $\rightarrow$ add; unsigned $\rightarrow$ addu.

Instrução	Exemplo	Significado	funct
and	and \$t0,\$t1,\$t2	$t0 = t1 \& t2$	100100
or	or \$t0,\$t1,\$t2	$t0 = t1 t2$	100101
xor	xor \$t0,\$t1,\$t2	$t0 = t1 \oplus t2$	100110
nor	nor \$t0,\$t1,\$t2	$t0 = \sim(t1 t2)$	100111
slt	slt \$t0,\$t1,\$t2	$t0 = (t1 < t2) ? 1 : 0$	101010
sll	sll \$t0,\$t1,2	$t0 = t1 \ll 2$	000000
srl	srl \$t0,\$t1,2	$t0 = t1 \gg 2$	000010

Aplicações práticas

and: mascaramento de bits sll: multiplicação por 2^n slt: comparação para desvios
nor \$t0, \$t1, \$zero: equivale a NOT(\$t1)



Três usos do formato I:

- ❶ **Aritmética com imediato:** `addi $t0, $t1, 42`
- ❷ **Acesso à memória:** `lw $t0, 8($s0)` (rt = destino, rs = base)
- ❸ **Desvio condicional:** `beq $t0, $t1, label` (offset relativo ao PC)

Atenção ao rt

No I-type, rt é o registrador **destino** (diferente do R-type onde é fonte 2).

Instrução	Exemplo	Significado	op
addi	addi \$t0,\$t1,5	$\$t0 = \$t1 + 5$	001000
addiu	addiu \$t0,\$t1,5	Sem exceção	001001
andi	andi \$t0,\$t1,0xFF	$\$t0 = \$t1 \& 0xFF$	001100
ori	ori \$t0,\$t1,0xFF	$\$t0 = \$t1 0xFF$	001101
slti	slti \$t0,\$t1,10	$\$t0 = (\$t1 < 10) ? 1 : 0$	001010
lui	lui \$t0,0x1234	$\$t0 = 0x12340000$	001111

Para carregar constantes de 32 bits

```
lui $t0, 0x1234      # parte alta: $t0 = 0x12340000
ori $t0, $t0, 0x5678 # parte baixa: $t0 = 0x12345678
```

Load Word:

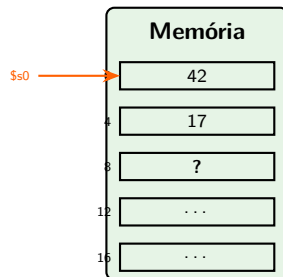
```
lw $t0, offset($base)  
$t0 ← Mem[$base + offset]
```

Store Word:

```
sw $t0, offset($base)  
Mem[$base + offset] ← $t0
```

Exemplo:

```
lw $t0, 0($s0)  
lw $t1, 4($s0)  
add $t2, $t0, $t1  
sw $t2, 8($s0)
```



Alinhamento

Endereços de lw/sw devem ser **múltiplos de 4** (palavra = 4 bytes).

Questão para reflexão

A instrução MIPS `lw $t0, 8($s1)` pertence ao **formato tipo I**. Identifique os campos corretos nos 32 bits da instrução:

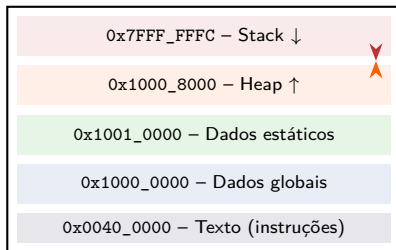
- ❶ `op=100011, rs=$t0, rt=$s1, imm=8`
- ❷ `op=100011, rs=$s1, rt=$t0, imm=8`
- ❸ `op=000000, rs=$s1, rt=$t0, funct=100011`
- ❹ `op=100011, rs=$t0, rt=$s1, imm=2`

Dica

Estrutura tipo I: `[op 6] [rs 5] [rt 5] [imm 16]`

Em `lw rt, imm(rs)`: quem é a **base** (rs) e quem é o **destino** (rt)? Cuidado com alternativas que invertem rs e rt!

Mapa de Memória MIPS



- **Texto:** código do programa (instruções)
- **Dados:** variáveis globais, strings, constantes
- **Heap:** alocação dinâmica (cresce ↑)
- **Stack:** variáveis locais, endereço de retorno (cresce ↓)

Código C:

```
f = (g + h) - (i + j);
```

Assumindo: $f \rightarrow \$s0$, $g \rightarrow \$s1$, $h \rightarrow \$s2$, $i \rightarrow \$s3$, $j \rightarrow \$s4$

Assembly MIPS:

```
add $t0, $s1, $s2    # $t0 = g + h
add $t1, $s3, $s4    # $t1 = i + j
sub $s0, $t0, $t1    # f = $t0 - $t1
```

Observação

O MIPS realiza **uma operação por instrução** (3 operandos max.). Expressões complexas precisam de temporários (\$t0, \$t1).

C: $A[12] = h + A[8];$
 $h \rightarrow \$s2$, base de $A \rightarrow \$s3$

MIPS:

```
lw $t0, 32($s3)    # $t0 = A[8]    (8×4 = 32 bytes de offset)
add $t0, $s2, $t0   # $t0 = h + A[8]
sw $t0, 48($s3)     # A[12] = $t0   (12×4 = 48)
```

Cálculo do offset

Em C, $A[i]$ está no endereço $\text{base} + i \times 4$ (para int de 32 bits).

Offset = índice $\times$ tamanho_da_palavra = $i \times 4$.

Dinâmica do Coding Dojo (60 min)

- Projetor com MARS aberto (simulador MIPS)
- **Driver:** digita o código (rodízio a cada 3 min)
- **Copiloto:** sugere o que escrever
- **Plateia:** ajuda quando solicitado

Desafio 1: Calcular $f = 2a + 3b - c$

Valores iniciais: $a = 5$, $b = 3$, $c = 4$.

Resultado esperado: $f = 10 + 9 - 4 = 15$.

Dica: Use `addi` para carregar constantes e `mult/mflo` ou `sll` para multiplicar.

Desafio 2: Somar os elementos de um vetor

Vetor: [10, 20, 30, 40, 50] armazenado no segmento de dados.

Resultado: soma = 150, armazene em \$s0.

Vocês vão precisar de:

- .data para declarar o vetor
- la para carregar endereço base
- Um loop com lw, add, addi e bne

Estrutura sugerida:

```
la $t0, vetor      # endereço base
li $t1, 5           # contador
li $s0, 0           # acumulador
loop: ...
```

O que aprendemos:

- 32 registradores e convenções
- Formatos R, I, J
- add, sub, and, or, slt, sll, srl
- addi, andi, ori, lui
- lw, sw (load/store)
- Compilação C → MIPS

Para a próxima aula:

- Instalar MARS no seu computador
- Estudar: instruções de desvio (beq, bne, j)
- Pensar: como implementar if-else em assembly?
- Pensar: como implementar while em assembly?

Desvios, Loops e Programação no MARS

Metodologia: Pair Programming no MARS

- Instruções de desvio condicional: beq, bne
- Saltos incondicionais: j, jal, jr
- Formato J e endereçamento
- Estruturas de controle: if-else, while, for
- Pair Programming: exercícios progressivos

Instrução	Sintaxe	Significado
beq	beq \$t0, \$t1, label	Se \$t0 == \$t1, desvia
bne	bne \$t0, \$t1, label	Se \$t0 ≠ \$t1, desvia

Formato I: $PC \leftarrow PC + 4 + (\text{offset} \times 4)$

Offset relativo

O campo imediato (16 bits) armazena o **deslocamento em palavras** relativo à instrução seguinte. É com **signal**: pode desviar para frente (+) ou para trás (–).

Alcance: -2^{15} a $+2^{15} - 1$ instruções $\approx \pm 128$ KB.

Inst.	Sintaxe	Significado	Formato
j	j label	$PC \leftarrow \text{endereço}$	J
jal	jal label	$\$ra \leftarrow PC+4$; $PC \leftarrow \text{end.}$	J
jr	jr \$ra	$PC \leftarrow \$ra$	R

Formato J: endereçamento

- 26 bits de endereço na instrução
- Endereço completo: $PC[31:28] \parallel \text{addr}[25:0] \parallel 00$
- Alcance: 256 MB (dentro do mesmo segmento de 256 MB)

jal + jr = chamada de função!

jal salva o endereço de retorno em \$ra.

jr \$ra retorna para onde a função foi chamada.

C:

```
if (a == b) { c = a + b; } else { c = a - b; }
```

$a \rightarrow \$s0$, $b \rightarrow \$s1$, $c \rightarrow \$s2$

MIPS:

```
bne $s0, $s1, else    # se  $a \neq b$ , pula para else
add $s2, $s0, $s1     #  $c = a + b$  (caso then)
j fim                 # pula o else
else:
sub $s2, $s0, $s1     #  $c = a - b$  (caso else)
fim:
```

Padrão

Desviar com a condição **inversa** para pular o bloco "then".

C:

```
while (i < n) { soma += A[i]; i++; }
```

$i \rightarrow \$t0$, $n \rightarrow \$s1$, $soma \rightarrow \$s0$, base $A \rightarrow \$s2$

MIPS:

```
li $t0, 0           # i = 0
li $s0, 0           # soma = 0
loop:
    slt $t1, $t0, $s1  # $t1 = (i < n) ? 1 : 0
    beq $t1, $zero, fim # se i ≥ n, sai do loop
    sll $t2, $t0, 2     # $t2 = i × 4 (byte offset)
    add $t2, $s2, $t2   # $t2 = endereço de A[i]
    lw $t3, 0($t2)      # $t3 = A[i]
    add $s0, $s0, $t3   # soma += A[i]
    addi $t0, $t0, 1    # i++
    j loop              # volta ao início
fim:
```

Pseudo	Equivale a	Significado
li \$t0, 100	addi \$t0, \$zero, 100	Carrega imediato
la \$t0, label	lui + ori	Carrega endereço
move \$t0, \$t1	add \$t0, \$t1, \$zero	Copia registrador
blt \$t0, \$t1, L	slt + bne	Branch if less than
bgt \$t0, \$t1, L	slt + bne	Branch if greater
bge \$t0, \$t1, L	slt + beq	Branch if $\geq$
nop	sll \$zero, \$zero, 0	No operation

Syscall – Chamadas de Sistema no MARS

```
li $v0, 1 → print integer ($a0 = valor)
li $v0, 4 → print string ($a0 = endereço)
li $v0, 5 → read integer (resultado em $v0)
li $v0, 10 → exit
```

Exercício 1: Soma de vetor (20 min)

Dado um vetor de 10 inteiros no segmento de dados, calcule a soma e imprima o resultado com `syscall`.

Exercício 2: Busca linear (20 min)

Dado um vetor e um valor-alvo, encontre a posição do elemento. Se não encontrar, imprima `-1`.

Exercício 3: Maior e menor (20 min)

Percorra um vetor e encontre o maior e o menor elemento. Imprima ambos.

Regras: Driver e Copiloto trocam a cada exercício. Documentem o código com comentários!

O que aprendemos:

- beq, bne: desvios condicionais
- j, jal, jr: saltos
- Formato J e endereçamento
- if-else, while, for em MIPS
- Pseudoinstruções e syscall

Para a próxima aula:

- Revisar: os 3 formatos (R, I, J)
- Praticar: converter add \$t0, \$t1, \$t2 para binário manualmente
- Estudar: codificação completa dos campos

Codificação de Instruções e Decodificação

- Assembly → Binário: codificação completa
- Binário → Assembly: decodificação
- Tabela de referência de opcodes e functs
- Projeto: mini-decodificador (manual ou Python)

Instrução	Formato	op (bin)	funct (bin)
add	R	000000	100000
sub	R	000000	100010
and	R	000000	100100
or	R	000000	100101
slt	R	000000	101010
sll	R	000000	000000
jr	R	000000	001000
addi	I	001000	–
lw	I	100011	–
sw	I	101011	–
beq	I	000100	–
bne	I	000101	–
j	J	000010	–
jal	J	000011	–

Números dos registradores: \$t0=8, \$t1=9, \$t2=10, ..., \$s0=16, \$s1=17, ...

Exemplo 1: Assembly → Binário (R-type)

Instrução: add \$t0, \$s1, \$s2

Passo 1: Identificar formato → R-type (operação entre registradores)

Passo 2: Preencher campos:

op	rs	rt	rd	shamt	funct
000000	\$s1=10001	\$s2=10010	\$t0=01000	00000	100000

Passo 3: Concatenar:

0000 0010 0011 0010 0100 0000 0010 0000

Passo 4: Converter para hex:

0x02324020

Exemplo 2: Assembly → Binário (I-type)

Instrução: lw \$t0, 32(\$s3)

Passo 1: Formato I (acesso à memória)

Passo 2: Preencher campos:

op	rs (base)	rt (dest)	imediato
100011	\$s3=10011	\$t0=01000	0000000000100000

Passo 3: Concatenar:

1000 1110 0110 1000 0000 0000 0010 0000

Hex: 0x8E680020

Dica

No MARS, use Settings → Show Machine Code para verificar!

Exemplo 3: Binário → Assembly (Decodificação)

Instrução em hex: 0x01095020

Passo 1: Converter para binário:

0000 0001 0000 1001 0101 0000 0010 0000

Passo 2: Extrair campos:

op	rs	rt	rd	shamt	funct
000000	01000	01001	01010	00000	100000
	\$t0=8	\$t1=9	\$t2=10	0	add

Passo 3: op=000000 → R-type. funct=100000 → add

Resultado: add \$t2, \$t0, \$t1

Problema (60 min)

Você recebeu um dump de memória com 8 instruções MIPS em hexadecimal. Decodifique **todas** para assembly e descubra o que o programa faz!

Endereço	Instrução (hex)
0x00400000	0x20080005
0x00400004	0x20090001
0x00400008	0x11000003
0x0040000C	0x01284822
0x00400010	0x01094820
0x00400014	0x2108FFFF
0x00400018	0x08100002
0x0040001C	0x0000000C

Dica: Comece identificando o opcode (primeiros 6 bits) de cada instrução.

End.	Assembly	Comentário
0x00	addi \$t0, \$zero, 5	n = 5
0x04	addi \$t1, \$zero, 1	resultado = 1
0x08	beq \$t0, \$zero, +3	se n==0, sai
0x0C	sub \$t1, \$t1, \$t0	(erro?)
0x10	add \$t1, \$t0, \$t1	resultado *= n
0x14	addi \$t0, \$t0, -1	n--
0x18	j 0x00400008	volta ao loop
0x1C	syscall	fim

O que o programa faz?

Calcula o **fatorial** de 5!. Linhas 0x0C–0x10 implementam $\text{resultado} = \text{resultado} * n$ usando sub+add (versão simplificada).

O que aprendemos:

- Codificação assembly → binário
- Decodificação binário → assembly
- Tabela de opcodes e functs
- Exercício completo de engenharia reversa

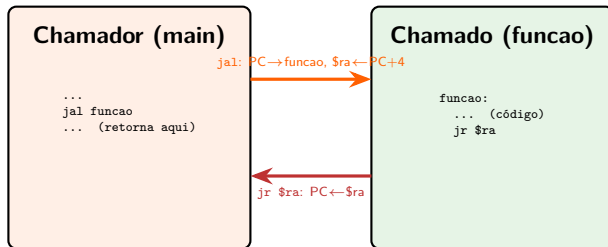
Para a próxima aula(Hackathon):

- Revisar **todas** as instruções
- Praticar syscalls no MARS
- Preparar-se para os desafios!
- Estudar: jal, jr, pilha (\$sp)

Procedimentos, Pilha e Hackathon MIPS

Metodologia: Mini-Hackathon Competitivo

- Chamada de procedimentos: jal, jr
- Convenção de chamada e a pilha (\$sp, \$ra)
- Funções recursivas em MIPS
- Hackathon: 3 desafios competitivos



Protocolo:

- 1 Argumentos em `$a0–$a3`
- 2 `jal funcao` (salva `PC+4` em `$ra`)
- 3 Função executa e coloca retorno em `$v0–$v1`
- 4 `jr $ra` retorna

Por que precisamos da pilha?

Se main chama f que chama g:
\$ra de f é sobrescrito por jal g!

Solução: **salvar \$ra na pilha** antes de chamar outra função.

Operações na pilha:

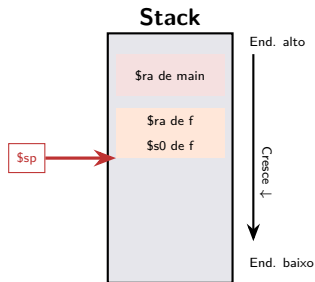
addi \$sp, \$sp, -4 (push: aloca espaço)

sw \$ra, 0(\$sp) (salva \$ra)

...

lw \$ra, 0(\$sp) (restaura \$ra)

addi \$sp, \$sp, 4 (pop: libera espaço)



Antes da chamada (caller):

- 1 Salvar registradores caller-saved que precisa preservar (\$t0–\$t9, \$a0–\$a3)
- 2 Colocar argumentos em \$a0–\$a3
- 3 Executar jal funcao

Início da função chamada (callee):

- 1 Alocar espaço na pilha: addi \$sp, \$sp, -N
- 2 Salvar \$ra e registradores \$s que vai usar

Fim da função chamada:

- 1 Colocar valor de retorno em \$v0
- 2 Restaurar \$ra e registradores \$s
- 3 Liberar pilha: addi \$sp, \$sp, N
- 4 Retornar: jr \$ra

Questão para reflexão

Dado o fragmento de código C abaixo, indique quais registradores MIPS seriam usados para cada variável, seguindo as convenções de chamada (\$a0–\$a3, \$v0–\$v1, \$t0–\$t9, \$s0–\$s7, \$sp, \$ra):

```
1 int soma(int a, int b) {  
2     int resultado = a + b;  
3     return resultado;  
4 }  
5 int main() {  
6     int x = 5, y = 3;  
7     int z = soma(x, y);  
8 }  
9
```

Dica

Parâmetros: \$a0, \$a1

Retorno: \$v0

Temporários: \$t0–\$t9

Salvos: \$s0–\$s7

Ret. address: \$ra

Qual é a diferença entre caller-saved e callee-saved?

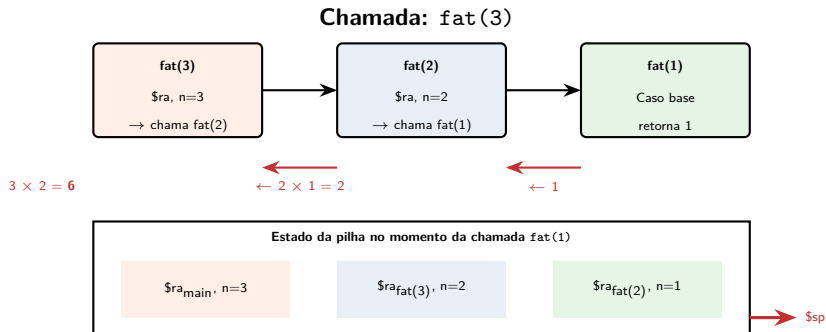
Exemplo: Função Fatorial (Recursiva)

```
C: int fat(int n) { return (n<=1) ? 1 : n * fat(n-1); }
```

MIPS:

```
fat:
    addi $sp, $sp, -8    # aloca 2 palavras
    sw $ra, 4($sp)       # salva $ra
    sw $a0, 0($sp)       # salva n
    slti $t0, $a0, 2     # n < 2?
    beq $t0, $zero, rec  # se n ≥ 2, recursão
    li $v0, 1            # caso base
    addi $sp, $sp, 8     # libera pilha
    jr $ra              # retorna
rec:
    addi $a0, $a0, -1    # n-1
    jal fat             # chamada recursiva
    lw $a0, 0($sp)       # restaura n
    lw $ra, 4($sp)       # restaura $ra
    addi $sp, $sp, 8     # libera pilha
    mul $v0, $a0, $v0    # n × fat(n-1)
    jr $ra              # retorna
```

Visualizando a Pilha no Fatorial



Competição (70 min)

- Equipes de 2–3 alunos
- 3 desafios com pontuação progressiva
- Critérios: **corretude** (60%), **eficiência** (20%), **documentação** (20%)
- Envio pelo Google Classroom: arquivo .asm por desafio

#	Desafio	Pontos
1	Fibonacci iterativo (N termos)	30
2	MDC (algoritmo de Euclides)	30
3	Ordenação por seleção de vetor	40

Especificação

Leia um inteiro n do teclado (`syscall 5`).

Imprima os primeiros n termos da sequência de Fibonacci.

Exemplo: $n = 8 \rightarrow 0 \ 1 \ 1 \ 2 \ 3 \ 5 \ 8 \ 13$

Dicas:

- Use 3 registradores: `prev`, `curr`, `next`
- Loop com contador
- `syscall 1` para imprimir cada número
- `syscall 11` com `$a0 = 32` para imprimir espaço

Especificação

Leia dois inteiros a e b do teclado.

Calcule e imprima o MDC usando o algoritmo de Euclides.

Exemplo: $\text{MDC}(48, 18) = 6$

Algoritmo em pseudocódigo:

```
while (b != 0):  
    temp = b  
    b = a mod b  
    a = temp  
return a
```

Dica: Use `div $s0, $s1` e `mfhi $t0` para obter o resto.

Desafio 3: Ordenação por Seleção (40 pts)

Especificação

Vetor predefinido no .data: [64, 25, 12, 22, 11]

Ordene em ordem crescente usando **Selection Sort**.

Imprima o vetor antes e depois da ordenação.

Bônus (+10 pts): Implementar como **procedimento** (função) que recebe endereço do vetor e tamanho como argumentos.

Selection Sort – Lembrete

Para cada posição i : encontre o menor elemento de i até $n - 1$ e troque com $A[i]$.

Critérios de avaliação por desafio

Critério	Peso
Programa executa corretamente	60%
Uso eficiente de registradores e instruções	20%
Código comentado e organizado	20%

Apresentação (10 min): As 2 melhores equipes apresentam suas soluções do Desafio 3, explicando as decisões de projeto.

Entregável do Módulo 4

Peso: 15% da nota

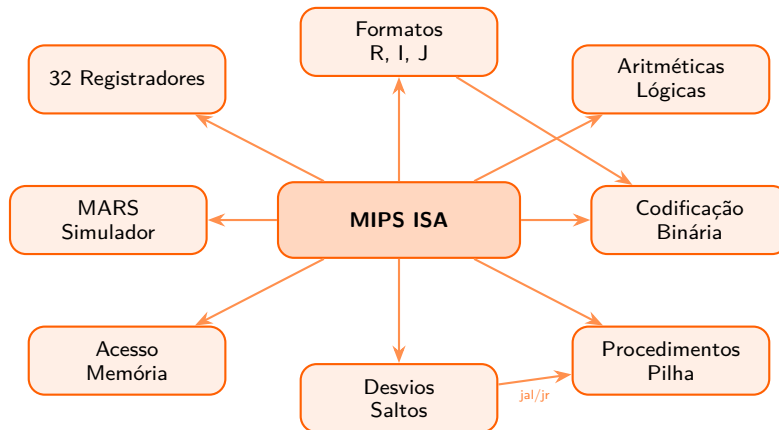
Formato: Individual ou dupla

Prazo: 1 semana após a Aula 13

Entregar 5 programas MIPS documentados:

- ❶ Calculadora básica (+, −, ×, ÷) com menu
- ❷ Busca linear em vetor (com entrada do usuário)
- ❸ Fibonacci **recursivo** (usando pilha)
- ❹ Inversão de string (leitura, inversão, impressão)
- ❺ Programa livre (criatividade – ex.: jogo, criptografia simples)

Cada programa deve incluir: cabeçalho com nome/matrícula, comentários em **todas** as linhas, e pelo menos 3 casos de teste documentados.



Datapath do Processador Monociclo

Metodologia: Construção Incremental Guiada

- O que é um processador monociclo?
- Componentes do datapath: PC, memórias, registradores, ULA, MUX
- Construção incremental: instrução por instrução
- Caminho de dados para R-type, lw e sw

O que é um Processador Monociclo?

Definição

No processador **monociclo**, cada instrução é executada em **exatamente um ciclo de clock**. Todas as fases (Fetch, Decode, Execute, Memory, WriteBack) acontecem dentro do mesmo ciclo.

Vantagens:

- Simples de projetar
- $CPI = 1$ (sempre!)
- Fácil de entender

Desvantagens:

- Clock limitado pela instrução mais lenta
- Hardware não é reutilizado
- Ineficiente para instruções rápidas

Consequência

$T_{clock} = T_{instruo_mais_lenta}$ Se 1w demora 800 ps, **todas** as instruções usam 800 ps, mesmo que add precise de apenas 600 ps.

Fórmula fundamental:

$$T_{exec} = N_{inst} \times CPI \times T_{clock}$$

No monociclo:

- $CPI = 1$ (uma instrução por ciclo)
- T_{clock} é determinado pelo **caminho crítico**

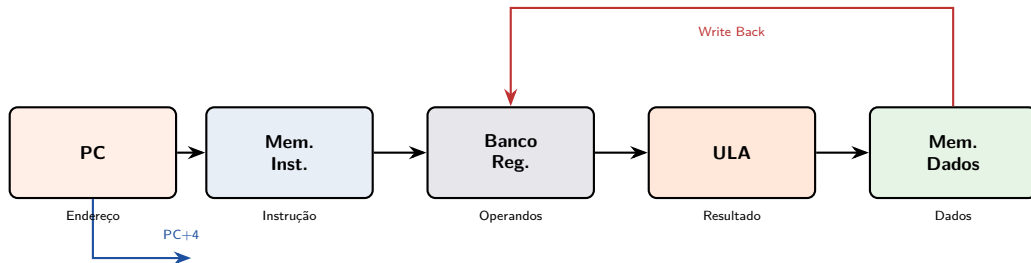
Exemplo

Programa com 100 instruções, $T_{clock} = 800$ ps:

$$T_{exec} = 100 \times 1 \times 800 \text{ ps} = 80.000 \text{ ps} = 80 \text{ ns}$$

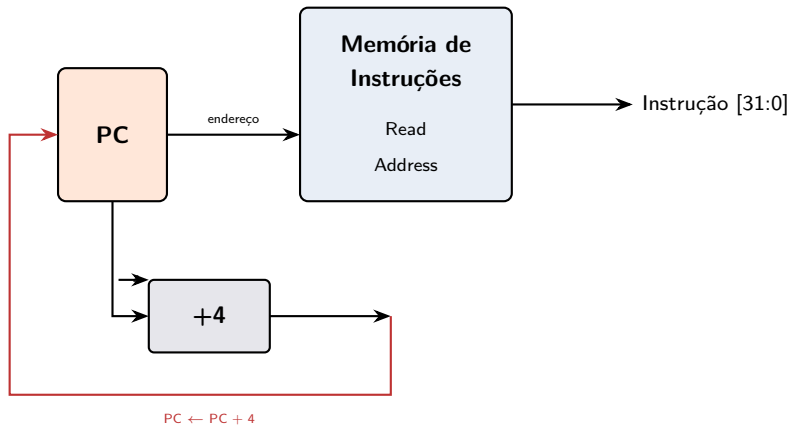
Pergunta: Se pudéssemos ter T_{clock} diferente para cada tipo de instrução, o que mudaria? (*Resposta: processador multiciclo!*)

Componentes do Datapath



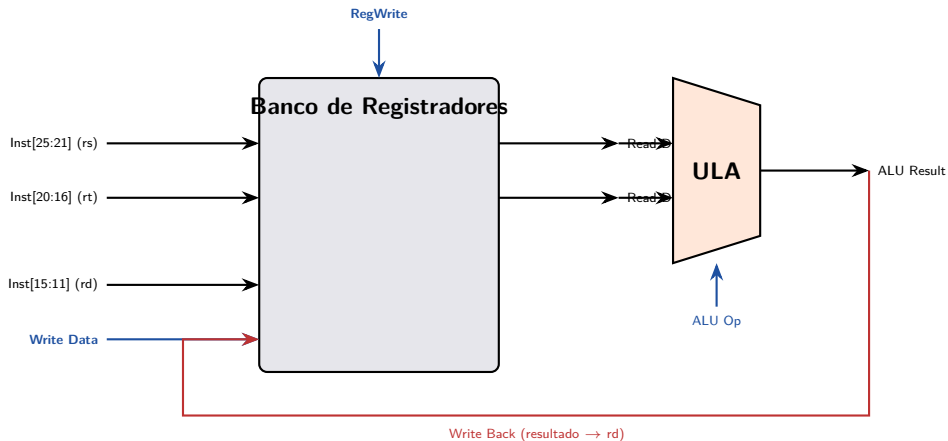
Fluxo básico: **PC** → **Memória de Inst.** → **Registradores** → **ULA** → **Memória de Dados** → **Registradores**

Construção Incremental – Passo 1: Fetch



Fetch: PC fornece endereço → Memória retorna instrução de 32 bits → PC incrementa em 4.

Construção Incremental – Passo 2: Decode + R-type



Para R-type: lê rs e rt → ULA opera → resultado volta para rd.

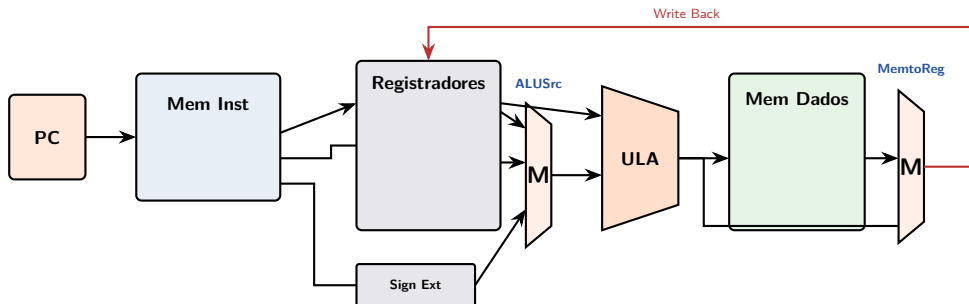
Para `lw $rt, offset($rs)`:

- 1 Lê \$rs do banco de registradores
- 2 ULA calcula endereço: $\$rs + \text{offset (sign-extended)}$
- 3 Memória de dados lê o endereço calculado
- 4 Resultado da memória é escrito em \$rt

Novos componentes necessários:

- **Sign-Extend:** estende imediato de 16 para 32 bits (com sinal)
- **MUX (ALUSrc):** seleciona entre Read Data 2 (R-type) e imediato sign-extended (lw)
- **Memória de Dados:** com entrada de endereço e saída de dados
- **MUX (MemtoReg):** seleciona entre resultado da ULA (R-type) e dado da memória (lw)

Datapath com R-type e lw



ALUSrc=0: operando da ULA vem do registrador (R-type)

ALUSrc=1: operando da ULA vem do imediato (lw/sw)

MemtoReg=0: resultado da ULA vai para registrador (R-type)

MemtoReg=1: dado da memória vai para registrador (lw)

Para sw \$rt, offset(\$rs):

- 1 Lê \$rs e \$rt do banco de registradores
- 2 ULA calcula endereço: $\$rs + \text{offset}$ (sign-extended)
- 3 Read Data 2 (\$rt) é escrito na memória de dados no endereço calculado

Diferenças em relação ao lw:

Sinal	lw	sw
MemRead	1	0
MemWrite	0	1
RegWrite	1	0
ALUSrc	1	1
MemtoReg	1	X (don't care)

A memória de dados agora precisa de uma **entrada Write Data** conectada a Read Data 2 do banco de registradores.

Para beq \$rs, \$rt, offset:

- 1 Lê \$rs e \$rt
- 2 ULA calcula $\$rs - \rt
- 3 Se resultado = 0 (flag Zero), desvia: $PC \leftarrow PC + 4 + (\text{offset} \times 4)$

Novos componentes:

- **Shift Left 2:** desloca o offset 2 posições (multiplica por 4)
- **Somador de branch:** calcula $PC + 4 + (\text{offset} \ll 2)$
- **MUX (PCSrc):** seleciona entre PC+4 (sequencial) e endereço de branch
- **Porta AND:** $\text{Branch} \wedge \text{Zero} \rightarrow \text{PCSrc}$

Sinal PCSrc

$\text{PCSrc} = \text{Branch AND Zero}$

Só desvia se a UC ativar Branch e a ULA indicar Zero.

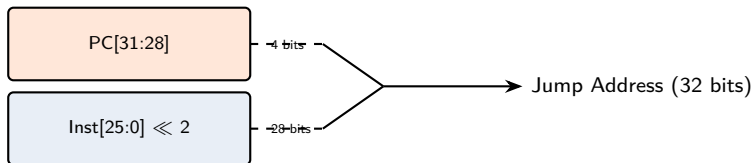
Adicionando j (Jump)

Para j target:

$$PC \leftarrow PC[31:28] \parallel Inst[25:0] \parallel 00$$

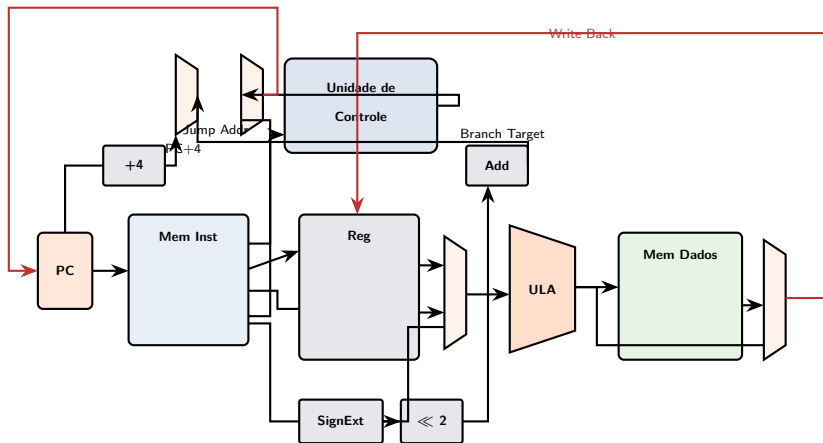
Novos componentes:

- **Shift Left 2** no campo de endereço (26 bits $\rightarrow$ 28 bits)
- Concatenação com PC[31:28] (4 bits mais significativos)
- **MUX (Jump)**: seleciona entre saída do MUX de branch e endereço de jump



Concatenação: 4 bits (PC) + 28 bits (offset)

Datapath Completo do Monociclo



O datapath completo suporta: R-type, lw, sw, beq, j.

	Reg Dst	Reg Write	ALU Src	Branch	Mem Read	Mem Write	Mem toReg	ALU Op	Jump
R-type	1	1	0	0	0	0	0	10	0
lw	0	1	1	0	1	0	1	00	0
sw	X	0	1	0	0	1	X	00	0
beq	X	0	0	1	0	0	X	01	0
j	X	0	X	X	0	0	X	XX	1

RegDst: destino é rd (1) ou rt (0)

ALUSrc: operando 2 é reg (0) ou imediato (1)

MemtoReg: escreve resultado da ULA (0) ou da memória (1)

Branch: indica instrução de desvio condicional

MemRead: lê memória de dados

MemWrite: escreve na memória de dados

RegWrite: escreve no banco de registradores

Jump: indica salto incondicional

Exercício: Rastrear add \$t0, \$t1, \$t2

Preencha o caminho que a instrução percorre:

- 1 PC = ? → Memória de Instruções retorna: ?
- 2 Banco de Registradores: rs = ?, rt = ?, rd = ?
- 3 ULA: operação = ?, operandos = ?, resultado = ?
- 4 Memória de Dados: ativada? MemRead = ?, MemWrite = ?
- 5 Write Back: o que é escrito, onde é escrito?

Sinais de controle para R-type: RegDst=1, ALUSrc=0, MemtoReg=0, RegWrite=1, MemRead=0, MemWrite=0, Branch=0

O que construímos:

- Datapath incremental
- Fetch: PC + Memória de Inst
- R-type: Reg $\rightarrow$ ULA $\rightarrow$ Reg
- lw/sw: ULA calcula endereço
- beq: ULA compara + branch
- j: concatenação de endereço
- Datapath completo integrado

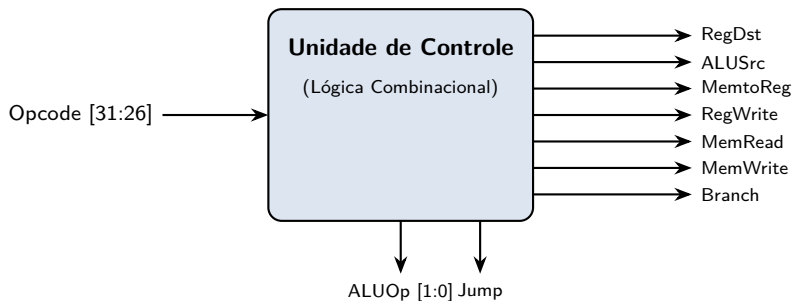
Para a próxima Aula:

- Memorizar os 9 sinais de controle e seus efeitos
- Revisar o datapath completo
- Preparar-se para o Role-Playing!

Unidade de Controle e Role-Playing

Metodologia: Role-Playing do Processador

- Unidade de Controle: como gerar sinais
- Controle da ULA: ALU Control
- Caminho de dados instrução por instrução
- Role-Playing: alunos "são" os componentes



A UC lê apenas os **6 bits do opcode** e gera **todos** os sinais de controle. É pura **lógica combinacional** (sem estado!).

Ideia

A UC gera um sinal genérico **ALUOp** (2 bits).

O **ALU Control** combina ALUOp com o campo **funct** para gerar o sinal final de 4 bits que seleciona a operação da ULA.

ALUOp	funct	ALU Control	Operação
00	XXXXXX	0010	ADD (lw/sw)
01	XXXXXX	0110	SUB (beq)
10	100000	0010	ADD
10	100010	0110	SUB
10	100100	0000	AND
10	100101	0001	OR
10	101010	0111	SLT

Para ALUOp=10 (R-type), o campo **funct** determina a operação exata.

lw \$t0, 12(\$s0) ($\$t0 \leftarrow \text{Mem}[\$s0 + 12]$)

- ❶ **Fetch:** PC $\rightarrow$ Mem Inst $\rightarrow$ Instrução. PC $\leftarrow$ PC+4
- ❷ **Decode:** UC lê opcode (100011). Gera sinais: RegDst=0, ALUSrc=1, MemtoReg=1, RegWrite=1, MemRead=1
- ❸ **Leitura:** Banco de Reg lê \$s0 (rs=10000). Sign-extend: 12 $\rightarrow$ 0x0000000C
- ❹ **Execute:** ULA soma \$s0 + 12. ALU Control = 0010 (ADD)
- ❺ **Memory:** Memória de Dados lê endereço calculado
- ❻ **WriteBack:** MUX MemtoReg=1 seleciona dado da memória. RegWrite escreve em \$t0 (rt=01000, pois RegDst=0)

beq \$s0, \$s1, label (se $\$s0 == \$s1$, $PC \leftarrow PC + 4 + \text{offset} \times 4$)

- ❶ **Fetch:** $PC \rightarrow \text{Mem Inst} \rightarrow \text{Instrução}$. $PC \leftarrow PC + 4$
- ❷ **Decode:** UC lê opcode (000100). Gera: Branch=1, ALUSrc=0, RegWrite=0
- ❸ **Leitura:** Banco de Reg lê \$s0 e \$s1
- ❹ **Execute:** ULA calcula $\$s0 - \$s1$. ALU Control = 0110 (SUB)
- ❺ **Branch:** Somador calcula $PC + 4 + (\text{offset} \ll 2)$
- ❻ **Decisão:**
 - Se Zero=1 (iguais): $PCSrc = \text{Branch} \wedge \text{Zero} = 1$. **Desvia.**
 - Se Zero=0 (diferentes): $PCSrc = 0$. **Sequencial.**

add \$t0, \$t1, \$t2 ($\$t0 \leftarrow \$t1 + \$t2$)

- ❶ **Fetch:** $PC \rightarrow \text{Mem Inst.}$ $PC \leftarrow PC+4$
- ❷ **Decode:** Opcode = 000000. UC gera: RegDst=1, ALUSrc=0, MemtoReg=0, RegWrite=1, ALUOp=10
- ❸ **Leitura:** Reg lê \$t1 (rs) e \$t2 (rt)
- ❹ **Execute:** ALU Control lê funct=100000 com ALUOp=10 $\rightarrow$ ADD. ULA soma valores.
- ❺ **Memory:** Não ativada (MemRead=0, MemWrite=0)
- ❻ **WriteBack:** MUX MemtoReg=0 seleciona resultado da ULA. RegDst=1 seleciona rd (\$t0). RegWrite=1 escreve.

Observação

Para R-type, a memória de dados é **atravessada sem uso** – hardware desperdiçado! Essa é uma limitação do monociclo.

Dinâmica (60 min)

Cada aluno assume o papel de um **componente** do processador.
Instruções são executadas fisicamente com cartões de dados.

Papéis (8–10 alunos):

#	Componente
1	PC (segura o endereço atual)
2	Memória de Instruções (tem as instruções em cartões)
3	Unidade de Controle (lê opcode, anuncia sinais)
4	Banco de Registradores (segura 8 cartões com valores)
5	MUX ALUSrc (escolhe entre reg ou imediato)
6	ULA (faz a conta manualmente)
7	Memória de Dados (armazena palavras em "prateleira")
8	MUX MemtoReg (escolhe resultado)

Programa a executar (4 instruções):

End.	Instrução
0x00	addi \$s0, \$zero, 10
0x04	addi \$s1, \$zero, 20
0x08	add \$s2, \$s0, \$s1
0x0C	sw \$s2, 0(\$s0)

Regras:

- 1 O professor anuncia "CLOCK!" para iniciar cada ciclo
- 2 A UC lê o opcode e **grita** os sinais de controle
- 3 Dados são passados como **cartões** de um componente para outro
- 4 Ao final, verifique: $\$s2 = 30$? $\text{Mem}[10] = 30$?

Discussão em grupo (15 min):

- 1 Qual componente ficou "ocioso" durante add? (*Memória de Dados*)
- 2 Qual instrução envolveu **mais** componentes? (*lw*)
- 3 O que aconteceria se o clock fosse muito rápido para lw? (*Dados inválidos*)
- 4 Por que o MUX ALUSrc é necessário? (*Distinguir R-type de I-type*)
- 5 Se quiséssemos adicionar addi, o que muda no datapath? (*Nada! Apenas ALU Control*)

Insight principal

No monociclo, **todo o hardware** está disponível o tempo todo, mas nem todas as instruções usam todos os componentes. Isso é **ineficiente** → motivação para o multiciclo!

O que aprendemos:

- UC gera sinais a partir do opcode
- ALU Control: 2 níveis de decodificação
- Caminho detalhado de lw, sw, beq, R-type
- Papel de cada MUX
- Vivência do processador (Role-Playing)

Para a próxima Aula:

- Instalar Logisim Evolution
- Revisar o datapath completo
- Preparar: quais componentes do Logisim correspondem ao datapath?

Caminho Crítico, Clock e Laboratório

Metodologia: Laboratório de Simulação

- Atrasos dos componentes
- Caminho crítico e período de clock
- Limitações do monociclo
- Laboratório: processador monociclo em C

Componente	Atraso Picosecond (ps)
Memória de Instruções	200
Banco de Registradores (leitura)	50
Banco de Registradores (escrita)	50
ULA	200
Memória de Dados	200
MUX	25
Somador	50
Sign-Extend	0 (instantâneo)
Unidade de Controle	50

Caminho crítico

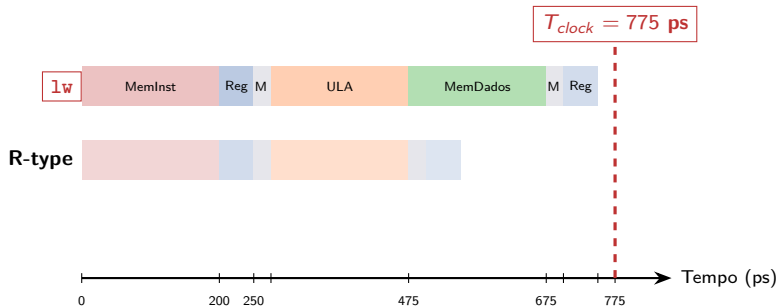
O período de clock deve ser $\geq$ ao atraso do **caminho mais longo** (a instrução que percorre mais componentes sequencialmente).

Instrução	Caminho	Atraso Picosecond (ps)
R-type	MemInst + RegRead + MUX + ULA + MUX + RegWrite	575
sw	MemInst + RegRead + MUX + ULA + MemData	700
beq	MemInst + RegRead + MUX + ULA + MUX	525
j	MemInst + MUX	225
lw	MemInst + RegRead + MUX + ULA + MemData + MUX + RegWrite	775

Resultado

O caminho crítico é o lw: **775 ps**.

Portanto: $T_{clock} \geq 775 \text{ ps} \Rightarrow f_{max} = \frac{1}{775 \times 10^{-12}} \approx 1,29 \text{ GHz}$



R-type termina em $\sim 575 \text{ ps}$, mas precisa esperar até 775 ps – **tempo desperdiçado!**

- ❶ **Clock limitado pela instrução mais lenta:** $T_{clock} = T_{lw} = 775$ ps, mesmo que add use apenas 575 ps
- ❷ **Hardware não reutilizado:** Precisa de 2 memórias separadas (instruções e dados) e 2 somadores
- ❸ **Ineficiência temporal:** Componentes ficam ociosos durante parte do ciclo
- ❹ **Não escala bem:** Instruções mais complexas (ex.: mult) aumentariam T_{clock} para *todas* as instruções

Exemplo numérico

Se 25% das instruções são lw e 75% são R-type:

Monociclo: Todas a 775 ps $\rightarrow$ médio = 775 ps

Ideal: $0.25 \times 775 + 0.75 \times 575 = 625$ ps

Desperdício de $\frac{775-625}{775} \approx 19\%$!

Aspecto	Monociclo	Multiciclo
CPI	1 (fixo)	Variável (3–5)
T_{clock}	Longo (inst. mais lenta)	Curto (estágio mais lento)
Memórias	2 separadas	1 compartilhada
ULA/Somadores	ULA + 2 somadores	Apenas 1 ULA
Controle	Combinacional	FSM
Eficiência	Menor	Maior

Prévia do Módulo 6

No multiciclo, cada instrução usa **apenas os ciclos que precisa**.

R-type: 4 ciclos lw: 5 ciclos j: 3 ciclos

Com T_{clock} menor, o desempenho total pode ser **melhor!**

Projeto (60 min)

Construir um processador monociclo simplificado no Logisim que execute 5 instruções: add, sub, lw, sw, beq.

Subcircuitos sugeridos:

#	Subcircuito
1	PC (registrador de 32 bits + incremento)
2	Memória de Instruções (ROM com programa)
3	Banco de Registradores (8 registradores $\times$ 32 bits)
4	ULA (add, sub com flag Zero)
5	Memória de Dados (RAM 256 palavras)
6	Unidade de Controle (lógica combinacional)
7	Main (integração de tudo)

Carregar na Memória de Instruções:

End.	Instrução
0x00	addi \$t0, \$zero, 7
0x04	addi \$t1, \$zero, 3
0x08	add \$t2, \$t0, \$t1
0x0C	sw \$t2, 0(\$zero)
0x10	lw \$t3, 0(\$zero)
0x14	sub \$t4, \$t0, \$t1
0x18	beq \$t2, \$t3, +2
0x1C	add \$t5, \$zero, \$zero
0x20	addi \$t5, \$zero, 1

Resultado esperado: \$t2=10, \$t3=10, \$t4=4, \$t5=1 (beq desvia!)

- ❶ **Comece pela ULA:** Somador + Subtrator com MUX de operação e flag Zero
- ❷ **Banco de Registradores:** Use componente "RAM" do Logisim com 2 portas de leitura
- ❸ **Sign-Extend:** Use "Bit Extender" ($16 \rightarrow 32$, sign extension)
- ❹ **UC:** Implementar com ROM (opcode como endereço, sinais como dados)
- ❺ **Teste incrementalmente:** Primeiro R-type, depois lw/sw, depois beq
- ❻ **Use probes e LEDs** para visualizar sinais em cada ponto

Simplificação permitida

Pode usar 8 registradores (3 bits) em vez de 32.

Pode usar memórias de 256 palavras em vez de 2^{30} .

O objetivo é entender a **organização**, não o tamanho real.

Entrega (por grupo)

- ❶ **Arquivo .circ** com o processador funcionando
- ❷ **Screenshots** mostrando a execução do programa de teste
- ❸ **Tabela de sinais de controle** para cada instrução implementada
- ❹ **Respostas:**
 - Qual o atraso estimado do seu datapath (em atrasos de porta)?
 - Qual componente é o gargalo?
 - Se adicionasse j, o que mudaria?

Entregável do Módulo 5

Peso: 15% da nota

Formato: Grupo de 2–3 alunos

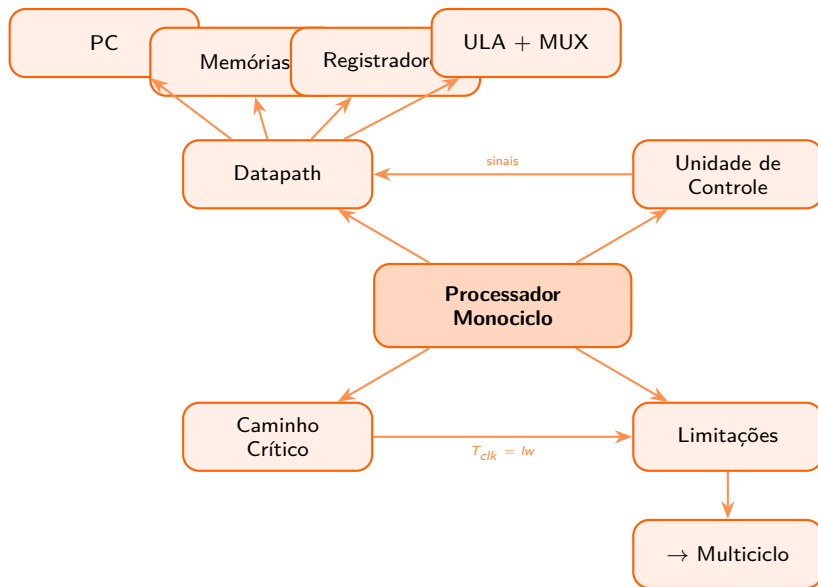
Prazo: 1 semana após a Aula

Requisitos:

- 1 Processador no Logisim com pelo menos 6 instruções: add, sub, and, lw, sw, beq
- 2 Programa de teste com mínimo 10 instruções demonstrando todas as funcionalidades
- 3 Unidade de Controle documentada (tabela de sinais)
- 4 Relatório (2–3 páginas): diagrama de blocos, tabela de sinais, análise do caminho crítico

Bônus (+2 pts): Implementar j e/ou slt

Mapa Conceitual do Módulo 5

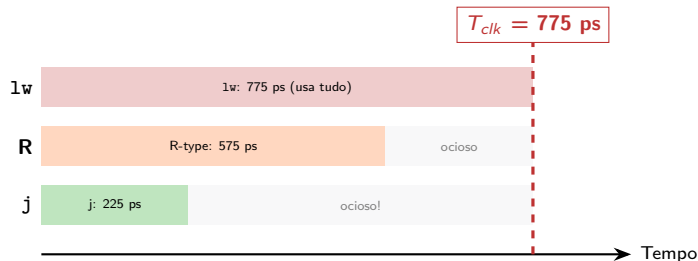


Datapath do Processador Multiciclo

Metodologia: Aprendizagem por Comparação

- Motivação: limitações do monociclo
- Princípio de reutilização de componentes
- Registradores intermediários: IR, MDR, A, B, ALUOut
- Datapath multiciclo completo
- Comparação lado a lado: monociclo vs multiciclo

Relembrando: O Problema do Monociclo

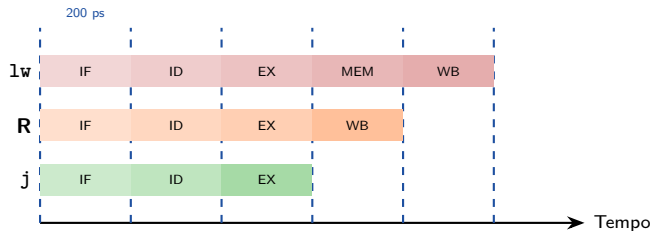


Problema central:

- T_{clock} é ditado pela instrução **mais lenta** (`lw`)
- Instruções rápidas (`j`, `R-type`) desperdiçam tempo
- Hardware usado uma única vez não pode ser reutilizado

Princípio

Dividir a execução em **estágios menores** e usar um clock com período igual ao estágio **mais lento** (não à instrução mais lenta). Cada instrução usa **apenas os ciclos que precisa**.



$T_{clock} = 200 \text{ ps}$ (estágio mais lento) em vez de 775 ps.

Ciclo	Estágio	O que faz
1	IF – Instruction Fetch	Busca instrução; $PC \leftarrow PC+4$
2	ID – Instruction Decode	Decodifica; lê registradores; calcula branch
3	EX – Execute	ULA opera (cálculo ou endereço)
4	MEM – Memory Access	Acessa memória de dados (lw/sw)
5	WB – Write Back	Escreve resultado no banco de reg.

Quantos ciclos por instrução?

Instrução	Ciclos	Estágios usados
R-type	4	IF, ID, EX, WB
lw	5	IF, ID, EX, MEM, WB
sw	4	IF, ID, EX, MEM
beq	3	IF, ID, EX
j	3	IF, ID, EX

Monociclo:

- 2 memórias separadas (Inst + Dados)
- ULA + 2 somadores (PC+4 e Branch)
- Tudo ativo simultaneamente
- Mais hardware, menos eficiente

Multiciclo:

- **1 memória** única (instruções e dados)
- **1 ULA** (faz tudo: PC+4, endereço, branch, operação)
- Componentes usados em ciclos diferentes
- Menos hardware, mais eficiente

Consequência

Como um componente é usado em ciclos diferentes, precisamos de **registradores intermediários** para guardar os resultados entre um ciclo e o próximo.

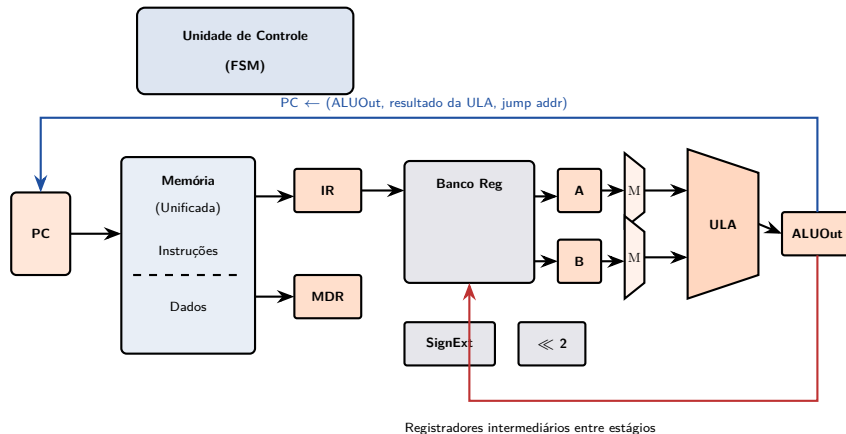
Registrador	Função
IR (Instruction Register)	Armazena a instrução lida no ciclo IF
MDR (Memory Data Register)	Armazena o dado lido da memória (ciclo MEM)
A	Valor lido do registrador rs (ciclo ID)
B	Valor lido do registrador rt (ciclo ID)
ALUOut	Resultado da ULA (ciclo EX)

Por que são necessários?

No monociclo, a informação flui de uma ponta a outra em 1 ciclo.

No multiciclo, ela precisa ser **armazenada** entre estágios, pois os componentes serão reutilizados no próximo ciclo com novos dados.

Datapath Multiciclo – Visão Geral



Memória única, ULA única, registradores intermediários entre estágios.

Recurso	Monociclo	Multiciclo
Memórias	2 (Inst + Dados)	1 (unificada)
ULAs / Somadores	1 ULA + 2 somadores	1 ULA (faz tudo)
MUX extras	3	4 (mais seletores)
Registradores extras	–	5 (IR, MDR, A, B, ALUOut)
Tipo de controle	Combinacional	FSM (sequencial)
CPI	1 (fixo)	3–5 (variável)
T_{clock}	Longo (775 ps)	Curto (200 ps)

Trade-off

Menos hardware (1 memória, 1 ULA), mas **mais complexidade de controle** (FSM em vez de combinacional) e **registradores extras** para armazenar resultados parciais.

A ULA é compartilhada, então precisa de MUX para selecionar entradas:

MUX	Entradas	Seleciona entre...
lorD	PC / ALUOut	Endereço para memória: PC (fetch) ou ALUOut (lw/sw)
ALUSrcA	PC / A	Operando A da ULA: PC (calc. PC+4) ou reg A
ALUSrcB	B / 4 / SignExt / SL2	Operando B da ULA (4 opções!)
MemtoReg	ALUOut / MDR	Dado para WriteBack: ALUOut (R) ou MDR (lw)
RegDst	rt / rd	Registrador destino: rt (lw) ou rd (R-type)
PCSrc	ALU / ALUOut / Jump	Próximo PC: ALU (PC+4), ALUOut (branch), Jump addr

ALUSrcB com 4 entradas!

00 = B (R-type) 01 = 4 (PC+4) 10 = SignExt (lw/sw) 11 = SignExt $\ll$ 2 (beq)

Ciclo 1 – Instruction Fetch (IF)

Ações (todas as instruções):

- 1 Memória lê instrução no endereço do PC
- 2 Instrução é armazenada em **IR**
- 3 ULA calcula $PC + 4$
- 4 $PC \leftarrow PC + 4$

Sinais de controle:

Sinal	Valor	Razão
lorD	0	Endereço vem do PC
MemRead	1	Lê memória
IRWrite	1	Salva instrução em IR
ALUSrcA	0	Entrada A = PC
ALUSrcB	01	Entrada B = 4
ALUOp	00	ADD ($PC + 4$)
PCWrite	1	Atualiza PC
PCSrc	00	Próximo PC = saída da ULA

Ações (todas as instruções):

- 1 UC decodifica opcode do IR
- 2 Banco de registradores lê rs e rt → resultados vão para **A** e **B**
- 3 ULA calcula endereço de branch *especulativamente*: $PC + (\text{SignExt} \ll 2)$
- 4 Resultado vai para **ALUOut**

Por que calcular branch no ID?

Mesmo que a instrução não seja beq, calculamos o endereço de branch **antecipadamente**. Se não for branch, simplesmente ignoramos o resultado. Isso **não custa nada** (a ULA estaria ociosa mesmo).

Sinais: ALUSrcA=0 (PC), ALUSrcB=11 ($\text{SignExt} \ll 2$), ALUOp=00 (ADD)

A partir do ciclo 3, o caminho **diverge** conforme o tipo:

R-type:

- ULA opera $A \text{ op } B$
- $ALUSrcA=1$ (A), $ALUSrcB=00$ (B)
- $ALUOp=10$ (depende de funct)
- Resultado $\rightarrow$ ALUOut

lw / sw:

- ULA calcula $A + \text{SignExt}$ (endereço)
- $ALUSrcA=1$, $ALUSrcB=10$
- $ALUOp=00$ (ADD)
- Endereço $\rightarrow$ ALUOut

beq:

- ULA calcula $A - B$
- $ALUSrcA=1$, $ALUSrcB=00$
- $ALUOp=01$ (SUB)
- Se $Zero=1$: $PC \leftarrow ALUOut$ (calculado no ID!)
- **Fim da instrução** (3 ciclos)

j:

- $PC \leftarrow IR[25:0] \ll 2$ concatenado com $PC[31:28]$
- **Fim da instrução** (3 ciclos)

lw – Ciclo 4 (MEM):

- Memória lê endereço do ALUOut
- lorD=1 (endereço de ALUOut)
- MemRead=1
- Dado → **MDR**

sw – Ciclo 4 (MEM):

- Memória escreve B no endereço de ALUOut
- lorD=1, MemWrite=1
- **Fim da instrução** (4 ciclos)

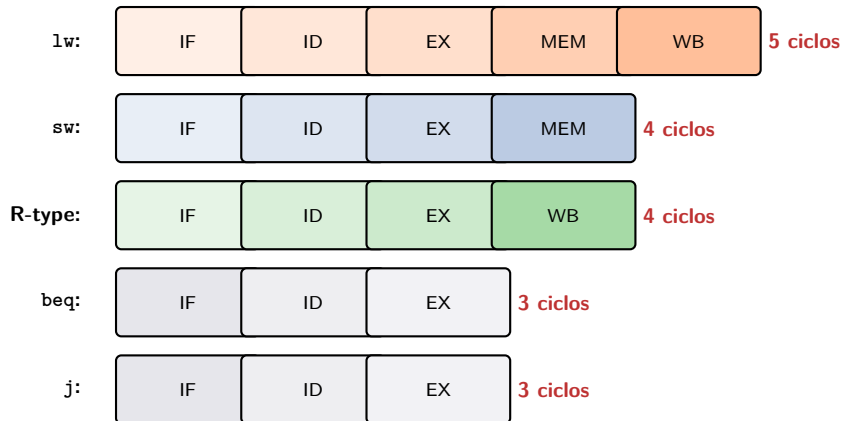
R-type – Ciclo 4 (WB):

- ALUOut → rd (banco de reg.)
- RegDst=1 (rd), MemtoReg=0 (ALUOut)
- RegWrite=1
- **Fim da instrução** (4 ciclos)

lw – Ciclo 5 (WB):

- MDR → rt (banco de reg.)
- RegDst=0 (rt), MemtoReg=1 (MDR)
- RegWrite=1
- **Fim da instrução** (5 ciclos)

Resumo: Ciclos por Instrução



Exercício em grupo (30 min)

Recebam os dois datapaths (mono e multi) impressos. Em grupo:

- 1 **Circulem** os componentes que aparecem no monociclo mas *não* no multiciclo
- 2 **Destaquem** os registradores intermediários que só existem no multiciclo
- 3 Para cada registrador (IR, MDR, A, B, ALUOut), expliquem: *por que ele é necessário?*
- 4 Preencham a tabela comparativa com vantagens e desvantagens

Pergunta para debate

Se a maioria das instruções em um programa são R-type (4 ciclos), e poucas são lw (5 ciclos), o multiciclo é mais vantajoso? Justifique com cálculo.

O que aprendemos:

- Motivação: limitações do monociclo
- Princípio: dividir em estágios menores
- Reutilização: 1 memória, 1 ULA
- Registradores intermediários: IR, MDR, A, B, ALUOut
- MUX extras para seleção
- Ciclos por instrução: 3–5

Para a Aula 18:

- Revisar os 5 estágios e suas ações
- Estudar: o que é uma FSM? (Módulo 2!)
- Pensar: quantos estados a FSM de controle do multiciclo precisa ter?

FSM de Controle e Análise de Desempenho

Metodologia: Simulação de FSM + Exercícios

- FSM da Unidade de Controle Multiciclo
- Diagrama de estados completo
- Sinais de controle estado por estado
- Rastreamento ciclo a ciclo de um programa
- Comparação de desempenho: monociclo vs multiciclo

Monociclo:

- Controle **combinacional**
- Opcode → sinais (instantâneo)
- Sinais fixos durante todo o ciclo
- Sem "memória" de estado

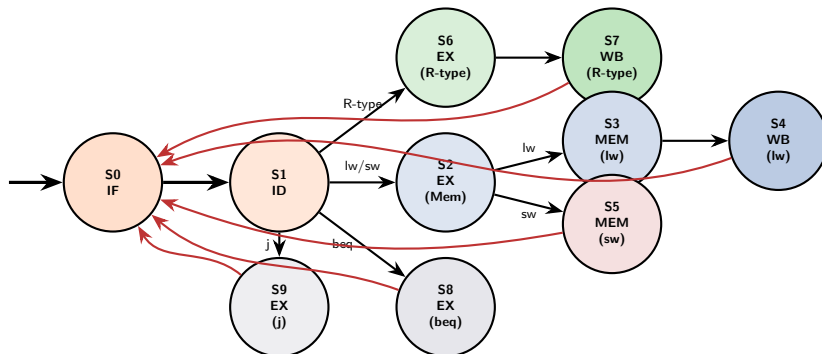
Multiciclo:

- Controle **sequencial (FSM)**
- Estado atual + opcode → sinais + próximo estado
- Sinais **mudam** a cada ciclo
- UC "lembra" em qual estágio está

Conexão com o Módulo 2

A UC do multiciclo é uma **FSM de Moore**: as saídas (sinais de controle) dependem **apenas do estado atual**. Cada estado corresponde a uma etapa da execução (IF, ID, EX, MEM, WB).

Diagrama de Estados da FSM



10 estados. Todas as instruções começam em S0 (IF) e terminam retornando a S0.

Estado	PC WRITE	PC WRITECOND	IORD	MEM READ	MEM WRITE	IR WRITE	ALU SrcA	ALU SrcB	ALU Op	REG WRITE	REG DST
S0 (IF)	1	0	0	1	0	1	0	01	00	0	X
S1 (ID)	0	0	X	0	0	0	0	11	00	0	X
S2 (EX-Mem)	0	0	X	0	0	0	1	10	00	0	X
S3 (MEM-lw)	0	0	1	1	0	0	X	XX	XX	0	X
S4 (WB-lw)	0	0	X	0	0	0	X	XX	XX	1	0
S5 (MEM-sw)	0	0	1	0	1	0	X	XX	XX	0	X
S6 (EX-R)	0	0	X	0	0	0	1	00	10	0	X
S7 (WB-R)	0	0	X	0	0	0	X	XX	XX	1	1
S8 (EX-beq)	0	1	X	0	0	0	1	00	01	0	X
S9 (EX-j)	1	0	X	0	0	0	X	XX	XX	0	X

PCWriteCond: Escreve PC apenas se flag Zero=1 (usado no beq).

PCSrc: S0 = resultado da ULA (PC+4), S8 = ALUOut (branch), S9 = Jump addr.

Ciclo	Estado	Ações
1	S0 (IF)	$IR \leftarrow \text{Mem}[PC]; PC \leftarrow PC+4$
2	S1 (ID)	$A \leftarrow \text{Reg}[\$s0]; B \leftarrow \text{Reg}[\$t0]; \text{ALUOut} \leftarrow PC + (\text{offset} \ll 2)$
3	S2 (EX-Mem)	$\text{ALUOut} \leftarrow A + \text{SignExt}(12) = \$s0 + 12$
4	S3 (MEM-lw)	$\text{MDR} \leftarrow \text{Mem}[\text{ALUOut}]$
5	S4 (WB-lw)	$\text{Reg}[\$t0] \leftarrow \text{MDR}$

Observação

O cálculo especulativo de branch (ciclo 2) é feito mas **descartado** para 1w. Não há custo porque a ULA estaria ociosa no ID mesmo.

Ciclo	Estado	Ações
1	S0 (IF)	$IR \leftarrow \text{Mem}[PC]; PC \leftarrow PC+4$
2	S1 (ID)	$A \leftarrow \text{Reg}[\$t1]; B \leftarrow \text{Reg}[\$t2]$
3	S6 (EX-R)	$ALUOut \leftarrow A + B$
4	S7 (WB-R)	$\text{Reg}[\$t0] \leftarrow ALUOut$

Somente 4 ciclos! No monociclo, add usava 775 ps (o mesmo que lw). No multiciclo, usa $4 \times 200 = 800$ ps. Hmm, quase igual...

Então por que o multiciclo é melhor?

Para programas com **mix variado** de instruções, o tempo médio é menor. Além disso, usa **menos hardware** (custo reduzido).

Ciclo	Estado	Ações
1	S0 (IF)	$IR \leftarrow \text{Mem}[PC]; PC \leftarrow PC+4$
2	S1 (ID)	$A \leftarrow \$s0; B \leftarrow \$s1; \text{ALUOut} \leftarrow PC + (\text{offset} \ll 2)$
3	S8 (EX-beq)	ULA: $A - B$; se $\text{Zero}=1$: $PC \leftarrow \text{ALUOut}$

Apenas 3 ciclos!

- O endereço de branch já foi calculado no ciclo 2 (ID) e está em ALUOut
- No ciclo 3, a ULA verifica se $A = B$ (subtração + flag Zero)
- Se $\text{Zero}=1$, PCWriteCond ativa e $PC \leftarrow \text{ALUOut}$
- Se $\text{Zero}=0$, PC permanece com $PC+4$ (calculado no IF)

Exercício (40 min)

Rastreie a execução **ciclo a ciclo** do programa abaixo:

End.	Instrução
0x00	addi \$t0, \$zero, 5
0x04	addi \$t1, \$zero, 3
0x08	add \$t2, \$t0, \$t1
0x0C	sw \$t2, 0(\$zero)
0x10	beq \$t0, \$t1, -3

Para cada instrução, preencha:

- 1 Quantos ciclos usa?
- 2 Quais estados da FSM percorre?
- 3 Quais sinais de controle estão ativos em cada estado?
- 4 Total de ciclos do programa inteiro?

Instrução	Tipo	Estados	Ciclos
addi \$t0, \$zero, 5	I (mem-like)	S0, S1, S2, S3*, S4*	4*
addi \$t1, \$zero, 3	I (mem-like)	S0, S1, S2, S3*, S4*	4*
add \$t2, \$t0, \$t1	R	S0, S1, S6, S7	4
sw \$t2, 0(\$zero)	sw	S0, S1, S2, S5	4
beq \$t0, \$t1, -3	beq	S0, S1, S8	3
Total:			19 ciclos

*addi usa o caminho de cálculo de endereço (como lw) mas com WriteBack direto de ALUOut. Depende da implementação: 4 ciclos (ALUOut→Reg) ou 5 ciclos (via MDR).

beq: \$t0=5 $\neq$ \$t1=3, então **não desvia**. PC continua sequencialmente.

Dados do exercício:

- 5 instruções, $T_{clock_mono} = 775$ ps, $T_{clock_multi} = 200$ ps

Monociclo:

$$T_{mono} = 5 \times 1 \times 775 = 3.875 \text{ ps}$$

Multiciclo:

$$T_{multi} = 19 \times 200 = 3.800 \text{ ps}$$

Resultado

Multiciclo é ligeiramente mais rápido ($-1,9\%$) mesmo com apenas 5 instruções!
A vantagem cresce com programas que têm mais beq/j (3 ciclos).

Mix típico (benchmark SPEC):

Tipo	Frequência	Ciclos (multi)
R-type	40%	4
lw	25%	5
sw	12%	4
beq/bne	18%	3
j/jal	5%	3

CPI médio (multiciclo):

$$\overline{CPI} = 0.40 \times 4 + 0.25 \times 5 + 0.12 \times 4 + 0.18 \times 3 + 0.05 \times 3 = \mathbf{4.02}$$

Comparação para $N = 1000$ instruções:

$$T_{mono} = 1000 \times 1 \times 775 = 775.000 \text{ ps}$$

$$T_{multi} = 1000 \times 4.02 \times 200 = 804.000 \text{ ps}$$

Quando o Multiciclo Vence?

Resultado surpreendente

No exemplo anterior, o multiciclo é **3,7% mais lento!**

Isso ocorre porque T_{clock_multi} (200 ps) não é suficientemente menor que T_{clock_mono} (775 ps) para compensar o $CPI > 1$.

O multiciclo vence quando:

- A diferença de complexidade entre instruções é **grande**
- Instruções rápidas (branch, jump) são **frequentes**
- O custo de hardware é uma preocupação (1 memória vs 2)

Importância real

O multiciclo não é usado na prática – ele é um passo didático fundamental para entender o **pipeline!** A ideia de dividir em estágios e reutilizar hardware é a **base** do pipeline.

Exercício individual (15 min)

Dado um programa com a seguinte distribuição:

R-type	30%
lw	30%
sw	10%
beq	25%
j	5%

- 1 Calcule o CPI médio do multiciclo
- 2 Para $N = 500$ instruções, calcule T_{exec} para monociclo e multiciclo
- 3 Qual implementação é mais rápida? Por quantos %?
- 4 Se $T_{clock_multi} = 150$ ps (ULA mais rápida), muda a resposta?

1. CPI médio:

$$\overline{CPI} = 0.30 \times 4 + 0.30 \times 5 + 0.10 \times 4 + 0.25 \times 3 + 0.05 \times 3 = 4.0$$

2. Tempos de execução ($N = 500$):

$$T_{mono} = 500 \times 1 \times 775 = 387.500 \text{ ps}$$

$$T_{multi} = 500 \times 4.0 \times 200 = 400.000 \text{ ps}$$

3. Monociclo é $\frac{400-387.5}{400} = 3.1\%$ mais rápido.

4. Com $T_{clock_multi} = 150 \text{ ps}$:

$$T_{multi} = 500 \times 4.0 \times 150 = 300.000 \text{ ps}$$

Agora o multiciclo é $\frac{387.5-300}{387.5} = 22.6\%$ mais rápido!

Atividade avaliativa

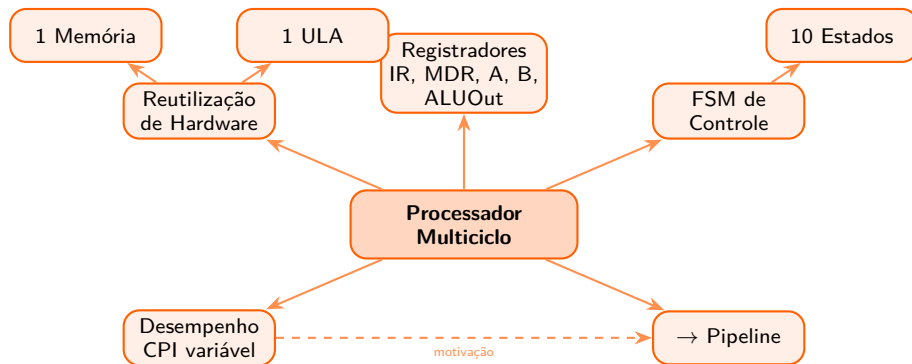
Peso: 5% da nota

Formato: Individual ou dupla

Prazo: 1 semana após a Aula 18

Entregar:

- ❶ **Tabela comparativa** detalhada: monociclo vs multiciclo (componentes, sinais, CPI, clock, vantagens, desvantagens)
- ❷ **Rastreamento ciclo a ciclo** de um programa de 8 instruções (fornecido pelo professor), preenchendo: estado da FSM, sinais ativos, conteúdo de IR, A, B, ALUOut, MDR a cada ciclo
- ❸ **Cálculo de desempenho:** CPI médio e comparação de T_{exec} para 3 mixes de instruções diferentes



Conceitos-chave:

- Dividir execução em estágios
- Reutilizar: 1 memória, 1 ULA
- 5 registradores intermediários
- FSM de Moore: 10 estados
- CPI variável: 3–5 ciclos
- T_{clock} = estágio mais lento
- $\overline{CPI}$ depende do mix
- Nem sempre mais rápido que mono

Competências desenvolvidas:

- Comparar datapaths
- Rastrear execução ciclo a ciclo
- Projetar FSM de controle
- Calcular CPI e T_{exec}
- Analisar trade-offs de projeto

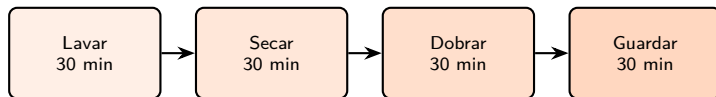
Próximo módulo:

Pipeline – a ideia de estágios levada ao extremo: **todas** as instruções em execução simultânea!

Conceito de Pipeline e Estágios MIPS

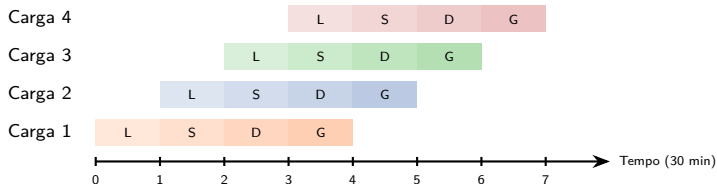
Metodologia: Analogia da Lavanderia + Simulação Corporal

- Analogia com a lavanderia
- Pipeline como "linha de montagem" de instruções
- Os 5 estágios: IF, ID, EX, MEM, WB
- Registradores de pipeline
- Speedup ideal e real
- Simulação corporal com a turma



Sem pipeline (sequencial): 4 cargas $\times$ 120 min = **480 min**

Com pipeline:



Com pipeline: 7 $\times$ 30 = **210 min** (speedup = $480/210 \approx 2.3\times$)

Ideia-chave

Enquanto uma instrução está no estágio EX, a **anterior** já está no MEM, a **seguinte** está no ID, e a **próxima** está sendo buscada (IF). Até 5 instruções em execução **simultânea**!

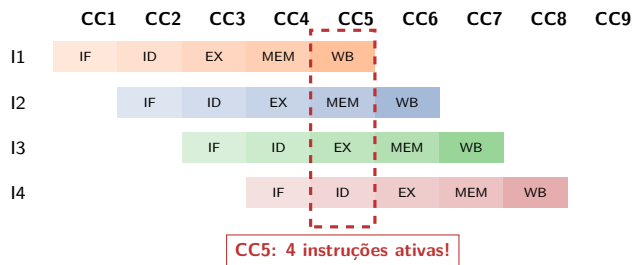
Os 5 estágios do pipeline MIPS:

Estágio	Nome	Componente principal
IF	Instruction Fetch	Memória de Instruções
ID	Instruction Decode	Banco de Registradores
EX	Execute	ULA
MEM	Memory Access	Memória de Dados
WB	Write Back	Banco de Registradores

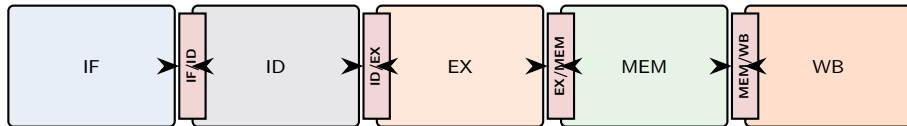
Cada estágio usa

um componente **diferente** ⇒ sem conflito!

Diagrama de Pipeline – Múltiplas Instruções



No ciclo CC5: I1 em WB, I2 em MEM, I3 em EX, I4 em ID. Pipeline **cheio**!



Função dos registradores de pipeline

Armazenam **todos os dados e sinais de controle** que uma instrução precisa passar de um estágio para o próximo. Sem eles, os dados de uma instrução seriam sobrescritos pela instrução seguinte.

Conteúdo de cada registrador:

- **IF/ID:** Instrução (32 bits), PC+4
- **ID/EX:** Valores de rs/rt, imediato sign-extended, sinais (EX, MEM, WB)
- **EX/MEM:** Resultado da ULA, endereço de branch, flag Zero, sinais (MEM, WB)
- **MEM/WB:** Dado da memória, resultado da ULA, sinais (WB)

Speedup ideal:

$$S_{ideal} = \frac{T_{sequencial}}{T_{pipeline}} = \frac{N \times k \times \tau}{(k + N - 1) \times \tau} = \frac{N \times k}{k + N - 1}$$

onde k = número de estágios, N = número de instruções, τ = período do clock.

Para $N \gg k$:

$$S_{ideal} \approx k = 5$$

Pipeline de 5 estágios: speedup máximo teórico = $5\times$!

Na prática, o speedup é menor devido a **hazards** (conflitos que impedem o fluxo contínuo do pipeline).

	Monociclo	Multiciclo	Pipeline
CPI	1	3–5	1 (ideal)
T_{clock}	Inst. mais lenta	Estágio mais lento	Estágio mais lento
Throughput	1 inst / T_{lw}	1 inst / $(\overline{CPI} \times \tau)$	1 inst / τ
Latência	T_{lw}	$\overline{CPI} \times \tau$	$k \times \tau$
Hardware	Duplicado	Reutilizado	Duplicado
Controle	Combinacional	FSM	Combinacional + hazard

Insight

Pipeline não reduz a **latência** de uma instrução individual (ela ainda passa por 5 estágios). Ele aumenta o **throughput** (vazão: instruções completadas por unidade de tempo).

Dinâmica (40 min)

5 alunos são os **estágios do pipeline**.

Cartões coloridos representam instruções passando pelos estágios.

A cada "CLOCK!" do professor, todos passam seus cartões adiante.

Papéis:

Aluno	Estágio
1	IF – busca a próxima instrução da pilha
2	ID – lê as informações do cartão, "decodifica"
3	EX – escreve o resultado da operação no cartão
4	MEM – consulta "memória" (post-its na parede) se necessário
5	WB – atualiza a "planilha de registradores" no quadro

Programa: 6 instruções simples (add, sub, lw, sw, add, sub).

Após pipeline cheio: **1 instrução completa por ciclo!**

Agora, troque o programa para:

```
1  add $t0, $t1, $t2
2  sub $t3, $t0, $t4    ← usa $t0!
3  and $t5, $t0, $t6    ← usa $t0!
```

Perguntas para a turma:

- 1 Quando a instrução 1 (add) escreve \$t0 no banco de registradores?
- 2 Quando a instrução 2 (sub) precisa ler \$t0?
- 3 Existe um conflito? O que acontece se não fizermos nada?

Descoberta

A instrução 2 lê \$t0 no estágio ID (ciclo 3), mas a instrução 1 só escreve \$t0 no estágio WB (ciclo 5). O valor lido está **desatualizado**! Isso é um **data hazard**.

O que aprendemos:

- Analogia da lavanderia
- 5 estágios: IF, ID, EX, MEM, WB
- Registradores de pipeline
- Speedup ideal $\approx k$
- Throughput vs latência
- Primeira observação de hazards

Para a próxima Aula:

- Estudar: 3 tipos de hazards (estrutural, dados, controle)
- Estudar: o que é "forwarding"?
- Pensar: como resolver o conflito que observamos na simulação?

Hazards e Técnicas de Resolução

Metodologia: Sala de Aula Invertida + Resolução de Hazards

- Hazards estruturais
- Hazards de dados (RAW, WAR, WAW)
- Hazards de controle
- Técnicas: forwarding, stalls, NOPs
- Cálculo de CPI efetivo com hazards

Tipo	Causa	Exemplo
Estrutural	Mesmo recurso usado por 2 instruções no mesmo ciclo	1 memória para IF e MEM (resolvido com Harvard)
Dados	Instrução depende de resultado ainda não disponível	add \$t0,... sub ..., \$t0,...
Controle	Próxima instrução depende do resultado de um branch	beq \$t0,\$t1,L ??? (qual buscar?)

No pipeline MIPS

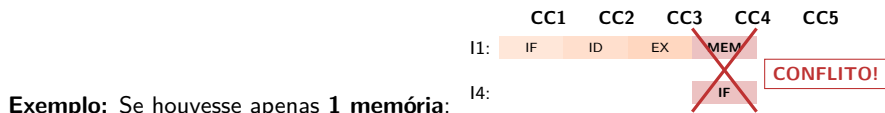
Estrutural: resolvido por design (memórias e portas separadas).

Dados: principal problema – resolvido com forwarding e stalls.

Controle: resolvido com prediction, stalls ou delayed branch.

Quando ocorre?

Dois estágios precisam do **mesmo hardware** no mesmo ciclo.

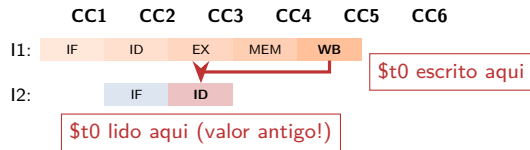


Exemplo: Se houvesse apenas **1 memória**:

Solução MIPS: Memórias separadas (Harvard) para instruções e dados.
No pipeline MIPS clássico, hazards estruturais **não ocorrem**.

O mais comum e problemático:

I1: add \$t0, \$t1, \$t2 (escreve \$t0 no WB – ciclo 5)
I2: sub \$t3, \$t0, \$t4 (lê \$t0 no ID – ciclo 3!)



I2 lê \$t0 no CC3, mas I1 só escreve no CC5. **2 ciclos de diferença!**

Forma mais simples (e mais lenta):

```
add $t0, $t1, $t2
nop      ← bolha
nop      ← bolha
sub $t3, $t0, $t4  agora $t0 já está disponível
```

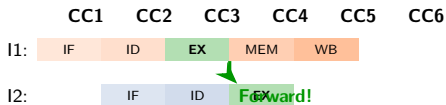
Problema

Cada NOP desperdiça 1 ciclo. Com muitas dependências, o pipeline fica cheio de bolhas e o speedup cai drasticamente.

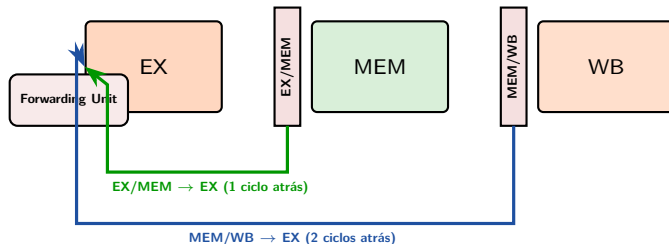
CPI efetivo = 1 + stalls por instrução

Ideia

O resultado da ULA já está disponível no final do estágio **EX** (registrador EX/MEM). Podemos "encaminhar" esse valor diretamente para a entrada da ULA no próximo ciclo, **sem esperar** pelo WB.



O resultado de I1 sai da ULA no CC3 e é encaminhado para a entrada da ULA de I2 no CC4. **Sem stall!**

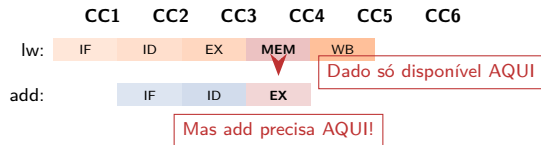


Dois caminhos de forwarding:

- **EX/MEM → EX:** resultado do ciclo anterior (1 instrução atrás)
- **MEM/WB → EX:** resultado de 2 ciclos atrás

A **Forwarding Unit** detecta dependências comparando os campos *rs/rt* da instrução no EX com os campos *rd/rt* das instruções nos estágios MEM e WB.

O Caso que Forwarding Não Resolve: Load-Use



lw \$t0, 0(\$s0) seguido de add \$t1, \$t0, \$t2

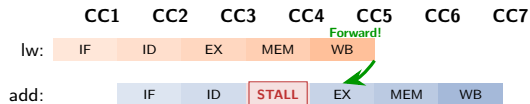
Problema Load-Use

O lw só produz o dado no final do **MEM** (CC4).

O add precisa do dado no início do **EX** (CC4).

Forward MEM→EX no mesmo ciclo? **Impossível** (muito tarde!).

Solução: **1 stall obrigatório** + forwarding MEM/WB→EX.



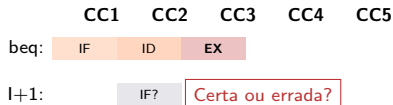
1 ciclo de stall (bolha) + forwarding de MEM/WB para EX. Penalidade: **1 ciclo**.

Reordenação de código (compiler trick)

O compilador pode mover uma instrução **independente** para o slot entre lw e o uso, eliminando o stall sem afetar a corretude!

Problema

Ao executar `beq`, o processador só sabe se deve desviar no estágio **EX** (ou MEM). Mas já buscou 1–2 instruções depois do branch!



Se o branch é **tomado**, `l+1` está errada e deve ser descartada (flush).

Se **não tomado**, `l+1` está correta e o pipeline continua normalmente.

- ❶ **Stall (sempre):** Após branch, congela pipeline até saber o resultado.
Penalidade: 1–2 ciclos por branch. Simples mas lento.
- ❷ **Predict Not Taken:** Assume que o branch **não** é tomado.
Continua buscando sequencialmente. Se errado, faz flush (1 ciclo perdido).
- ❸ **Predict Taken:** Assume que **é** tomado.
Busca do endereço-alvo. Se errado, flush.
- ❹ **Delayed Branch (MIPS):** A instrução **após** o branch é sempre executada ("branch delay slot"). O compilador coloca uma instrução útil ali.
- ❺ **Branch Prediction Dinâmica:** Hardware que "aprende" o padrão de branches.
Usa tabela de histórico. Acerto > 90%!

Exercício (30 min)

Para a sequência abaixo, identifique **todos** os hazards de dados e indique a solução (forwarding, stall, ou reordenação):

```
1:  lw $t0, 0($s0)
2:  add $t1, $t0, $t2
3:  sub $t3, $t0, $t4
4:  and $t5, $t1, $t3
5:  sw $t5, 4($s0)
```

Para cada hazard encontrado, responda:

- ① Qual o tipo? (RAW)
- ② Entre quais instruções?
- ③ Forwarding resolve? Se não, quantos stalls?
- ④ É possível reordenar para eliminar stalls?

Hazards identificados:

Dep.	Tipo	Solução	Stalls
I1→I2 (\$t0)	RAW (load-use!)	Stall + Forward MEM/WB→EX	1
I1→I3 (\$t0)	RAW	Forward MEM/WB→EX	0
I2→I4 (\$t1)	RAW	Forward EX/MEM→EX	0
I3→I4 (\$t3)	RAW	Forward EX/MEM→EX	0
I4→I5 (\$t5)	RAW	Forward EX/MEM→EX	0

Total: 1 stall (load-use entre I1 e I2). Todas as demais resolvidas por forwarding.

CPI efetivo

5 instruções, 1 stall $\Rightarrow$ 6 ciclos para completar.

$$CPI_{efetivo} = 1 + 1/5 = 1.2$$

$$CPI_{efetivo} = CPI_{ideal} + Stalls_{dados}/inst + Stalls_{controle}/inst$$

Exemplo:

- $CPI_{ideal} = 1$
- 20% das instruções são loads seguidos de uso imediato $\rightarrow$ 1 stall cada
- 15% das instruções são branches, predict not taken, 40% tomados $\rightarrow$ 1 ciclo de penalidade

$$Stalls_{dados} = 0.20 \times 1 = 0.20$$

$$Stalls_{controle} = 0.15 \times 0.40 \times 1 = 0.06$$

$$CPI_{efetivo} = 1 + 0.20 + 0.06 = \mathbf{1.26}$$

$$\text{Speedup real: } \frac{CPI_{ideal}}{CPI_{efetivo}} \times k = \frac{1}{1.26} \times 5 \approx 3.97 \times (\text{vs } 5 \times \text{ideal}).$$

O que aprendemos:

- 3 tipos de hazards
- RAW: principal hazard de dados
- Forwarding: EX/MEM e MEM/WB $\rightarrow$ EX
- Load-use: 1 stall obrigatório
- Reordenação pelo compilador
- Hazard de controle e soluções
- Cálculo de $CPI_{efetivo}$

Para a próxima Aula:

- Pesquisar: branch prediction estática vs dinâmica
- Estudar: tabelas de predição (BHT)
- Preparar argumentos para o debate

Branch Prediction e Análise de Desempenho

Metodologia: Debate Técnico + Estudo de Caso

- Branch prediction estática vs dinâmica
- Tabela de histórico de branches (BHT)
- Preditor de 1 bit e 2 bits
- Estudo de caso com dados de benchmarks
- Debate: estática vs dinâmica

Regras fixas, definidas em tempo de compilação:

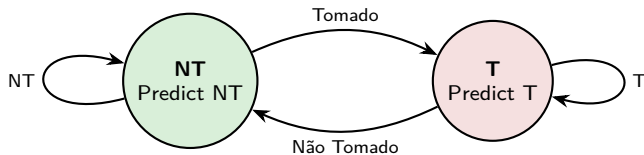
Estratégia	Regra	Taxa de acerto
Always Not Taken	Nunca desvia	~50–60%
Always Taken	Sempre desvia	~60–70%
BTFN	Backward Taken, Forward Not Taken	~65–75%
Profile-guided	Baseado em execução prévia	~75–85%

BTFN – Por que funciona?

Branches para trás geralmente são **loops** (tomados na maioria das iterações).
Branches para frente geralmente são **if-else** (nem sempre tomados).

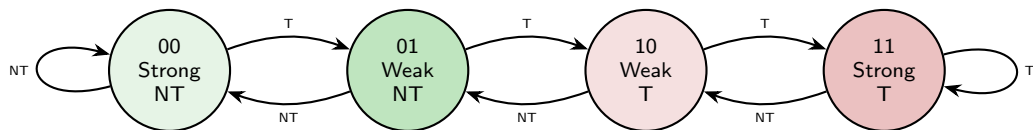
Ideia

Manter uma **tabela** (BHT – Branch History Table) indexada pelo endereço do branch. Cada entrada armazena 1 bit: última vez foi **Tomado (T)** ou **Não Tomado (NT)**.



Problema: loops!

Em um loop de 10 iterações, o preditor de 1 bit erra **2 vezes**: na primeira iteração (prediz NT, era T) e na última (prediz T, era NT). Taxa de acerto: $8/10 = 80\%$. Pode melhorar!



Vantagem

Precisa errar **2 vezes consecutivas** para mudar a predição.

No loop de 10 iterações: erra apenas 1 vez (a última).

Taxa: $9/10 = 90\%$!

Exemplo: Preditor de 2 Bits em um Loop

Loop executa 4 vezes (T, T, T, NT). Preditor inicia em "Strong T".

Iteração	Real	Estado	Predição	Acerto?
1	T	11 (Strong T)	T	✓
2	T	11	T	✓
3	T	11	T	✓
4 (saída)	NT	11 → 10	T	×

2ª execução do loop:

1	T	10 (Weak T)	T	✓
2	T	10 → 11	T	✓
3	T	11	T	✓
4 (saída)	NT	11 → 10	T	×

Taxa de acerto: $6/8 = 75\%$ (incluindo a 1ª execução). Em regime: $3/4 = 75\%$.

Taxas de acerto com preditor de 2 bits (4096 entradas):

Benchmark	Tipo	Taxa de acerto
gcc	Compilador	85%
bzip2	Compressão	91%
mcf	Otimização	78%
gobmk	IA (jogo de Go)	82%
hmmer	Bioinformática	97%
sjeng	Xadrez	86%
Média		~87%

Interpretação

Com preditor de 2 bits: apenas ~13% dos branches causam penalidade.
Processadores modernos usam preditores mais sofisticados (>95% de acerto).

Dados: 15% das instruções são branches. Penalidade de branch: 2 ciclos.

Estratégia	Miss rate	$CPI_{efetivo}$
Sem predição (stall)	100%	$1 + 0.15 \times 2 = 1.30$
Always Not Taken	$\sim 40\%$	$1 + 0.15 \times 0.40 \times 2 = 1.12$
Preditor 1 bit	$\sim 20\%$	$1 + 0.15 \times 0.20 \times 2 = 1.06$
Preditor 2 bits	$\sim 13\%$	$1 + 0.15 \times 0.13 \times 2 = 1.04$
Preditor moderno	$\sim 3\%$	$1 + 0.15 \times 0.03 \times 2 = 1.009$

Conclusão

A diferença entre "sem predição" ($CPI=1.30$) e preditor moderno ($CPI \approx 1.01$) é uma melhoria de $\sim 29\%$ no desempenho – **enorme!**

Regras (30 min)

Turma dividida em 2 grupos (posição sorteada):

- **Grupo A:** "Predição estática é suficiente e mais simples"
- **Grupo B:** "Predição dinâmica é essencial para desempenho"

Argumentos para estática:

- Hardware simples e barato
- Sem tabelas de predição (economia de área)
- BTFN funciona bem para loops
- Compilador pode otimizar o código

Argumentos para dinâmica:

- Acurácia muito superior ($>90\%$)
- Se adapta ao comportamento em tempo real
- Essencial em pipelines profundos
- Custo de hardware é aceitável hoje

Exercício (20 min)

Para o programa abaixo com pipeline de 5 estágios e forwarding:

```
1:  lw $t0, 0($s0)
2:  lw $t1, 4($s0)
3:  add $t2, $t0, $t1
4:  sw $t2, 8($s0)
5:  beq $t2, $zero, label
```

- 1 Identifique todos os hazards de dados
- 2 Quais são resolvidos por forwarding?
- 3 Quantos stalls são necessários?
- 4 Desenhe o diagrama de pipeline ciclo a ciclo
- 5 Calcule o $CPI_{efetivo}$ (assuma branch not taken, correto)

Hazards:

- I1→I3 (\$t0): load-use, **mas** I2 entre eles → forward MEM/WB→EX. **Sem stall!**
- I2→I3 (\$t1): load-use direto! **1 stall.**
- I3→I4 (\$t2): forward EX/MEM→EX. Sem stall.
- I3→I5 (\$t2): forward MEM/WB→EX. Sem stall.

Total: 5 instruções + 1 stall = **6 ciclos** (após preenchimento do pipeline)

$$CPI_{efetivo} = 1 + \frac{1}{5} = 1.2$$

Otimização possível

Reordenar: mover I4 (sw) para entre I2 e I3 → elimina o stall!

$CPI_{efetivo} = 1.0$ (pipeline ideal!)

Lista de exercícios

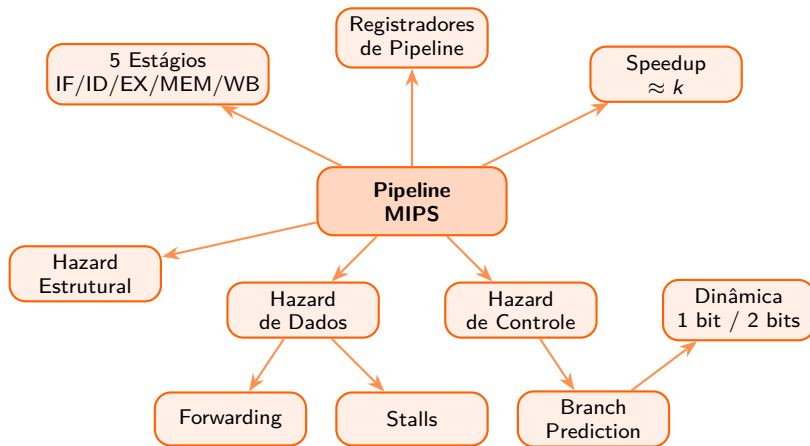
Peso: 10% da nota

Formato: Individual

Prazo: 1 semana após a Aula 21

Conteúdo (10 exercícios):

- 1 Desenhar diagrama de pipeline para 6 instruções
- 2 Identificar hazards em 3 sequências diferentes
- 3 Aplicar forwarding e indicar stalls restantes
- 4 Calcular CPI efetivo com dados e branch hazards
- 5 Comparar desempenho: pipeline vs monociclo vs multiciclo
- 6 Simular preditor de 2 bits para sequência de 12 branches
- 7 Reordenar código para eliminar stalls
- 8 Calcular speedup com diferentes taxas de branch



Conceitos-chave:

- Pipeline: "linha de montagem"
- 5 estágios, registradores entre eles
- Speedup ideal $\approx k = 5$
- Throughput vs latência
- 3 hazards: estrutural, dados, controle
- RAW resolvido com forwarding
- Load-use: 1 stall obrigatório
- Branch prediction: 1 bit, 2 bits
- $CPI_{ef} = 1 + \text{stalls}/\text{inst}$

Competências desenvolvidas:

- Desenhar diagramas de pipeline
- Identificar e resolver hazards
- Aplicar forwarding
- Calcular CPI efetivo e speedup
- Simular preditores de branch
- Reordenar código

Próximo módulo:

Exceções e Interrupções – o que acontece quando algo dá errado no pipeline?

Exceções e Interrupções no MIPS

Metodologia: Estudo de Caso Prático + Simulação

- Conceitos: exceção vs interrupção
- Registradores especiais: EPC e Cause
- Fluxo de tratamento de exceções
- Impacto no pipeline: flush de instruções
- Tipos: overflow, instrução inválida, syscall
- Estudo de caso: "Seu programa gerou um overflow"
- Programando um handler no MARS

Cenário

Seu programa MIPS está executando normalmente e de repente:

- Uma instrução add gera **overflow**
- Ou uma instrução com opcode inválido é encontrada
- Ou um dispositivo de E/S precisa de atenção urgente

O que o processador faz?

Resposta

O processador **interrompe** a execução normal, **salva o contexto** (onde parou), **desvia** para um código especial (handler) que trata o problema, e depois **retoma** a execução.

Característica	Exceção	Interrupção
Origem	Interna (CPU)	Externa (dispositivo)
Causa	Instrução em execução	Evento de E/S
Momento	Síncrona (previsível)	Assíncrona (qualquer momento)
Exemplos	Overflow, opcode inválido, page fault, syscall	Teclado, disco, timer, rede, USB

Terminologia no MIPS

O MIPS usa o termo "**exceção**" para ambos os casos. Outros autores (Intel) usam "interrupção" para eventos externos e "exceção" para internos. Patterson & Hennessy tratam tudo como exceção.

Código	Tipo	Descrição
0	Int	Interrupção externa (hardware)
4	AdEL	Endereço inválido no load (desalinhado)
5	AdES	Endereço inválido no store (desalinhado)
6	IBE	Erro no barramento (busca de instrução)
7	DBE	Erro no barramento (dados)
8	Sys	Syscall (syscall)
9	Bp	Breakpoint (break)
10	RI	Instrução reservada (opcode inválido)
12	Ov	Overflow aritmético (add, sub)

Os 3 tipos mais comuns na disciplina

Overflow (12): add/sub com resultado fora do intervalo.

Instrução inválida (10): opcode não reconhecido.

Syscall (8): chamada de sistema voluntária.

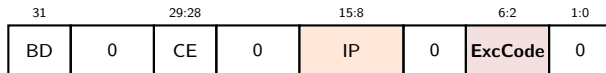
O MIPS usa registradores do Coprocessador 0 (CP0) para exceções:

Registrador	CP0 #	Função
EPC	14	Exception Program Counter – endereço da instrução que causou a exceção
Cause	13	Código da causa da exceção (campo ExcCode nos bits [6:2])
Status	12	Máscara de interrupções, modo (kernel/user), habilitação global
BadVAddr	8	Endereço que causou erro de acesso

Instruções de acesso ao CP0:

`mfc0 $t0, $14` (move from CP0: $\$t0 \leftarrow \text{EPC}$)

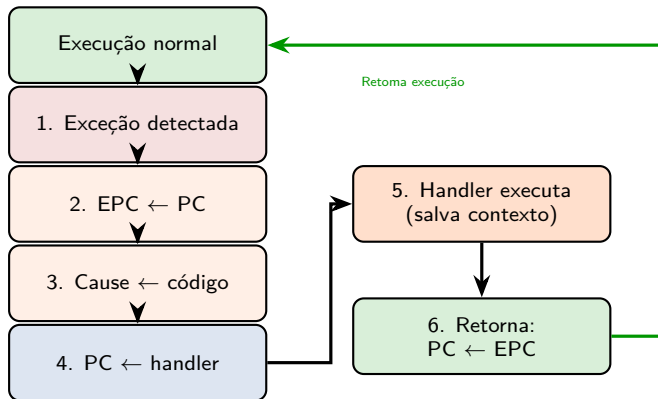
`mtc0 $t0, $12` (move to CP0: $\text{Status} \leftarrow \$t0$)



- **BD (bit 31):** 1 se a exceção ocorreu no *branch delay slot*
- **IP (bits 15:8):** Interrupt Pending – quais interrupções estão pendentes
- **ExcCode (bits 6:2):** Código da exceção (os valores da tabela anterior)

Para ler o ExcCode no handler:

```
mfc0 $t0, $13      # $t0 ← Cause
srl $t0, $t0, 2     # desloca para alinhar ExcCode
andi $t0, $t0, 0x1F # mascara 5 bits
```



Duas abordagens para localizar o handler:

Endereço fixo (MIPS):

- Todas as exceções desviam para **um mesmo endereço**: 0x80000180
- O handler lê o Cause para decidir o que fazer
- Mais simples (1 ponto de entrada)

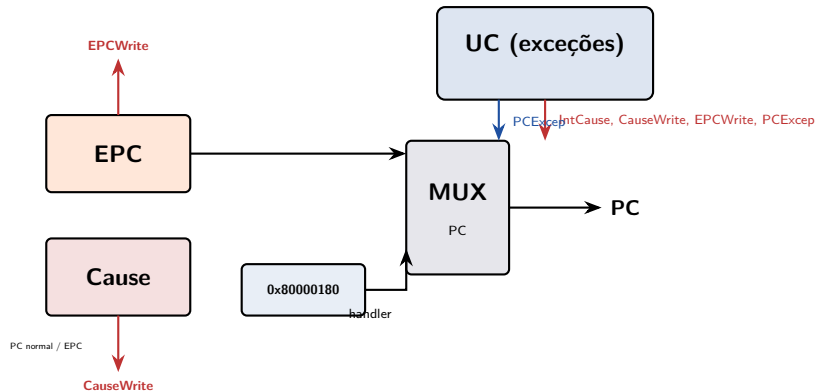
Vectored (Intel x86):

- Cada tipo de exceção tem um endereço **diferente**
- Tabela de vetores (IDT) na memória
- Desvio direto sem leitura do Cause
- Mais rápido para muitos tipos

No MARS

O handler é carregado no endereço 0x80000180 do segmento *kernel text*. O MARS configura isso automaticamente.

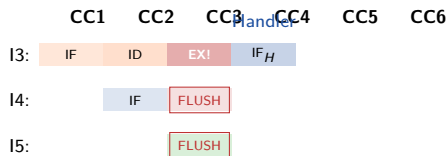
Adições ao datapath para suportar exceções:



Novos sinais: EPCWrite, CauseWrite, IntCause (seleciona o código), PCExcep (seleciona o handler no MUX do PC).

Cenário

Instrução I3 gera overflow no estágio EX. Mas I4 (no ID) e I5 (no IF) já estão no pipeline! O que fazer com elas?



Solução: Flush (limpar) as instruções posteriores:

- 1 Substituir instruções nos estágios IF e ID por **NOPs** (bolhas)
- 2 Salvar PC de I3 no EPC
- 3 $PC \leftarrow$ endereço do handler
- 4 Pipeline recomeça buscando o handler

Exceção precisa (MIPS):

- Todas as instruções **antes** da exceção completam normalmente
- Nenhuma instrução **depois** da exceção tem efeito
- EPC aponta **exatamente** para a instrução causadora
- Mais difícil de implementar
- Essencial para SO e debugging

Exceção imprecisa:

- Instruções posteriores podem ter executado parcialmente
- Pipeline não garante ordem exata
- Mais simples de implementar
- Dificulta o tratamento pelo SO
- Raro em processadores modernos

No pipeline MIPS de 5 estágios

Exceções são **precisas**: o pipeline é limpo (flush) e o estado é consistente. Em processadores out-of-order, exceções precisas são garantidas pelo **Reorder Buffer** (Módulo 13).

Exceções podem ocorrer em estágios diferentes:

Estágio	Exceção	Exemplo
IF	Erro de endereço	PC desalinhado ou inválido
ID	Instrução inválida	Opcode não reconhecido
EX	Overflow aritmético	add com resultado fora do range
MEM	Erro de endereço	lw/sw em endereço desalinhado
MEM	Page fault	Página não presente na memória

E se duas exceções ocorrem no mesmo ciclo?

Quando I1 está no MEM e I3 está no ID, ambas podem gerar exceção.

Regra: Tratar a exceção da instrução **mais antiga** primeiro (I1). As posteriores são descartadas (flush).

Cenário (atividade guiada)

O programa abaixo é executado no MARS:

```
0x00400000:  li $t0, 2147483647    # MAX_INT
0x00400008:  addi $t0, $t0, 1      # overflow!
0x0040000C:  li $v0, 1
0x00400010:  move $a0, $t0
0x00400014:  syscall
```

O que acontece na linha 2?

Resposta:

- ❶ ULA detecta overflow ($C_{30} \oplus C_{31} = 1$)
- ❷ $EPC \leftarrow 0x00400008$ (endereço da instrução addi)
- ❸ $Cause \leftarrow$ código 12 (overflow)
- ❹ $PC \leftarrow 0x80000180$ (handler padrão)

O que o Handler Padrão do MARS Faz?

Se não houver handler customizado:

- 1 MARS exibe mensagem: *"Arithmetic Overflow"*
- 2 Execução é **abortada**

Com handler customizado, podemos:

- Imprimir uma mensagem de erro amigável
- Ignorar a instrução e continuar (skip)
- Registrar o erro em um log
- Tentar corrigir (ex.: saturar o valor)
- Abortar graciosamente

Vamos escrever um handler!

Objetivo: detectar o tipo de exceção, imprimir uma mensagem, e retomar a execução na instrução seguinte.

Estrutura do handler (carregado em 0x80000180):

```
.ktext 0x80000180      # Seção do kernel
    mfc0 $k0, $13      # $k0 ← Cause
    srl $k0, $k0, 2     # Alinha ExcCode
    andi $k0, $k0, 0x1F # Mascara 5 bits

    beq $k0, 12, overflow_h # Código 12 = overflow
    beq $k0, 10, invalid_h  # Código 10 = instr. inválida
    j default_h             # Outros casos

overflow_h:
    li $v0, 4            # print string
    la $a0, msg_ov       # mensagem
    syscall
    j return
```

```
invalid_h:
    li $v0, 4
    la $a0, msg_inv
    syscall
    j return

default_h:
    li $v0, 4
    la $a0, msg_def
    syscall

return:
    mfc0 $k1, $14      # $k1 ← EPC
    addi $k1, $k1, 4    # Pula a instrução problemática
    mtc0 $k1, $14      # Atualiza EPC
    eret               # Retorna da exceção
```

eret – Exception Return

eret faz: $PC \leftarrow EPC$ e restaura o modo de execução. Diferente de jr \$ra, pois também restaura bits de Status.

Registradores \$k0 e \$k1 – Por que Existem?

Problema

O handler de exceção precisa usar registradores para trabalhar. Mas se usar \$t0 ou \$s0, pode corromper o estado do programa!

Solução: \$k0 (\$26) e \$k1 (\$27)

- Reservados para uso **exclusivo do kernel** (SO / handler)
- O programa do usuário **nunca** deve usar \$k0/\$k1
- O handler pode usá-los **sem salvar/restaurar**
- Se o handler precisar de mais registradores, deve salvar na pilha do kernel

Convenção

É por isso que a convenção MIPS reserva \$k0 e \$k1: para garantir que o handler tenha registradores "seguros" disponíveis imediatamente, sem acessar a memória.

Exercício (40 min)

- ❶ Abra o MARS e crie dois arquivos:
 - programa.asm: programa que gera overflow
 - handler.asm: handler de exceção customizado
- ❷ Configure o MARS: Settings → Exception Handler e aponte para handler.asm
- ❸ Execute o programa e observe:
 - A mensagem de erro é exibida pelo handler?
 - O programa continua após o overflow?
 - Qual o valor de EPC?
- ❹ Modifique o programa para gerar **instrução inválida**: `.word 0xFFFFFFFF` (opcode inválido)
- ❺ Verifique se o handler trata ambos os casos corretamente

```
.data
msg1: .asciiz "Antes do overflow\n"
msg2: .asciiz "Depois do overflow\n"

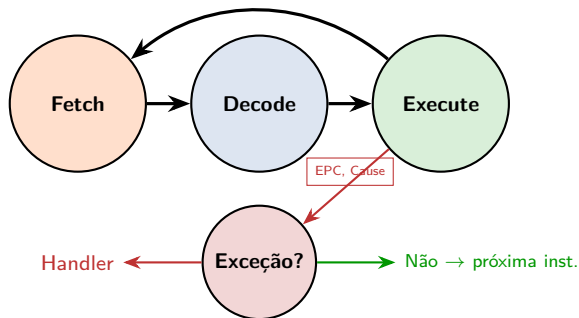
.text
main:
    li $v0, 4
    la $a0, msg1
    syscall                                # Imprime "Antes"

    li $t0, 0x7FFFFFFF
    addi $t0, $t0, 1                      # MAX_INT
                                           # OVERFLOW!

    li $v0, 4
    la $a0, msg2
    syscall                                # Imprime "Depois"

    li $v0, 10
    syscall                                # Exit
```

Com handler: imprime "Antes", mensagem de overflow, depois "Depois". Continua!



Visão completa: A cada instrução, o processador verifica se houve exceção. Se sim, desvia para o handler. Se não, continua normalmente.

Módulo	Conexão com Exceções
M1 (Circuitos)	Deteção de overflow: $C_{n-1} \oplus C_n$ na ULA
M2 (Sequencial)	Registradores EPC/Cause são flip-flops
M3 (Arquitetura)	Ciclo de instrução inclui verificação de exceção
M4 (MIPS ISA)	syscall, break, registradores CP0
M5 (Monociclo)	Adição de EPC, Cause e MUX ao datapath
M6 (Multiciclo)	Estado extra na FSM para tratamento
M7 (Pipeline)	Flush de instruções posteriores à exceção

Tudo conectado!

Exceções são o ponto onde **hardware** (deteção de overflow, flush) e **software** (handler, SO) se encontram.

Mini-relatório

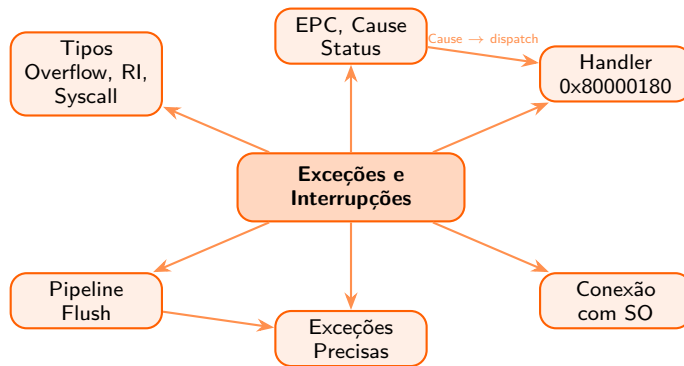
Peso: 3% da nota

Formato: Individual

Prazo: 1 semana após a Aula 22

Conteúdo:

- ❶ **Diagrama de fluxo** do tratamento de exceção (desde a detecção até o retorno com `eret`)
- ❷ **Código do handler** funcional no MARS (tratando overflow e instrução inválida)
- ❸ **Screenshots** da execução mostrando: EPC, Cause e a mensagem do handler
- ❹ **Respostas:**
 - Qual a diferença entre exceção e interrupção?
 - Por que exceções precisas são importantes?
 - O que acontece no pipeline quando ocorre exceção?



Conceitos-chave:

- Exceção (interna) vs Interrupção (externa)
- EPC: endereço da instrução causadora
- Cause: código do tipo de exceção
- Handler em endereço fixo (0x80000180)
- Fluxo: detectar → salvar → handler → retornar
- Pipeline: flush de instruções posteriores
- Exceções precisas
- \$k0/\$k1 reservados para kernel

Competências desenvolvidas:

- Entender o fluxo de tratamento
- Escrever handler no MARS
- Usar mfc0/mtc0/eret
- Analisar impacto no pipeline
- Conectar HW (detecção) com SW (handler)

Próximo tópico:

Entrada e Saída – polling, interrupções de E/S e DMA!

Conceitos de Entrada e Saída

Metodologia: Aprendizagem Baseada em Problemas (PBL)

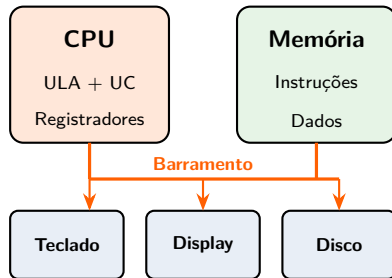
- Problema motivador: teclado vs USB
- Modelo de E/S mapeada em memória (MMIO)
- Polling vs Interrupções
- DMA (Direct Memory Access)
- Interfaces, controladores e buffers
- Gargalos de E/S e benchmarks

Cenário

Por que o teclado do seu computador responde **instantaneamente** quando você digita, mas transferir um arquivo grande para um pendrive USB demora **vários segundos**?

Discutam em grupo (10 min):

- 1 O teclado envia 1 byte por tecla. O USB envia milhões de bytes. Isso explica tudo?
- 2 O processador fica "esperando" o teclado o tempo todo?
- 3 E durante a transferência USB, o processador fica parado?
- 4 Existe um mecanismo para transferir dados *sem* o processador?



Problema fundamental:

- CPU opera em GHz (nanossegundos)
- Dispositivos de E/S operam em kHz–MHz (microssegundos a milissegundos)
- Diferença de velocidade de 10^3 a 10^6 vezes!

Duas abordagens:

E/S com portas dedicadas (x86):

- Instruções especiais: IN, OUT
- Espaço de endereçamento separado
- Portas de E/S numeradas (0–65535)
- Hardware mais complexo

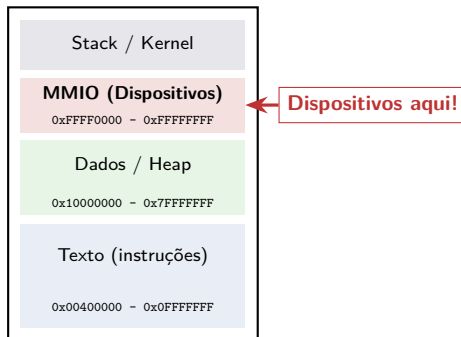
E/S mapeada em memória – MMIO (MIPS):

- Dispositivos recebem **endereços de memória**
- Acessados com 1w e sw normais
- Sem instruções especiais
- Mais simples e uniforme

MIPS usa MMIO

No MIPS, cada dispositivo é mapeado em uma faixa de endereços. Ler de um endereço "especial" acessa o dispositivo, não a RAM.

Espaço de Endereçamento



Endereços MMIO no MARS:

Dispositivo	Endereço
Receiver Control (teclado ready)	0xFFFF0000
Receiver Data (teclado dado)	0xFFFF0004
Transmitter Control (display ready)	0xFFFF0008
Transmitter Data (display dado)	0xFFFF000C

Receiver (Teclado):

0xFFFF0000 – Control

Bit 0: **Ready**
(1 = tecla pressionada)

0xFFFF0004 – Data

Bits 7:0: **Caractere**
(código ASCII)

Transmitter (Display):

0xFFFF0008 – Control

Bit 0: **Ready**
(1 = pode enviar)

0xFFFF000C – Data

Bits 7:0: **Caractere**
(a ser exibido)

Protocolo

Teclado: Espere Ready=1, então leia Data (Ready volta para 0).

Display: Espere Ready=1, então escreva Data (Ready volta para 0).

Definição

A CPU fica em um **loop** verificando repetidamente se o dispositivo está pronto. Quando o bit Ready = 1, a CPU lê ou escreve o dado.

Pseudocódigo para ler do teclado:

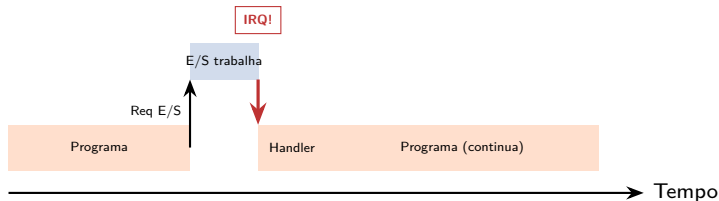
```
poll:
    lw $t0, 0xFFFF0000    # Lê Receiver Control
    andi $t0, $t0, 1       # Isola bit Ready
    beq $t0, $zero, poll   # Se não pronto, repete
    lw $t1, 0xFFFF0004    # Lê caractere
```

Problema

Enquanto espera, a CPU está **100% ocupada** no loop!
Se o teclado leva 100 ms para responder e a CPU opera a 1 GHz, são ~100 milhões de iterações desperdiçadas.

Definição

O dispositivo **avisa** a CPU quando está pronto, gerando uma **interrupção**. A CPU continua executando normalmente até receber o aviso.



A CPU fica **livre** enquanto o dispositivo trabalha. Só é interrompida quando necessário.

Característica	Polling	Interrupção
Uso da CPU	100% (espera ativa)	Livre durante E/S
Latência	Muito baixa (imediata)	Overhead do handler
Complexidade HW	Simples	Controlador de interrupts
Complexidade SW	Loop simples	Handler + prioridades
Ideal para	Dispositivos rápidos	Dispositivos lentos
Throughput	Baixo (1 dispositivo)	Alto (múltiplos)

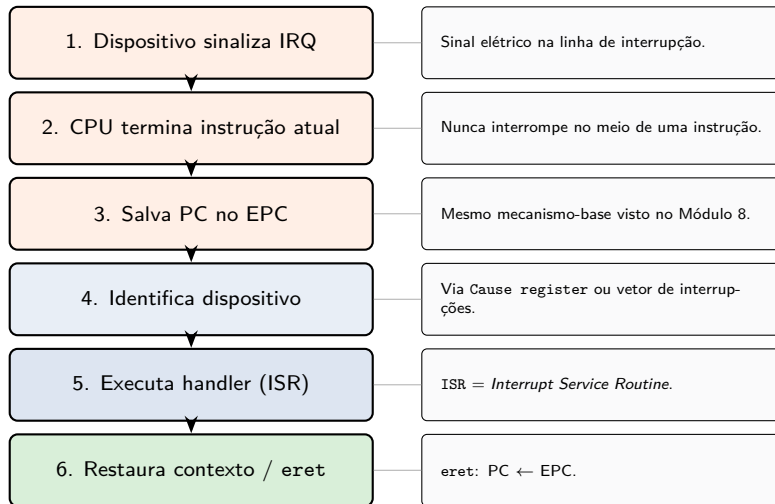
Na prática: abordagem híbrida

Polling: usado quando a resposta é esperada em poucos ciclos (registradores de status).

Interrupções: usado para dispositivos lentos (teclado, rede, disco).

Muitos sistemas alternam: interrupção para acordar, polling para drenar buffer.

Interrupções – Fluxo Detalhado



Problema

Para transferir 1 MB de dados do disco para a RAM, a CPU teria que executar ~ 250.000 instruções lw/sw. Mesmo com interrupções, cada byte exige intervenção da CPU.

Solução: DMA

Um controlador dedicado (**DMA controller**) transfere dados **diretamente** entre o dispositivo e a memória, **sem** passar pela CPU.

A CPU apenas:

- 1 Programa o DMA: endereço de origem, destino e tamanho
- 2 Dá o comando "iniciar transferência"
- 3 Continua executando outras instruções
- 4 Recebe uma **interrupção** quando a transferência termina

Comparação: Polling vs Interrupção vs DMA

Polling:

CPU 100% ocupada esperando + transferindo

Interrupção:

Programa

ISR

...

DMA:

Prog

CPU livre! (DMA transfere em background)

ISR

→ Tempo

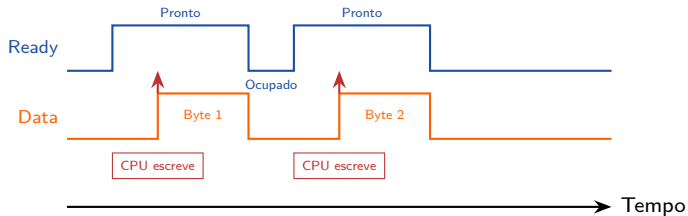
	Polling	Interrupção	DMA
CPU livre	Não	Parcialmente	Sim
HW extra	Nenhum	Ctrl. de Int.	Ctrl. DMA
Ideal para	Baixa latência	E/S moderada	Grandes blocos



O controlador de E/S:

- Traduz comandos da CPU para sinais do dispositivo
- Armazena dados em **buffers** temporários
- Gerencia o protocolo de comunicação (*handshaking*)
- Sinaliza Ready/Busy e gera interrupções
- Exemplos: controlador de disco SATA, controlador USB, controlador de rede

Exemplo: CPU envia byte para display via MMIO



- 1 CPU verifica Ready = 1 (polling ou interrupção)
- 2 CPU escreve dado no registrador Data
- 3 Controlador seta Ready = 0 (ocupado processando)
- 4 Controlador processa o byte e seta Ready = 1 novamente

Taxa de transferência de dispositivos comuns:

Dispositivo	Taxa	Latência
Teclado	10 bytes/s	100 ms
Mouse	200 bytes/s	5 ms
Ethernet 1 Gbps	125 MB/s	< 1 ms
SSD NVMe	3–7 GB/s	10–50 μ s
HDD	100–200 MB/s	5–10 ms
USB 3.0	625 MB/s	< 1 ms
PCIe 4.0 (x16)	32 GB/s	< 1 μ s

Lei de Amdahl aplicada a E/S

Se 10% do tempo é E/S e melhoramos a CPU em 10 $\times$:

$$\text{Speedup} = \frac{1}{0.10 + 0.90/10} = \frac{1}{0.19} \approx 5.3\times \text{ (não } 10\times\text{!)}$$

O E/S é o **gargalo** – melhorar só a CPU não basta.

Por que o teclado responde instantaneamente?

- Poucos bytes por interação (1 tecla = 1 byte)
- CPU atende via **interrupção** rapidamente
- Latência < limiar de percepção humana (~ 50 ms)

Por que a cópia para USB demora?

- Grandes volumes de dados (MB/GB)
- Transferência via **DMA**, mas limitada pela banda do USB
- USB 2.0: 60 MB/s $\rightarrow$ 1 GB leva ~ 17 s
- USB 3.0: 625 MB/s $\rightarrow$ 1 GB leva $\sim 1,6$ s
- CPU **não** é o gargalo – é o dispositivo!

O que aprendemos:

- MMIO: dispositivos com endereços de memória
- Registradores de controle e dados
- Polling: simples mas desperdiça CPU
- Interrupções: CPU livre, handler trata
- DMA: transferência sem CPU
- Controladores, buffers, handshaking
- Gargalos de E/S e Lei de Amdahl

Para a próxima Aula:

- Instalar MARS (se ainda não tem)
- Habilitar MMIO: Tools → Keyboard and Display MMIO Simulator
- Revisar: loops, lw, sw, andi
- Pensar: como implementar um "echo" de teclado?

Laboratório: MMIO no MARS

Metodologia: Laboratório Hands-On

- Configuração do MMIO no MARS
- Programa 1: Echo de teclado (polling)
- Programa 2: Display formatado
- Desafio: Buffer circular com eco

Passo a passo:

- 1 Abra o MARS
- 2 Menu Tools → Keyboard and Display MMIO Simulator
- 3 Janela aparece com dois painéis:
 - **Keyboard:** campo para digitar (input)
 - **Display:** área que mostra saída (output)
- 4 Clique em "**Connect to MIPS**" na janela do MMIO
- 5 Agora os endereços 0xFFFF0000-0xFFFF000C estão ativos

Importante

O simulador MMIO deve estar **conectado** antes de executar o programa. Se não conectar, os acessos a 0xFFFF0000 lerão lixo!

Objetivo: Ler teclas e ecoar no display (polling).

```
.text
main:
wait_key:
    lw $t0, 0xFFFF0000    # Receiver Control
    andi $t0, $t0, 1      # Bit Ready
    beq $t0, $zero, wait_key # Loop até tecla
    lw $t1, 0xFFFF0004    # Lê caractere

wait_disp:
    lw $t0, 0xFFFF0008    # Transmitter Control
    andi $t0, $t0, 1      # Bit Ready
    beq $t0, $zero, wait_disp # Loop até pronto
    sw $t1, 0xFFFF000C    # Envia ao display

    j wait_key             # Volta para o teclado
```

Teste: Digite no campo do teclado e veja as letras aparecerem no display!

Modificação (15 min)

Altere o programa de echo para que:

- 1 Se o caractere for minúsculo (a–z), converta para maiúsculo
- 2 Se for maiúsculo ou outro caractere, ecoe sem alterar
- 3 Dica: 'a' = 97, 'z' = 122, diferença = 32

Lógica de conversão:

```
slti $t2, $t1, 97      # char < 'a'?  
bne $t2, $zero, no_conv # Se sim, não converte  
slti $t2, $t1, 123     # char < 'z'+1?  
beq $t2, $zero, no_conv # Se não, não converte  
addi $t1, $t1, -32     # Converte para maiúscula  
no_conv:
```

Objetivo: Imprimir uma string do segmento de dados no display MMIO.

```
.data
msg: .asciiz "Hello, MMIO!"
.text
main:
    la $t0, msg                # Endereço da string
loop:
    lb $t1, 0($t0)             # Lê byte da string
    beq $t1, $zero, done       # Null terminator?
wait:
    lw $t2, 0xFFFF0008         # Display pronto?
    andi $t2, $t2, 1
    beq $t2, $zero, wait
    sw $t1, 0xFFFF000C         # Envia caractere
    addi $t0, $t0, 1           # Próximo byte
    j loop
done:
    li $v0, 10
    syscall
```

Desafio (40 min)

Implemente um **buffer circular** de 16 caracteres:

- 1 Leia caracteres do teclado via polling
- 2 Armazene em um buffer de 16 bytes no segmento de dados
- 3 Ecoe cada caractere no display
- 4 Quando o buffer enche, comece a sobrescrever do início
- 5 Se o usuário digitar `Enter` (ASCII 10), imprima todo o conteúdo do buffer no display e limpe

Dicas:

- Use dois ponteiros: `head` (onde escrever) e `count`
- Para circular: `head = (head + 1) % 16` → use `andi head, head, 0xF`
- `sb` (store byte) e `lb` (load byte) para acessar o buffer

Perguntas para discussão (10 min):

- ❶ No programa de echo com polling, a CPU está fazendo algo útil enquanto espera a tecla? (*Não – loop infinito*)
- ❷ Se quiséssemos que a CPU calculasse algo enquanto espera teclas, o polling funcionaria? (*Não – precisaria de interrupções*)
- ❸ O MARS suporta interrupções de MMIO. Como seria o handler? (*Receiver gera IRQ; handler lê dado e ecoa*)
- ❹ Para transferir um arquivo de 1 MB, polling seria viável? (*Não – usaríamos DMA*)

Conexão com sistemas reais

Drivers de dispositivo em sistemas operacionais implementam exatamente essas técnicas: registradores MMIO, interrupções e DMA. O que fizemos no MARS é uma versão simplificada do que um driver faz!

Programa MIPS + Respostas PBL

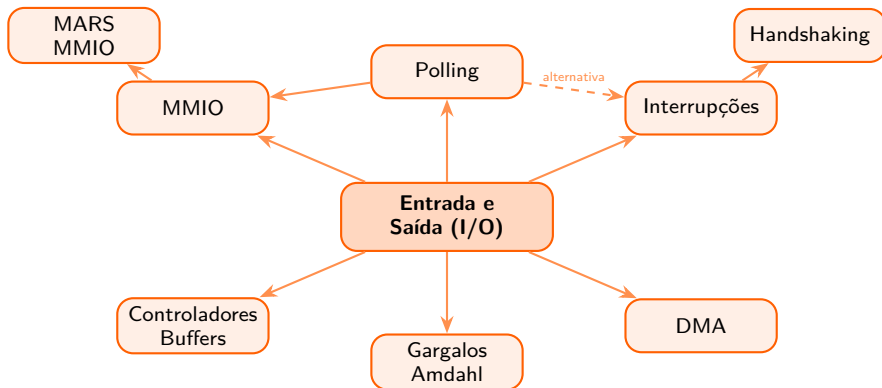
Peso: 5% da nota

Formato: Individual ou dupla

Prazo: 1 semana após a Aula 24

Entregar:

- ❶ **Programa de echo com conversão** (maiúscula) funcional no MARS
- ❷ **Programa do desafio** (buffer circular) ou programa de impressão de string via MMIO
- ❸ **Respostas ao PBL:**
 - Explique por que o teclado responde instantaneamente
 - Explique por que a cópia USB demora
 - Compare polling, interrupções e DMA com exemplos
 - Em que situação DMA é essencial?



Conceitos-chave:

- MMIO: lw/sw para dispositivos
- Endereços: 0xFFFF0000–0C
- Registradores Control + Data
- Polling: loop verificando Ready
- Interrupções: IRQ + handler
- DMA: transferência sem CPU
- Controladores e handshaking
- Lei de Amdahl em E/S

Competências desenvolvidas:

- Programar MMIO no MARS
- Implementar polling em assembly
- Comparar polling/int/DMA
- Analisar gargalos de E/S
- Projetar buffer circular

Próximo módulo:

Hierarquia de Memória – por que a CPU não acessa a RAM diretamente e como a cache resolve isso!

Princípios da Hierarquia de Memória

Metodologia: Analogia da Biblioteca + Atividade Prática

- Analogia: da mesa ao depósito
- Por que memória rápida é cara e pequena?
- Princípio da localidade (temporal e espacial)
- Níveis da hierarquia: registradores, cache, RAM, disco
- Métricas: hit rate, miss rate, AMAT
- Atividade: simulando acessos com tempos reais

O problema fundamental

Queremos memória que seja simultaneamente:

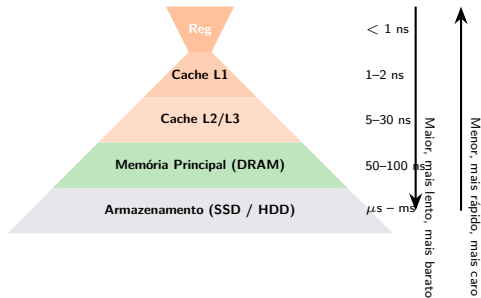
- **Rápida** (como registradores: < 1 ns)
- **Grande** (como disco: terabytes)
- **Barata** (como HDD: centavos por GB)

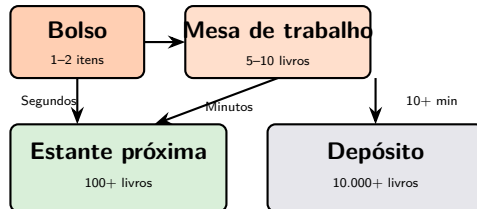
Mas não podemos ter as três ao mesmo tempo!

Tecnologia	Velocidade	Custo/GB	Tamanho típico
SRAM (cache)	~ 1 ns	R\$ 500+	1–64 MB
DRAM (RAM)	~ 50 ns	R\$ 3–5	8–64 GB
SSD (NVMe)	~ 50 μ s	R\$ 0,40	256 GB–4 TB
HDD	~ 5 ms	R\$ 0,10	1–20 TB

Ideia

Organizar a memória em **níveis**, do mais rápido/caro/pequeno ao mais lento/barato/grande. Os dados mais usados ficam nos níveis superiores (mais rápidos).





Você não vai ao depósito toda vez que precisa de um livro!

Traz os livros mais usados para a mesa. Se precisar de outro, consulta a estante próxima. O depósito só entra em cena em último caso.

A hierarquia de memória funciona **exatamente assim**.

Observação empírica

Programas tendem a acessar um conjunto **pequeno** de dados **repetidamente** e a acessar dados **próximos** entre si.

Localidade Temporal:

- Se um dado foi acessado agora, provavelmente será acessado **novamente em breve**
- Exemplos: variáveis de loop, contadores, ponteiros de pilha
- `for (i=0; i<N; i++)` → `i` é lido/escrito a cada iteração

Localidade Espacial:

- Se um dado foi acessado, dados em endereços **próximos** serão acessados em breve
- Exemplos: vetores, structs, instruções sequenciais
- `A[0]`, `A[1]`, `A[2]`, ... → endereços contíguos

Exemplo 1: Soma de vetor (ambas localidades)

```
soma = 0;           // soma: localidade temporal
for (i=0; i<N; i++) // i: localidade temporal
    soma += A[i];    // A[i]: localidade espacial
```

Exemplo 2: Multiplicação de matrizes

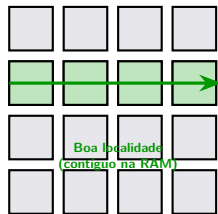
```
for (i=0; i<N; i++)
    for (j=0; j<N; j++)
        for (k=0; k<N; k++)
            C[i][j] += A[i][k] * B[k][j];
```

Pergunta

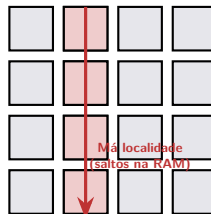
A[i][k] tem boa localidade espacial? E B[k][j]?

(Dica: em C, matrizes são armazenadas por linha – row-major.)

Matriz A (por linha)



Matriz B (por coluna)



$A[i][k]$: acesso por **linha** (k varia) → endereços contíguos ✓

$B[k][j]$: acesso por **coluna** (k varia) → saltos de N posições ×

Consequência

A multiplicação de matrizes “ingênua” tem péssima localidade para B. Técnicas como **blocking/tiling** reorganizam os acessos para melhorar a localidade → veremos no Módulo 9.

Definições fundamentais:

- **Hit:** dado encontrado no nível atual da hierarquia
- **Miss:** dado **não** encontrado → busca no nível inferior
- **Hit Rate** (h): fração dos acessos que são hits
- **Miss Rate** (m): fração dos acessos que são misses ($m = 1 - h$)
- **Hit Time** (t_h): tempo para acessar o dado no nível atual
- **Miss Penalty** (t_p): tempo **adicional** para buscar no nível inferior

AMAT – Average Memory Access Time

$$AMAT = t_h + m \times t_p$$

O tempo médio de acesso combina a probabilidade de hit/miss com os respectivos tempos.

Dados:

- Cache L1: $t_h = 1$ ns, miss rate = 5%
- Memória RAM (miss penalty): $t_p = 100$ ns

Cálculo:

$$AMAT = 1 + 0.05 \times 100 = 1 + 5 = \mathbf{6 \text{ ns}}$$

Interpretação

Mesmo com miss rate de apenas 5%, o AMAT é $6\times$ maior que o hit time!

A **penalidade do miss é alta** – cada miss custa 100 ns.

Reduzir o miss rate de 5% para 2% melhoraria AMAT para $1 + 0.02 \times 100 = 3$ ns – uma melhoria de 50%!

Hierarquia com L1 e L2:

$$AMAT = t_{L1} + m_{L1} \times (t_{L2} + m_{L2} \times t_{RAM})$$

Dados:

Nível	Hit time	Miss rate
L1	1 ns	5%
L2	5 ns	20% (dos que falharam em L1)
RAM	100 ns	–

Cálculo:

$$\begin{aligned}AMAT &= 1 + 0.05 \times (5 + 0.20 \times 100) \\&= 1 + 0.05 \times (5 + 20) \\&= 1 + 0.05 \times 25 = 1 + 1.25 = \mathbf{2.25 \text{ ns}}\end{aligned}$$

O L2 reduziu o AMAT de 6 ns (sem L2) para 2.25 ns – melhoria de 63%!

Dinâmica (30 min)

Em grupos, simulem a hierarquia de memória fisicamente:

- **Bolso** (registrador): 2 cartões
- **Mesa** (cache L1): 5 cartões
- **Estante** (RAM): 20 cartões em outra mesa (5m de distância)
- **Depósito** (disco): caixa no final da sala (15m)

Regra: Para cada acesso, meçam o **tempo real** (cronômetro). Se o cartão está na mesa = rápido. Se não, vá à estante ou ao depósito. **Sequência de 20 acessos:**

A, B, A, C, A, B, D, E, A, F, A, B, G, H, A, C, B, I, J, A

Calcule: hit rate na mesa, tempo total, AMAT.

Análise da sequência: A, B, A, C, A, B, D, E, A, F, A, B, G, H, A, C, B, I, J, A

- **A** aparece 7 vezes → forte localidade temporal
- **B** aparece 4 vezes → localidade temporal moderada
- Demais: 1–2 vezes cada

Com mesa de 5 cartões e política LRU:

- Hits na mesa: ~12–14 dos 20 acessos
- Hit rate: ~60–70%
- Se hit = 1s e miss = 10s: $AMAT \approx 1 + 0.35 \times 10 = 4.5s$

Discussão

O que acontece se aumentarmos a mesa para 10 cartões?
E se mudarmos a ordem dos acessos para eliminar localidade?
Essas são exatamente as questões de projeto de cache!

Contexto

Se melhorarmos a CPU em $10\times$ mas a memória continua lenta, qual o speedup real?

Suponha: 40% do tempo é gasto em acessos à memória.

$$S = \frac{1}{(1 - f) + \frac{f}{k}} = \frac{1}{0.40 + \frac{0.60}{10}} = \frac{1}{0.40 + 0.06} = \frac{1}{0.46} \approx 2.17\times$$

Conclusão

Mesmo melhorando a CPU em $10\times$, o speedup é apenas $2.17\times$!

A memória é o **gargalo**. Por isso a hierarquia de memória e a otimização de cache são **críticas** para desempenho.

O que aprendemos:

- Dilema: rápido vs grande vs barato
- Hierarquia: Reg > Cache > RAM > Disco
- Localidade temporal e espacial
- Métricas: hit rate, miss rate
- $AMAT = t_h + m \times t_p$
- AMAT multinível
- Lei de Amdahl e memória

Para a Aula 26:

- Pesquisar especificações de cache de um processador real (Intel, AMD ou Apple M-series)
- Anotar: tamanhos L1/L2/L3, latências, miss rates (se encontrar)

Análise de Processadores Reais e AMAT

Metodologia: Análise de Dados Reais + Cálculo de AMAT

- Especificações de cache de processadores reais
- Cálculo de AMAT para diferentes processadores
- Impacto do tamanho e associatividade da cache
- Por que o Apple M-series é competitivo?
- Exercícios de cálculo e comparação

Processador	L1 I	L1 D	L2	L3	Ano
Intel i7-12700K	32 KB	48 KB	1.25 MB	25 MB	2021
AMD Ryzen 9 7950X	32 KB	32 KB	1 MB	64 MB	2022
Apple M2	192 KB	128 KB	16 MB	–	2022
Apple M2 Pro	192 KB	128 KB	16 MB	–	2023

Observações interessantes

- **Apple M2:** L1 enorme (192+128 KB) comparado com Intel/AMD (~80 KB)
- **AMD 7950X:** L3 gigante (64 MB!) com tecnologia 3D V-Cache
- **Intel:** L1 assimétrica ($I \neq D$) – otimização específica
- Todos usam caches separadas para instruções (I) e dados (D) em L1 (Harvard!)

Nível	Latência	Em ciclos (~ 3 GHz)
L1	1–2 ns	3–4 ciclos
L2	3–10 ns	10–30 ciclos
L3	10–30 ns	30–100 ciclos
RAM (DDR5)	50–80 ns	150–250 ciclos

Perspectiva

Quando a L1 falha e precisa ir à RAM, a CPU espera o equivalente a **200+ ciclos** – tempo suficiente para executar centenas de instruções!

É como se você estivesse lendo um livro e tivesse que esperar **3 minutos** cada vez que precisa virar a página.

Exercício 1: AMAT de Processadores Reais

Calcule o AMAT para cada processador (15 min)

Dados estimados:

	t_{L1}	m_{L1}	t_{L2}	m_{L2}	t_{L3}	m_{L3}
Intel i7	1 ns	5%	5 ns	15%	20 ns	5%
AMD R9	1 ns	5%	4 ns	10%	15 ns	2%
Apple M2	1 ns	3%	8 ns	8%	—	—

Miss penalty final (RAM): 70 ns para todos.

Fórmula:

$$AMAT = t_{L1} + m_{L1} \times (t_{L2} + m_{L2} \times (t_{L3} + m_{L3} \times t_{RAM}))$$

Para o Apple M2 (sem L3): $AMAT = t_{L1} + m_{L1} \times (t_{L2} + m_{L2} \times t_{RAM})$

Intel i7:

$$\begin{aligned}AMAT &= 1 + 0.05 \times (5 + 0.15 \times (20 + 0.05 \times 70)) \\&= 1 + 0.05 \times (5 + 0.15 \times 23.5) \\&= 1 + 0.05 \times 8.525 = \mathbf{1.43 \text{ ns}}\end{aligned}$$

AMD Ryzen 9:

$$\begin{aligned}AMAT &= 1 + 0.05 \times (4 + 0.10 \times (15 + 0.02 \times 70)) \\&= 1 + 0.05 \times (4 + 0.10 \times 16.4) \\&= 1 + 0.05 \times 5.64 = \mathbf{1.28 \text{ ns}}\end{aligned}$$

Apple M2:

$$\begin{aligned}AMAT &= 1 + 0.03 \times (8 + 0.08 \times 70) \\&= 1 + 0.03 \times 13.6 = \mathbf{1.41 \text{ ns}}\end{aligned}$$

Discussão: Por que o Apple M2 é Competitivo?

Processador	AMAT
AMD Ryzen 9 7950X	1.28 ns
Apple M2	1.41 ns
Intel i7-12700K	1.43 ns

Fatores-chave do Apple M2:

- L1 enorme (192+128 KB) → miss rate L1 muito baixo (3% vs 5%)
- Mesmo sem L3, compensa com L1 grande e L2 de 16 MB
- Memória unificada (CPU e GPU compartilham) → menos cópias
- Arquitetura ARM: instruções fixas → pipeline mais eficiente

Lição

Não é só "mais cache = melhor". É **onde** colocar a cache e como minimizar o miss rate nos níveis mais importantes (L1!).

O que impacta mais o AMAT? (15 min)

Usando o Intel i7 como base ($AMAT = 1.43$ ns), calcule o novo AMAT para:

- 1 Reduzir miss rate L1 de 5% para 2% (mantendo o resto)
- 2 Reduzir miss rate L3 de 5% para 1% (mantendo o resto)
- 3 Reduzir latência da RAM de 70 ns para 50 ns
- 4 Dobrar o tamanho do L2 (reduz m_{L2} de 15% para 10%)

Qual modificação tem o maior impacto?

Modificação	Novo AMAT	Melhoria
Base (original)	1.43 ns	–
(a) m_{L1} : 5% $\rightarrow$ 2%	$1 + 0.02 \times 8.525 = 1.17$ ns	–18%
(b) m_{L3} : 5% $\rightarrow$ 1%	$1 + 0.05 \times 8.125 = 1.41$ ns	–1.4%
(c) t_{RAM} : 70 $\rightarrow$ 50 ns	$1 + 0.05 \times 8.225 = 1.41$ ns	–1.4%
(d) m_{L2} : 15% $\rightarrow$ 10%	$1 + 0.05 \times 7.35 = 1.37$ ns	–4.2%

Conclusão

Reduzir o miss rate do L1 é disparadamente o mais impactante! Cada acesso à memória começa no L1 – se o hit rate é alto ali, os demais níveis pouco importam. Isso explica por que o Apple M2 investe em L1 enorme.

Exercício 3: Impacto no Tempo de Execução

Exercício (15 min)

Um programa com 10^9 instruções executa em processador a 3 GHz.
30% das instruções acessam memória. Use o AMAT do Intel i7 (1.43 ns).

- 1 Qual o tempo de execução com $AMAT = 1.43$ ns?
- 2 Se o AMAT fosse ideal (= L1 hit time = 1 ns)?
- 3 Qual o speedup da cache comparado com acesso direto à RAM (70 ns)?

Fórmulas:

$$T_{exec} = N \times CPI \times T_{clock}$$

$$CPI_{mem} = CPI_{base} + \text{fração_mem} \times \frac{AMAT}{T_{clock}}$$

Assuma $CPI_{base} = 1$ (pipeline ideal sem stalls de memória).

$$T_{clock} = 1/3 \text{ GHz} = 0.333 \text{ ns}$$

1. Com cache (AMAT = 1.43 ns):

$$CPI_{mem} = 1 + 0.30 \times \frac{1.43}{0.333} = 1 + 0.30 \times 4.29 = 2.29$$

$$T_{exec} = 10^9 \times 2.29 \times 0.333 \text{ ns} = \mathbf{0.76 \text{ s}}$$

2. Cache ideal (AMAT = 1 ns):

$$CPI_{mem} = 1 + 0.30 \times 3 = 1.90 \quad \Rightarrow \quad T_{exec} = \mathbf{0.63 \text{ s}}$$

3. Sem cache (acesso direto à RAM, 70 ns):

$$CPI_{mem} = 1 + 0.30 \times 210 = 64 \quad \Rightarrow \quad T_{exec} = \mathbf{21.3 \text{ s}}$$

Speedup da cache: $21.3/0.76 \approx \mathbf{28\times}$ mais rápido com cache!

Exercícios de AMAT + Análise Comparativa

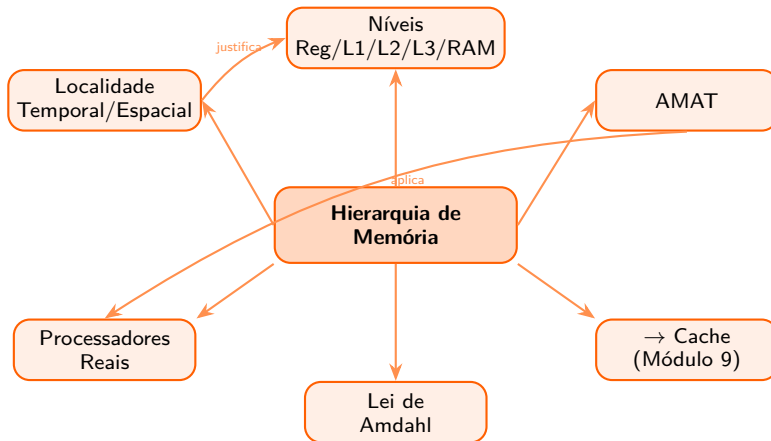
Peso: 5% da nota

Formato: Individual ou dupla

Prazo: 1 semana após a Aula 26

Conteúdo:

- 1 Calcular AMAT para 3 configurações de cache diferentes (dados fornecidos)
- 2 Análise de sensibilidade: qual parâmetro mais impacta o AMAT?
- 3 Pesquisar especificações de cache de **um processador à sua escolha** e calcular o AMAT estimado
- 4 Comparar com os processadores da aula e justificar diferenças
- 5 Calcular T_{exec} com e sem cache para um programa hipotético



Conceitos-chave:

- Dilema: rápido $\times$ grande $\times$ barato
- Hierarquia resolve com níveis
- Localidade temporal e espacial
- Hit, Miss, Hit Rate, Miss Rate
- $AMAT = t_h + m \times t_p$
- AMAT multinível
- Miss rate L1 é o fator dominante
- Cache dá speedup de $\sim 28\times$

Competências desenvolvidas:

- Calcular AMAT (1 e múltiplos níveis)
- Analisar sensibilidade de parâmetros
- Comparar processadores reais
- Aplicar Lei de Amdahl à memória
- Entender trade-offs de projeto

Próximo tópico:

Memória Cache – como funciona internamente: mapeamento, substituição e escrita!

Mapeamento de Cache

Metodologia: Sala de Aula Invertida + Exercícios Progressivos

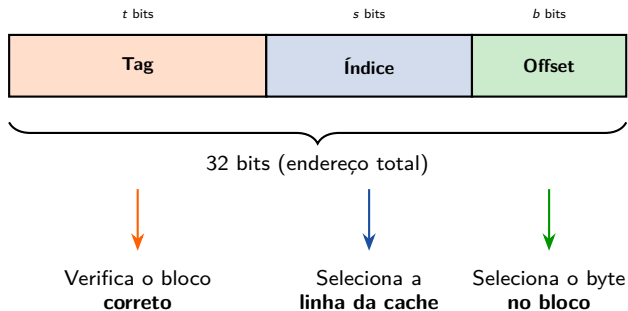
- Estrutura interna da cache
- Mapeamento direto: tag, índice, offset
- Mapeamento totalmente associativo
- Mapeamento associativo por conjunto
- Exercícios progressivos: dado um endereço, onde vai na cache?

Vídeo pré-aula sobre mapeamento direto

- 1 O que determina **onde** um bloco de memória é armazenado na cache?
- 2 Quantos bits tem o campo "tag"? E o "índice"? E o "offset"?
- 3 Se a cache tem 256 linhas e blocos de 16 bytes, quantos bits de índice e offset?

Respostas

1. O **endereço** do bloco determina (via função de mapeamento).
2. Depende da configuração (tamanho da cache, tamanho do bloco, tipo de mapeamento).
3. Índice: $\log_2 256 = 8$ bits. Offset: $\log_2 16 = 4$ bits. Tag: $32 - 8 - 4 = 20$ bits.



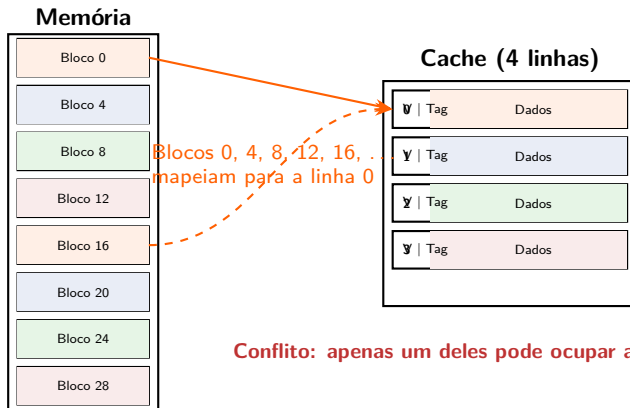
Fórmulas:

- Offset: $b = \log_2(\text{tamanho do bloco em bytes})$
- Índice: $s = \log_2(\text{número de linhas da cache})$
- Tag: $t = 32 - s - b$

Regra

Cada bloco de memória pode ser armazenado em **exatamente uma linha** da cache, determinada pelo campo **índice** do endereço.

$\text{linha} = \text{endereço_bloco} \bmod \text{número_de_linhas}$



Conflito: apenas um deles pode ocupar a linha 0 por vez

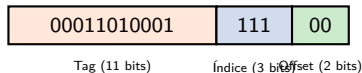
Blocos 0, 4, 8, 12, 16, ... competem pela mesma linha da cache.

Exemplo: Mapeamento Direto

Configuração: Cache com 8 linhas, blocos de 4 bytes, endereço de 16 bits.

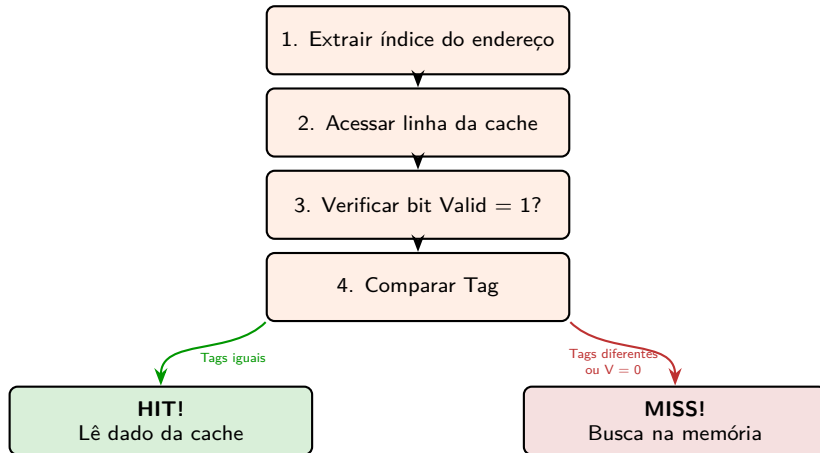
- Offset: $\log_2 4 = 2$ bits
- Índice: $\log_2 8 = 3$ bits
- Tag: $16 - 3 - 2 = 11$ bits

Endereço 0x1A3C = 0001 1010 0011 1100₂



- **Linha:** 111₂ = 7 (vai para a linha 7 da cache)
- **Byte no bloco:** 00₂ = 0 (primeiro byte do bloco)
- **Tag armazenada:** 00011010001 (identifica qual bloco está na linha)

Processo de Acesso: Hit ou Miss?



Cada linha da cache armazena: **Valid bit** (1 bit) + **Tag** (t bits) + **Dados** (bloco de 2^b bytes).

Exercício (15 min)

Cache: 4 linhas, bloco de 1 palavra (4 bytes), endereço de 32 bits.

Inicialmente vazia ($V=0$ em todas as linhas).

Sequência de acessos a **endereços de byte**: 0, 4, 8, 12, 16, 0, 4, 16, 0

Configuração:

- Offset: $\log_2 4 = 2$ bits
- Índice: $\log_2 4 = 2$ bits
- Tag: $32 - 2 - 2 = 28$ bits

Preencha para cada acesso:

Endereço	Índice	Tag	Hit/Miss	Ação
0	?	?	?	?
4	?	?	?	?
...				

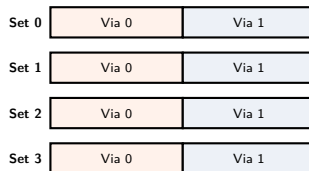
End.	Índ.	Tag	H/M	Observação
0	00	0x0000000	Miss	Carrega bloco 0 na linha 0
4	01	0x0000000	Miss	Carrega bloco 1 na linha 1
8	10	0x0000000	Miss	Carrega bloco 2 na linha 2
12	11	0x0000000	Miss	Carrega bloco 3 na linha 3
16	00	0x0000001	Miss	Conflito! Substitui bloco 0
0	00	0x0000000	Miss	Conflito! Substitui bloco 16
4	01	0x0000000	Hit	Bloco 1 ainda na linha 1 ✓
16	00	0x0000001	Miss	Conflito com bloco 0 de novo!
0	00	0x0000000	Miss	Thrashing! 0 e 16 competem

Hit rate: $1/9 \approx 11\%$ – Péssimo! Blocos 0 e 16 competem pela mesma linha.

Compromisso

A cache é dividida em **conjuntos** (sets). Cada bloco mapeia para um conjunto específico (pelo índice), mas dentro do conjunto pode ir para **qualquer via** (way).

Cache 2-way, 4 conjuntos



***n*-way set-associative:** *n* vias por conjunto. $\text{Conjunto} = \text{endereço_bloco} \bmod \text{num_conjuntos}$

- **Índice:** seleciona o **conjunto**
- Dentro do conjunto: busca associativa (compara tag em todas as vias)
- Número de conjuntos = $\frac{\text{linhas totais}}{\text{associatividade}}$

Comparação dos 3 Tipos de Mapeamento

	Direto	Set-Assoc.	Totalmente Assoc.
Associatividade	1-way	n -way	L -way
Conjuntos	L	L/n	1
Comparadores	1	n	L
Conflitos	Muitos	Moderados	Nenhum
Hardware	Simples	Moderado	Complexo
Hit rate	Menor	Bom	Melhor
Uso típico	L1 (simples)	L1/L2/L3	TLB

Na prática

L1: 4-way ou 8-way set-associative (rápida, poucos conflitos)

L2/L3: 8-way a 16-way (mais associativa, menos conflitos)

TLB: Totalmente associativa (pequena, precisa de 0 conflitos)

Exercício (15 min)

Mesma sequência do Exercício 1, mas agora com cache **2-way set-associative** com 2 conjuntos (total = 4 linhas), blocos de 4 bytes.

Sequência: 0, 4, 8, 12, 16, 0, 4, 16, 0

Configuração:

- Offset: 2 bits
- Índice (conjunto): $\log_2 2 = 1$ bit
- Tag: $32 - 1 - 2 = 29$ bits

Mapeamento:

Endereços com bit 2 = 0 (0, 8, 16, ...)	Conjunto 0 (2 vias)
Endereços com bit 2 = 1 (4, 12, 20, ...)	Conjunto 1 (2 vias)

End.	Set	H/M	Vias do Set	Obs.
0	0	Miss	[0, -]	Via 0
4	1	Miss	[4, -]	Via 0
8	0	Miss	[0, 8]	Via 1
12	1	Miss	[4, 12]	Via 1
16	0	Miss	[16, 8] LRU	Substitui 0 (mais antigo)
0	0	Miss	[16, 0] LRU	Substitui 8
4	1	Hit	[4, 12]	✓
16	0	Hit	[16, 0]	✓
0	0	Hit	[16, 0]	✓

Hit rate: $3/9 \approx 33\%$ – Melhorou de 11% (direto) para 33%!

O thrashing entre 0 e 16 foi resolvido pois ambos cabem no conjunto 0 (2 vias).

O que aprendemos:

- Endereço = Tag + Índice + Offset
- Mapeamento direto: simples, conflitos
- Totalmente associativo: sem conflitos, caro
- Set-associative: compromisso
- Processo de hit/miss

Para a próxima Aula:

- Revisar os 3 tipos de mapeamento
- Instalar Python (para o simulador)
- Estudar: políticas de substituição (LRU, FIFO)

Políticas de Cache e Simulação

Metodologia: Simulação de Cache em Planilha/Código

- Políticas de substituição: LRU, FIFO, Random
- Políticas de escrita: write-through vs write-back
- Cache multinível: inclusiva vs exclusiva
- Simulador de cache em Python

Quando ocorre um miss e o conjunto está cheio, qual bloco substituir?

Política	Critério	Hardware
LRU	Menos recentemente usado	Complexo
FIFO	Primeiro que entrou, primeiro sai	Moderado
Random	Aleatório	Simple
LFU	Menos frequentemente usado	Muito complexo

LRU na prática

LRU é a melhor escolha teórica (explora localidade temporal), mas é caro de implementar para alta associatividade. Processadores reais usam **pseudo-LRU** (aproximação mais barata). Surpreendentemente, Random é competitivo para associatividade ≥ 8 .

Cache 2-way set-associative, 1 conjunto. Sequência: A, B, C, A, B, D, A

LRU:

	Via 0	Via 1	H/M
A	A	—	M
B	A	B	M
C	C	B	M (subst. A)
A	C	A	M (subst. B)
B	B	A	M (subst. C)
D	B	D	M (subst. A)
A	A	D	M (subst. B)

Hits: 0/7

FIFO:

	Via 0	Via 1	H/M
A	A	—	M
B	A	B	M
C	C	B	M (subst. A)
A	C	A	M (subst. B)
B	B	A	M (subst. C)
D	B	D	M (subst. A)
A	A	D	M (subst. B)

Hits: 0/7

(péssimo para esta sequência)

(igual neste caso)

Para 2-way, LRU e FIFO frequentemente empatam. A diferença aparece em ≥ 4 -way.

Quando a CPU escreve um dado na cache, quando atualizar a memória?

Write-Through:

- Escreve na cache e na **memória** simultaneamente
- Memória sempre atualizada
- Mais simples, mais tráfego no barramento
- Usa **write buffer** para absorver latência

Write-Back:

- Escreve **apenas na cache**
- Marca o bloco como "dirty" (modificado)
- Só escreve na memória quando o bloco é **substituído**
- Menos tráfego, mais complexo
- Precisa do **dirty bit**

E no miss de escrita?

Write-allocate: traz o bloco para a cache, depois escreve (usado com write-back).

No-write-allocate: escreve direto na memória, sem trazer para cache (usado com write-through).

Inclusiva:

- $L1 \subseteq L2 \subseteq L3$
- Todo dado em L1 também está em L2
- Simplifica coerência
- "Desperdício" de espaço em L2
- Usado por: Intel

Exclusiva:

- $L1 \cap L2 = \emptyset$
- Dado está em L1 **ou** L2, nunca ambos
- Capacidade efetiva = $L1 + L2$
- Mais complexo de gerenciar
- Usado por: AMD (parcialmente)

Por que importa?

Com cache **inclusiva**, L2 de 1 MB com L1 de 64 KB = apenas 960 KB "extras".

Com cache **exclusiva**, L2 de 1 MB + L1 de 64 KB = 1088 KB efetivos.

Para caches grandes como L3 de 64 MB (AMD), a diferença é significativa.

Atividade (60 min)

Implementar um simulador de cache em Python (ou planilha) que:

- 1 Recebe: tamanho da cache, tamanho do bloco, associatividade
- 2 Recebe: sequência de endereços de acesso
- 3 Simula mapeamento e política LRU
- 4 Reporta: hit/miss para cada acesso e hit rate final

Estrutura sugerida em Python:

```
class CacheLine:    valid, tag, last_used
class CacheSet:     lines[] (n vias)
class Cache:        sets[], access(), stats()
```

Testes: Rodarem a mesma sequência dos exercícios manuais para validar.

Após validar, experimentem:

- ❶ **Variando associatividade:** 1-way, 2-way, 4-way, 8-way
Mesma sequência. Como muda o hit rate?
- ❷ **Variando tamanho de bloco:** 4, 8, 16, 32 bytes
Sequência com localidade espacial (endereços consecutivos).
- ❸ **Variando tamanho da cache:** 256 B, 1 KB, 4 KB
Sequência com working set de ~ 2 KB.
- ❹ **Padrão de acesso:** sequencial vs aleatório vs stride

Pergunta para o relatório

Qual parâmetro tem o maior impacto no hit rate: associatividade, tamanho de bloco, ou tamanho total da cache? Justifique com os dados da simulação.

O que aprendemos:

- Substituição: LRU, FIFO, Random
- Escrita: write-through vs write-back
- Write-allocate vs no-write-allocate
- Inclusiva vs exclusiva
- Simulação de cache

Para a próxima Aula:

- Finalizar o simulador (se não terminou)
- Pesquisar: por que multiplicação de matrizes é lenta?
- Estudar: técnica de blocking/tiling

Estudo de Caso: Otimização para Cache

Metodologia: Estudo de Caso + Análise

- Por que a multiplicação de matrizes é lenta?
- Análise do padrão de acesso ingênuo
- Técnica de blocking (tiling)
- Estimativa de miss rates para cada abordagem
- Os 3 tipos de misses: compulsório, capacidade, conflito

Código ingênuo (C):

```
for (i=0; i<N; i++)  
    for (j=0; j<N; j++)  
        for (k=0; k<N; k++)  
            C[i][j] += A[i][k] * B[k][j];
```

Análise de acessos (loop mais interno: k varia):

- $C[i][j]$: fixo $\rightarrow$ excelente localidade temporal
- $A[i][k]$: k varia na **mesma linha** $\rightarrow$ boa localidade espacial
- $B[k][j]$: k varia entre **linhas** (stride = N) $\rightarrow$ **péssima** localidade

Resultado

Para $N = 1024$ com double (8 bytes), cada acesso a B salta 8 KB na memória. Cada acesso pode causar um **cache miss**!

Parâmetros: $N = 512$, double (8 bytes), cache L1 de 32 KB, blocos de 64 bytes.

Elementos por bloco de cache: $64/8 = 8$ doubles

Para o loop mais interno ($k = 0$ a $N - 1$):

- $A[i][k]$: acesso sequencial. Miss a cada 8 elementos. Miss rate $= 1/8 = 12.5\%$
- $B[k][j]$: stride $= N \times 8$ bytes. **Miss a cada acesso** se stride $>$ tamanho da cache. $512 \times 8 = 4096$ bytes de stride. Cache = 32 KB. Miss rate $\approx 100\%$!

Total de misses para B: $N^3 = 512^3 \approx 1.34 \times 10^8$ misses!

Com miss penalty de 100 ns

Tempo extra: $1.34 \times 10^8 \times 100 \text{ ns} = 13.4 \text{ s}$ de penalidade de cache.

Ideia

Dividir as matrizes em **blocos pequenos** ($B \times B$) que cabem na cache. Multiplicar bloco por bloco em vez de linha inteira por coluna inteira.

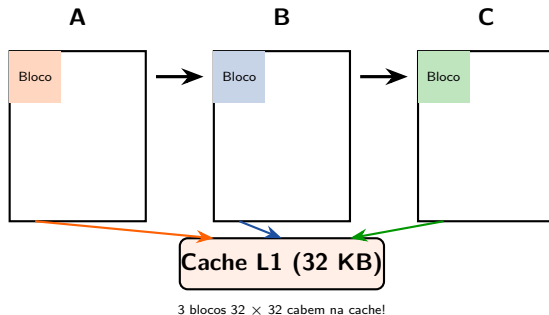
Código com blocking:

```
for (ii=0; ii<N; ii+=B)
  for (jj=0; jj<N; jj+=B)
    for (kk=0; kk<N; kk+=B)
      for (i=ii; i<ii+B; i++)
        for (j=jj; j<jj+B; j++)
          for (k=kk; k<kk+B; k++)
            C[i][j] += A[i][k] * B[k][j];
```

Se B é escolhido tal que 3 blocos de $B \times B$ doubles cabem na cache:

$$3 \times B^2 \times 8 \leq 32\text{KB} \Rightarrow B \leq 36 \Rightarrow \text{use } B = 32.$$

Blocking – Por que Funciona?



Cada sub-bloco de A, B e C é **reutilizado** B vezes antes de ser substituído. Os acessos a B agora têm **boa localidade** dentro do sub-bloco.

	Ingênuo	Blocking ($B = 32$)
Miss rate B	$\sim 100\%$	$\sim 12.5\%$ (1/8 por bloco)
Misses totais (B)	N^3	N^3/B
Fator de melhoria	–	$B = 32\times$ menos misses
Tempo de penalidade	13.4 s	0.42 s

Resultado

Blocking reduziu as penalidades de cache em $\sim 32\times$!

O mesmo algoritmo, o mesmo hardware, apenas **reorganizando o acesso**. Isso demonstra o poder da otimização para cache.

Tipo	Causa	Como reduzir
Compulsório (cold miss)	Primeiro acesso a um bloco (inevitável)	Aumentar tamanho do bloco Prefetching
Capacidade	Cache não cabe todo o working set	Aumentar tamanho da cache Blocking
Conflito	Dois blocos mapeiam para o mesmo conjunto/linha	Aumentar associatividade Victim cache

Regra prática

Em caches **grandes**: a maioria dos misses é compulsória.

Em caches **pequenas**: capacidade domina.

Em caches de **baixa associatividade**: conflito é significativo.

Técnica	Alvo	Como funciona
Blocking/Tiling	Capacidade	Reutiliza dados na cache
Loop interchange	Espacial	Reordena loops para acesso contíguo
Prefetching	Compulsório	Carrega dados antecipadamente
Padding	Conflito	Evita aliasing de endereços
Aumentar bloco	Compulsório	Mais dados por miss (localidade espacial)
Aumentar assoc.	Conflito	Menos substituições forçadas
Victim cache	Conflito	Pequena cache para blocos substituídos

Simulador de Cache + Relatório

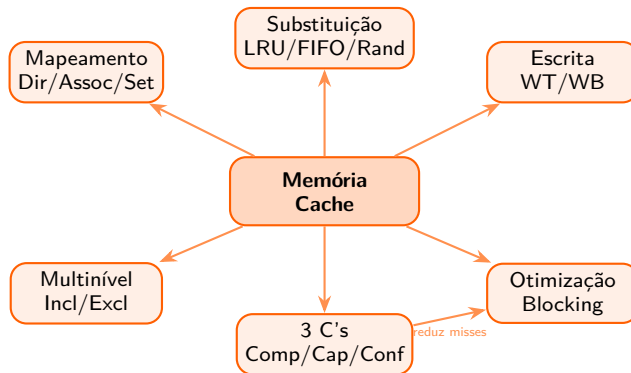
Peso: 10% da nota

Formato: Individual ou dupla

Prazo: 1 semana após a Aula 29

Entregar:

- ❶ **Código do simulador** (Python) funcional para mapeamento direto e set-associative com LRU
- ❷ **Relatório** com análise de desempenho:
 - Variação de associatividade (1, 2, 4, 8-way) – gráfico de hit rate
 - Variação de tamanho de bloco (4, 8, 16, 32 bytes) – gráfico
 - Análise do impacto de blocking na multiplicação de matrizes
 - Identificação dos 3 C's para cada configuração
- ❸ **Conclusão:** qual configuração oferece o melhor custo-benefício?



Conceitos-chave:

- Endereço: Tag + Índice + Offset
- Mapeamento direto: 1 via
- Totalmente associativo: L vias
- Set-associative: n vias
- LRU, FIFO, Random
- Write-through vs write-back
- Inclusiva vs exclusiva
- 3 C's: compulsório, capacidade, conflito
- Blocking reduz misses de capacidade

Competências desenvolvidas:

- Calcular tag/índice/offset
- Simular hit/miss manualmente
- Implementar simulador de cache
- Analisar impacto de parâmetros
- Otimizar código para cache

Próximo módulo:

Memória Principal – SRAM vs DRAM, DDR, e gerenciamento pelo SO!

Tecnologias de Memória e Evolução DDR

Metodologia: Pesquisa Orientada + Painel de Especialistas

- SRAM vs DRAM vs Flash/SSD
- Organização interna da DRAM: bancos, linhas, colunas
- Ciclo de refresh e suas implicações
- Evolução: DDR, DDR2, DDR3, DDR4, DDR5
- Painel de especialistas: cada grupo apresenta uma geração

SRAM (Static RAM):

- 6 transistores por bit
- Dado armazenado em **flip-flop**
- Não precisa de refresh
- Muito rápida (1–2 ns)
- Cara e grande (área de silício)
- **Uso:** Cache L1/L2/L3

DRAM (Dynamic RAM):

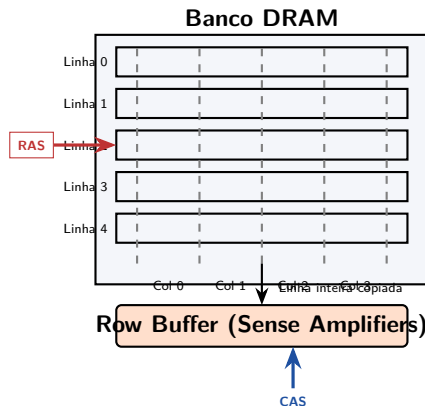
- 1 transistor + 1 capacitor por bit
- Dado armazenado como **carga elétrica**
- Precisa de refresh periódico
- Mais lenta (50–100 ns)
- Barata e densa
- **Uso:** Memória principal



Flash NAND

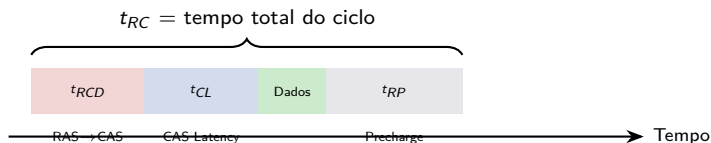
- Armazenamento **não volátil** (mantém dados sem energia)
- Baseado em **floating gate transistors**
- Leitura rápida ($\sim 25 \mu s$), escrita mais lenta ($\sim 200 \mu s$)
- Número limitado de ciclos de escrita (~ 3.000 – 100.000)
- Organizado em páginas (4–16 KB) e blocos (256–512 KB)

	SRAM	DRAM	Flash
Volátil?	Sim	Sim	Não
Latência leitura	$\sim 1 \text{ ns}$	$\sim 50 \text{ ns}$	$\sim 25 \mu s$
Custo/GB	R\$ 500+	R\$ 3–5	R\$ 0,40
Refresh?	Não	Sim (64 ms)	Não
Endurance	Ilimitada	Ilimitada	Limitada



Acesso DRAM (2 passos):

- ❶ **RAS** (Row Address Strobe): seleciona a linha, copia para o row buffer
- ❷ **CAS** (Column Address Strobe): seleciona a coluna no row buffer, retorna dado



Parâmetros-chave (exemplo DDR4-3200):

Parâmetro	Valor	Significado
t_{RCD}	13.75 ns	Tempo para ativar linha
t_{CL} (CAS Latency)	13.75 ns	Tempo para selecionar coluna
t_{RP}	13.75 ns	Tempo para fechar linha
Frequência	1600 MHz (DDR)	Clock efetivo
Bandwidth	25.6 GB/s	Vazão máxima

Por que a DRAM precisa de refresh?

O capacitor que armazena cada bit **perde carga** ao longo do tempo (corrente de fuga). Sem refresh, o dado é perdido em ~ 64 ms.

Mecanismo:

- Controlador de memória lê cada linha periodicamente
- A leitura "recarrega" os capacitores (refresh = leitura + reescrita)
- Todas as linhas devem ser refreshed a cada **64 ms**
- Durante o refresh, a DRAM fica **indisponível**

Impacto

Com 8192 linhas e refresh de 64 ms: uma linha a cada $\sim 7.8 \mu\text{s}$. O refresh consome $\sim 1\text{--}5\%$ da banda disponível. Em DRAMs maiores (16 Gb+), refresh se torna um gargalo significativo.

	DDR	DDR2	DDR3	DDR4	DDR5
Ano	2000	2003	2007	2014	2020
Voltagem	2.5 V	1.8 V	1.5 V	1.2 V	1.1 V
Clock (MHz)	100–200	200–533	400–1066	800–1600	2400–4800
Taxa máx.	3.2 GB/s	8.5 GB/s	17 GB/s	25.6 GB/s	51.2 GB/s
Prefetch	2n	4n	8n	8n	16n
Capacidade	1 GB	4 GB	16 GB	64 GB	128 GB

Tendência a cada geração

- **Menor voltagem** → menor consumo de energia
- **Maior prefetch** → mais dados por acesso (compensa latência)
- **Maior bandwidth** → mais dados por segundo
- **Latência absoluta pouco muda!** ($\sim 13\text{--}15$ ns de CAS latency em todas)

Principais inovações do DDR5:

- ❶ **Dois canais por DIMM:** Cada módulo DDR5 tem 2 canais independentes de 32 bits (vs 1 canal de 64 bits no DDR4). Melhor paralelismo.
- ❷ **On-die ECC:** Correção de erros dentro do próprio chip DRAM. Não substitui ECC do sistema, mas melhora confiabilidade.
- ❸ **Regulador de voltagem no DIMM:** O PMIC (Power Management IC) fica no próprio módulo, melhorando eficiência energética.
- ❹ **Maior densidade:** Até 64 Gb por die (vs 16 Gb no DDR4).
- ❺ **Same-bank refresh:** Permite refresh em um banco enquanto outros trabalham.

Impacto prático

DDR5-6400: 51.2 GB/s por canal → ideal para workloads de IA e servidores.

Dinâmica (50 min)

Cada grupo pesquisou previamente uma geração de DDR.

Agora, apresentem em formato de **painel** (5 min por grupo):

- 1 Qual o contexto tecnológico da época? (computadores, SO, aplicações)
- 2 Quais as especificações-chave? (clock, bandwidth, voltagem, prefetch)
- 3 Qual a principal inovação em relação à geração anterior?
- 4 Qual o impacto nos sistemas reais da época?

Distribuição:

Grupo 1	DDR (2000) – A revolução do "Double Data Rate"
Grupo 2	DDR2 (2003) – Notebooks e eficiência energética
Grupo 3	DDR3 (2007) – A era dos smartphones e cloud
Grupo 4	DDR4 (2014) – Big Data e virtualização
Grupo 5	DDR5 (2020) – IA, HPC e o futuro

Perguntas para debate (15 min):

- ❶ A latência da DRAM (CAS latency em ns) mudou significativamente de DDR para DDR5? Por que não?
- ❷ Se a bandwidth dobra a cada geração, por que a CPU não fica $2\times$ mais rápida?
- ❸ Qual o papel da cache nesse contexto? (Esconde a latência da DRAM!)
- ❹ SSDs NVMe são mais rápidos que DDR4? (*Não em latência: $\sim 50 \mu s$ vs $\sim 50 ns = 1000\times$ mais lento*)
- ❺ Faz sentido usar DRAM como "cache de SSD"? (*Sim! É exatamente o que o SO faz com page cache*)

O que aprendemos:

- SRAM (6T, cache), DRAM (1T1C, RAM), Flash (não volátil)
- DRAM: bancos, linhas, colunas, row buffer
- RAS $\rightarrow$ CAS $\rightarrow$ Dados
- Refresh a cada 64 ms
- DDR a DDR5: mais bandwidth, menos voltagem
- Latência absoluta pouco muda

Para a próxima Aula:

- Revisar: segmentação e paginação (conceitos básicos)
- Pesquisar: algoritmos FCFS, SSTF, SCAN
- Pensar: SSDs precisam de disk scheduling?

Gerenciamento de Memória e Scheduling de Disco

Metodologia: Simulação de Algoritmos de Scheduling

- Gerenciamento de memória pelo SO: segmentação e paginação
- Visão geral de endereçamento virtual
- Organização de discos (HDD): trilhas, setores, cilindros
- Algoritmos de scheduling: FCFS, SSTF, SCAN (Elevator)
- Simulação prática dos algoritmos
- SSDs: scheduling ainda importa?

Problema

Múltiplos programas precisam compartilhar a memória física. Como garantir **isolamento**, **proteção** e **eficiência**?

Segmentação:

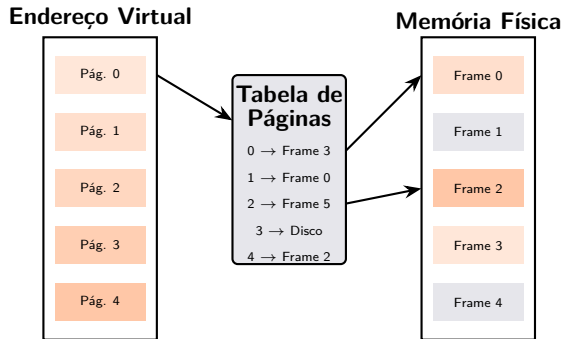
- Divide a memória em **segmentos** lógicos (código, dados, pilha)
- Tamanho variável
- Endereço: segmento + offset
- Problema: **fragmentação externa**

Paginação:

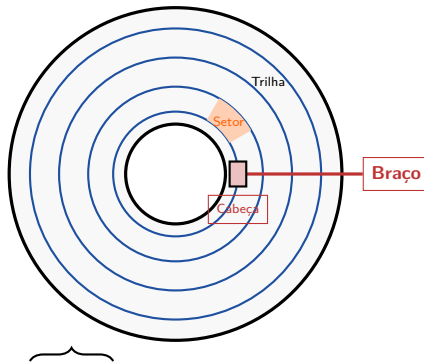
- Divide em **páginas** de tamanho fixo (4 KB típico)
- Mapeamento via **tabela de páginas**
- Sem fragmentação externa
- Fragmentação interna (mínima)
- Base da memória virtual!

Prévia do Módulo 11

Na próxima aula, estudaremos paginação e memória virtual em detalhe, incluindo TLB e page faults. Aqui, apenas a visão geral.



Cada programa vê um espaço **virtual** contínuo. O SO mapeia páginas virtuais para frames físicos (possivelmente não contíguos). Página 3 está no **disco** (page fault!).



Cilindro (mesma trilha em todos os pratos)

$$\text{Tempo de acesso HDD} = t_{seek} + t_{rotao} + t_{transferencia}$$

t_{seek} : mover braço até a trilha ($\sim 3\text{--}10$ ms)

t_{rotao} : esperar setor girar (~ 4 ms para 7200 RPM)

$t_{transferia}$: ler/escrever dados (~ 0.01 ms por setor)

Cenário

Múltiplos processos fazem requisições ao disco simultaneamente. O SO precisa decidir **em que ordem** atender as requisições para minimizar o tempo total de seek.

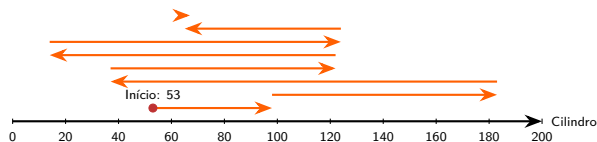
Exemplo – Fila de requisições (números de cilindros):

Posição atual da cabeça: **53**

Fila: 98, 183, 37, 122, 14, 124, 65, 67

Objetivo: Minimizar o **número total de movimentos** do braço (distância total percorrida em cilindros).

Regra: Atende na ordem de chegada.



Movimentos: $|53 - 98| + |98 - 183| + |183 - 37| + |37 - 122| + |122 - 14| + |14 - 124| + |124 - 65| + |65 - 67|$
 $= 45 + 85 + 146 + 85 + 108 + 110 + 59 + 2 = \mathbf{640}$ cilindros

Regra: Atende a requisição mais **próxima** da posição atual.

Início: 53 → 65 → 67 → 37 → 14 → 98 → 122 → 124 → 183

Movimentos: $12 + 2 + 30 + 23 + 84 + 24 + 2 + 59 = 236$ cilindros

Vantagens:

- Muito melhor que FCFS
- Minimiza movimentos localmente

Desvantagens:

- Pode causar **starvation** de requisições distantes
- Não é ótimo globalmente

Regra

O braço se move em uma direção, atendendo requisições no caminho. Ao chegar ao extremo, inverte a direção. Como um **elevador**.

Início: 53, movendo para a direita:

$53 \rightarrow 65 \rightarrow 67 \rightarrow 98 \rightarrow 122 \rightarrow 124 \rightarrow 183 \rightarrow [199] \rightarrow 37 \rightarrow 14$

Movimentos: $(199 - 53) + (199 - 14) = 146 + 185 = \mathbf{331}$ cilindros

Variante: C-SCAN (Circular SCAN)

Ao chegar ao extremo, volta ao **início** sem atender (como o elevador que só sobe). Mais justo: todos os cilindros têm tempo de espera similar.

Algoritmo	Movimentos	Starvation?	Complexidade
FCFS	640	Não	Nenhuma
SSTF	236	Sim	$O(n)$ por requisição
SCAN	331	Não	Baixa
C-SCAN	~350	Não	Baixa

Na prática

Sistemas operacionais modernos usam variantes do **SCAN** ou **C-SCAN** por oferecerem bom desempenho sem starvation. O Linux usa o "deadline scheduler" e o "mq-deadline" que combinam SCAN com prioridades e deadlines.

Exercício (40 min)

Dado: posição inicial = 50, fila de requisições:
82, 170, 43, 140, 24, 16, 190, 100

Para cada algoritmo (FCFS, SSTF, SCAN):

- 1 Determine a ordem de atendimento
- 2 Calcule o número total de movimentos
- 3 Desenhe o caminho do braço no diagrama

Bônus: Implemente em Python e compare automaticamente.

Para o SCAN

Assuma direção inicial: para a **direita** (cilindros maiores). Extremo: cilindro 199.

HDD:

- Acesso **mecânico** (braço + rotação)
- Seek time domina ($\sim 5\text{--}10\text{ ms}$)
- Scheduling é **crítico**
- Acesso sequencial $\gg$ aleatório

SSD:

- Acesso **eletrônico** (sem partes móveis)
- Latência uniforme ($\sim 50\text{ }\mu\text{s}$)
- Sem seek time mecânico
- Acesso aleatório $\approx$ sequencial

Resposta

Para SSDs, scheduling de requisições é **muito menos importante**. O kernel Linux usa o scheduler **none** (NOOP) para SSDs NVMe. Porém, SSDs têm seus próprios desafios: **wear leveling**, **garbage collection**, e **write amplification**.

Apresentação em Pannel + Exercícios

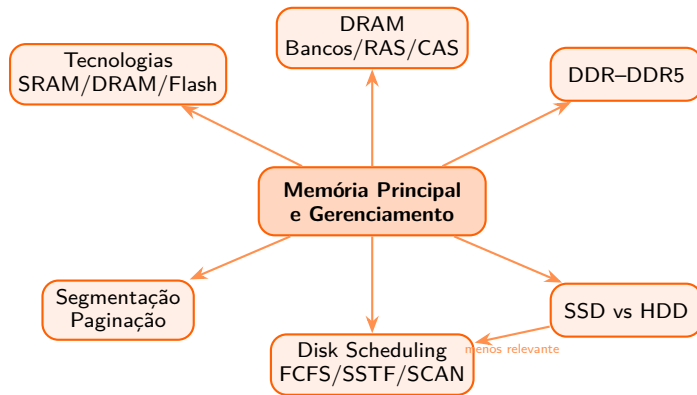
Peso: 5% da nota

Formato: Grupo (painel) + Individual (exercícios)

Prazo: 1 semana após a Aula 31

Entregar:

- ❶ **Slides do painel DDR** (grupo): 5–8 slides com especificações, inovações e impacto da geração pesquisada
- ❷ **Exercícios de scheduling** (individual):
 - Resolver FCFS, SSTF e SCAN para 2 filas de requisições
 - Calcular movimentos totais para cada
 - Comparar e justificar qual é melhor em cada caso
 - Responder: SSDs precisam de disk scheduling? Por que?



Conceitos-chave:

- SRAM: 6T, rápida, cache
- DRAM: 1T1C, densa, RAM
- Flash: não volátil, SSD
- DRAM: RAS/CAS/refresh
- DDR → DDR5: mais bandwidth
- Latência absoluta pouco muda
- Segmentação vs paginação
- FCFS, SSTF, SCAN
- SSDs: scheduling menos relevante

Competências desenvolvidas:

- Comparar tecnologias de memória
- Analisar especificações DDR
- Entender organização da DRAM
- Aplicar algoritmos de scheduling
- Argumentar sobre HDD vs SSD

Próximo módulo:

Memória Virtual – paginação, tabela de páginas, TLB e page faults!

Memória Virtual – A Ilusão da Memória Infinita

Metodologia: Aprendizagem Baseada em Problemas (PBL)

- Problema motivador: 8 GB de programa em 4 GB de RAM
- Conceito e motivação da memória virtual
- Paginação: páginas, frames, tabela de páginas
- Tradução de endereços: virtual → físico
- Page faults: o que acontece quando a página não está na RAM

Cenário

Seu computador tem **4 GB de RAM**. Você está rodando simultaneamente:

- Navegador Chrome (2 GB)
- IDE (Visual Studio Code, 1 GB)
- Jogo (3 GB)
- Sistema operacional (1.5 GB)

Total: **7.5 GB** de demanda em **4 GB** de RAM!

Pergunta: Como isso é possível? Por que o computador não trava?

Discutam em grupo (10 min):

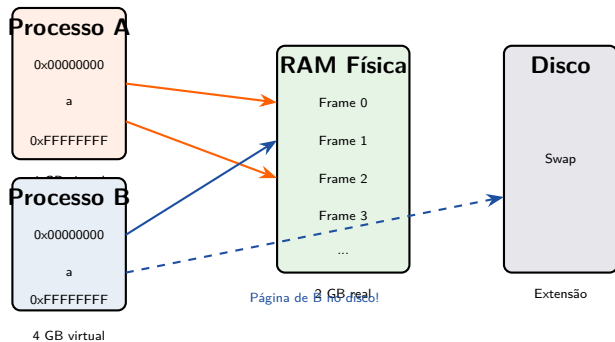
- 1 Todos os 7.5 GB precisam estar na RAM ao mesmo tempo?
- 2 O que cada programa "vê" como seu espaço de memória?
- 3 O disco pode ajudar de alguma forma?

Definição

Memória virtual é um mecanismo que cria a **ilusão** de que cada programa tem um espaço de endereçamento grande, contínuo e exclusivo, mesmo que a memória física disponível seja menor.

Três objetivos fundamentais:

- ❶ **Abstração:** Cada processo vê um espaço virtual de 2^{32} (ou 2^{64}) bytes, independente da RAM real
- ❷ **Proteção:** Um processo não pode acessar a memória de outro (isolamento)
- ❸ **Capacidade:** Usa o disco como extensão da RAM (swap/paging), permitindo rodar programas maiores que a RAM



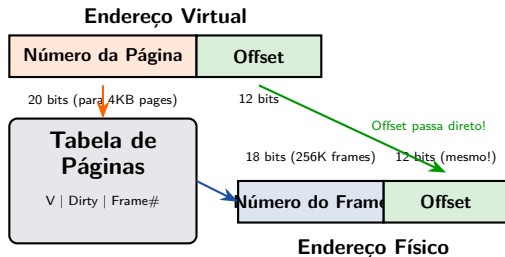
Processos A e B têm o **mesmo endereço virtual** (`0x00000000–0xFFFFFFFF`) mas são mapeados para **frames físicos diferentes**. Páginas não usadas ficam no disco.

Termo	Definição
Página (page)	Bloco de tamanho fixo no espaço virtual
Frame (page frame)	Bloco de mesmo tamanho na memória física
Tabela de páginas	Mapeamento: página virtual → frame físico
Page fault	Página acessada não está na RAM (está no disco)
Swap	Área do disco usada para armazenar páginas
Valid bit	Indica se a página está na RAM (1) ou no disco (0)
Dirty bit	Indica se a página foi modificada (precisa ser escrita no disco)

Tamanho típico de página: 4 KB (MIPS, x86) ou 16 KB (ARM, Apple M-series)

Para endereço de 32 bits: $2^{32}/4\text{KB} = 2^{20} \approx 1$ milhão de páginas.

Tradução de Endereço Virtual → Físico



Processo:

- 1 Extrair o **número da página** (bits mais significativos)
- 2 Usar como **índice** na tabela de páginas
- 3 Obter o **número do frame** correspondente
- 4 Concatenar frame + offset → endereço físico

Configuração: Endereço virtual de 16 bits, páginas de 256 bytes (2^8), RAM de 4 KB.

- Offset: $\log_2 256 = 8$ bits
- Número da página: $16 - 8 = 8$ bits $\rightarrow$ 256 páginas virtuais
- Número do frame: $\log_2(4096/256) = 4$ bits $\rightarrow$ 16 frames

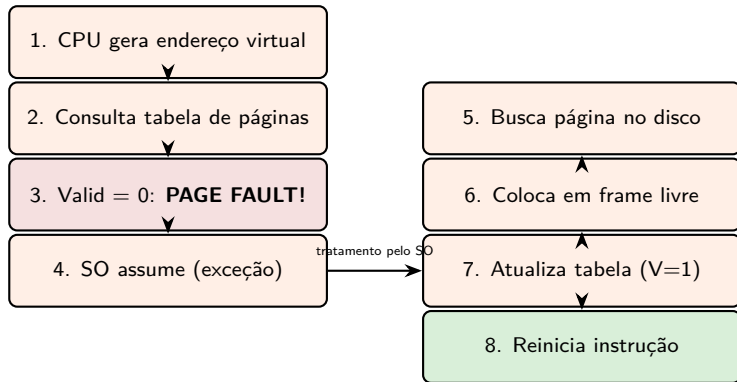
Tabela de páginas (parcial):

Página	Valid	Frame
0	1	5
1	1	2
2	0	disco
3	1	11

Endereço virtual 0x013A = 00000001 00111010

Página = 00000001 = 1 $\rightarrow$ Frame = 2 $\rightarrow$ Endereço físico = 0010 00111010 = 0x23A

E se Valid = 0? Page Fault!



Custo de um page fault

Acesso ao disco: $\sim 5\text{--}10\text{ ms}$ (HDD) ou $\sim 50\text{ }\mu\text{s}$ (SSD).

Comparado com acesso à RAM ($\sim 50\text{ ns}$): **100.000**× mais lento (HDD)!

Por isso, a taxa de page faults deve ser **extremamente baixa** ($< 0.001\%$).

Se não há frame livre, o SO precisa escolher uma vítima:

- 1 Selecionar uma página para substituir (algoritmo de substituição)
- 2 Se a página vítima está **dirty** (modificada): escrever no disco
- 3 Se está **clean** (não modificada): simplesmente descartar
- 4 Carregar a nova página no frame liberado
- 5 Atualizar ambas entradas na tabela de páginas

Analogia com cache

Memória virtual é uma "cache gigante" onde:

RAM = cache, Disco = memória principal.

Page fault = cache miss com penalidade **enorme**.

Por isso: **totalmente associativa** (página pode ir para qualquer frame) e sempre **write-back** (nunca write-through para disco!).

Para endereço virtual de 32 bits, páginas de 4 KB:

$$\text{Entradas} = \frac{2^{32}}{4 \times 1024} = 2^{20} = 1.048.576 \text{ entradas}$$

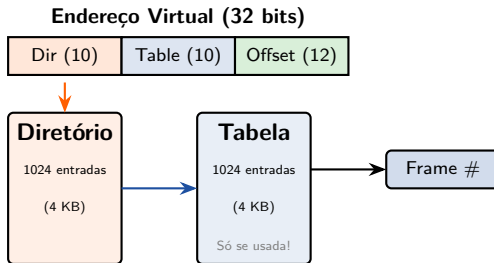
Se cada entrada tem 4 bytes: tabela = 4 MB **por processo!**

Para 64 bits: $2^{64}/4\text{KB} = 2^{52}$ entradas → **impossível!**

Soluções:

- **Tabela multinível:** Dividir em níveis (diretório + tabela). Só alocar os níveis usados.
- **Tabela invertida:** Uma entrada por **frame físico** (não por página virtual). Usa hash.
- **Páginas maiores:** 2 MB ou 1 GB ("huge pages") → menos entradas.

Tabela de Páginas Multinível (2 Níveis)



Tabelas de 2^o nível são alocadas sob demanda

Economia enorme para processos pequenos!

Economia: Se um processo usa apenas 8 MB de memória virtual, precisa de 1 diretório (4 KB) + 2 tabelas (8 KB) = **12 KB** em vez de 4 MB!

Retomando: como rodar 7.5 GB em 4 GB de RAM?

- ❶ Cada processo tem seu **espaço virtual** (4 GB no MIPS 32-bit)
- ❷ Apenas as **páginas ativas** de cada processo ficam na RAM
- ❸ Páginas não usadas ficam no **disco** (swap)
- ❹ Quando uma página é necessária: **page fault** → SO carrega do disco
- ❺ O **princípio da localidade** garante que poucas páginas são usadas de cada vez

Pergunta extra

O que acontece se **muitos** processos competem por RAM e os page faults se tornam frequentes? (**Thrashing!**
O sistema passa mais tempo trocando páginas do que executando. Solução: mais RAM ou fechar programas.)

O que aprendemos:

- Memória virtual: abstração, proteção, capacidade
- Páginas e frames (4 KB típico)
- Tabela de páginas: virtual $\rightarrow$ físico
- Tradução: página + offset $\rightarrow$ frame + offset
- Page fault: busca no disco
- Tabela multinível: economia de espaço

Para a próxima Aula:

- Revisar tradução de endereços
- Estudar: o que é TLB?
- Estudar: algoritmos FIFO e LRU para substituição de páginas

TLB e Algoritmos de Substituição de Páginas

Metodologia: Simulação de Tabela de Páginas + TLB

- TLB (Translation Lookaside Buffer): cache de traduções
- Fluxo completo: TLB → tabela de páginas → page fault
- Algoritmos de substituição: FIFO, LRU, Clock (Second Chance)
- Exercício prático: simular traduções e page faults

Problema

Cada acesso à memória agora requer **dois** acessos:

- 1 Acessar a **tabela de páginas** (na RAM) para obter o frame
- 2 Acessar o **dado** (na RAM) com o endereço físico

Resultado: desempenho cai pela **metade**!

Com tabela multinível: 3–4 acessos à RAM por acesso a dado!

Solução: TLB (Translation Lookaside Buffer)

Uma **cache pequena e rápida** que armazena as traduções mais recentes (página → frame). Tipicamente:

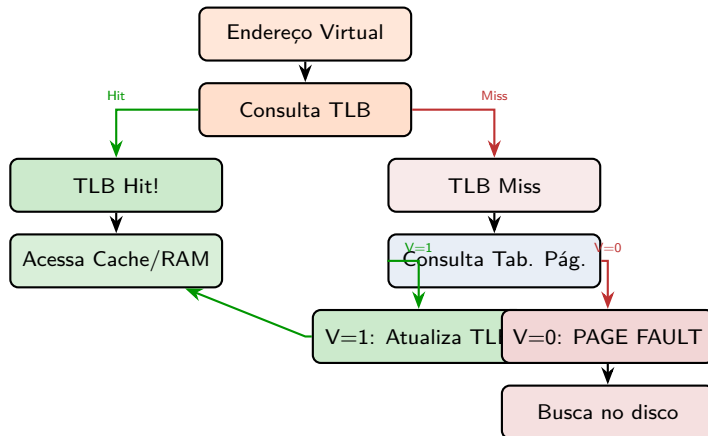
- 32–128 entradas
- Totalmente associativa
- Acesso em ~ 1 ciclo (paralelo com cache L1!)
- Hit rate: > **99%** em programas típicos

TLB (Translation Lookaside Buffer)

Valid	Tag (Página)	Frame	Dirty	Ref
1	0x0A3	5	0	1
1	0x001	2	0	1
1	0x0FF	11	0	1
0	—	—	0	1

Campos de cada entrada:

- **Valid:** entrada válida?
- **Tag:** número da página virtual
- **Frame:** número do frame físico correspondente
- **Dirty:** página foi modificada?
- **Reference:** página foi acessada recentemente? (para LRU)



Caso ideal: TLB hit → tradução em 1 ciclo → acesso à cache L1.
Caso raro: TLB miss + page fault → milhões de ciclos (acesso ao disco).

Exemplo: Acesso à memória sem e com TLB.

Cenário	Tempo de tradução	Total (tradução + dado)
Sem TLB (2 níveis)	$2 \times 50 \text{ ns} = 100 \text{ ns}$	$100 + 50 = \mathbf{150 \text{ ns}}$
Com TLB (hit)	1 ns	$1 + 50 = \mathbf{51 \text{ ns}}$
Com TLB (99% hit)	$0.99 \times 1 + 0.01 \times 100$	$\sim 2 + 50 = \mathbf{52 \text{ ns}}$

Resultado

Sem TLB: 150 ns (3× mais lento!).

Com TLB (99% hit rate): 52 ns (~4% de overhead).

A TLB torna a memória virtual **praticamente gratuita** em termos de desempenho!

Quando ocorre page fault e a RAM está cheia, qual página remover?

Algoritmo	Critério	Ótimo?
Ótimo (Bélády)	Remove a que será usada mais tarde	Sim (impossível!)
FIFO	Remove a mais antiga na RAM	Não
LRU	Remove a menos recentemente usada	Quase ótimo
Clock	Aproximação de LRU usando reference bit	Bom

Na prática

LRU é muito caro para implementar exatamente em hardware (precisaria ordenar todas as páginas a cada acesso). Sistemas operacionais usam o **algoritmo Clock** (Second Chance) como aproximação eficiente.

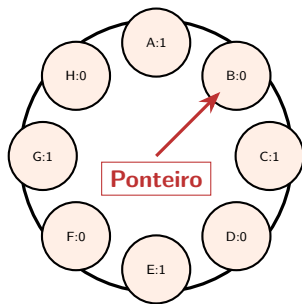
Exemplo: FIFO vs LRU vs Ótimo

Sequência de páginas: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5
3 frames na RAM.

Pág	FIFO			H/M	LRU			H/M	Ótimo			H/M
	F1	F2	F3		F1	F2	F3		F1	F2	F3	
1	1			M	1			M	1			M
2	1	2		M	1	2		M	1	2		M
3	1	2	3	M	1	2	3	M	1	2	3	M
4	4	2	3	M	4	2	3	M	1	2	4	M
1	4	1	3	M	4	1	3	M	1	2	4	H
2	4	1	2	M	4	1	2	M	1	2	4	H
5	5	1	2	M	5	1	2	M	5	2	4	M
1	5	1	2	H	5	1	2	H	5	2	4	M
2	5	1	2	H	5	1	2	H	5	2	4	H

Page faults: FIFO: 9 LRU: 8 Ótimo: 7 (de 12 acessos)

Algoritmo Clock (Second Chance)



Ref=1: página usada recentemente

Ref=0: candidata à substituição

Algoritmo:

Se Ref=1: muda para 0, avança

Se Ref=0: substitui esta página

O ponteiro "varre" as páginas em círculo. Páginas com Ref=1 ganham uma **segunda chance** ($\text{Ref} \leftarrow 0$). A primeira com Ref=0 é substituída.

Exercício (40 min)

Configuração: Endereço virtual 16 bits, páginas de 1 KB (2^{10}), RAM com 4 frames.

- Offset: 10 bits. Número da página: 6 bits (64 páginas virtuais)

Sequência de endereços virtuais: 0x0A3F, 0x1B00, 0x0A10, 0x2C80, 0x3D00, 0x1B44, 0x0A22, 0x4E00

Para cada endereço:

- 1 Extraia o número da página e o offset
- 2 Verifique se está na TLB (inicialmente vazia)
- 3 Se TLB miss: consulte a tabela de páginas
- 4 Se page fault: carregue no frame livre (ou substitua com FIFO)
- 5 Monte o endereço físico

Endereço 0x0A3F = 000010 1000111111

Página = 000010 = 2 Offset = 1000111111

TLB miss → Tabela de páginas: page fault → Carrega em Frame 0

Endereço físico: Frame 0 + offset = 00 1000111111 = 0x23F

Endereço 0x1B00 = 000110 1100000000

Página = 000110 = 6 Page fault → Frame 1

Endereço físico: 0x700

Endereço 0x0A10 = 000010 1000010000

Página = 2 → **TLB hit!** (carregada antes) Frame = 0

Endereço físico: 0x210

Continue para os demais endereços...

Lista de Exercícios + Simulação

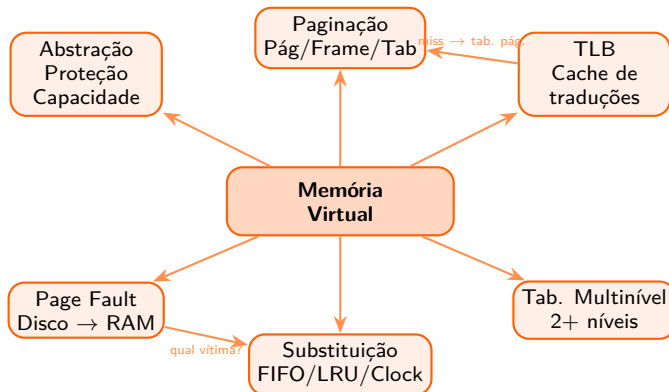
Peso: 7% da nota

Formato: Individual

Prazo: 1 semana após a Aula 33

Conteúdo:

- ❶ **Tradução de endereços:** 5 endereços virtuais → físicos (com tabela de páginas fornecida)
- ❷ **Page faults com FIFO e LRU:** simular sequência de 15 acessos a páginas com 4 frames. Contar page faults de cada.
- ❸ **TLB:** Simular 10 acessos com TLB de 4 entradas (LRU). Contar TLB hits, TLB misses e page faults.
- ❹ **Implementação:** Simulador simples em Python que recebe sequência de páginas, número de frames e algoritmo (FIFO/LRU), e reporta page faults.



Conceitos-chave:

- Memória virtual: ilusão de espaço grande
- Paginação: páginas (4 KB) + frames
- Tabela de páginas: virtual → físico
- Page fault: busca no disco ($\sim$ ms)
- TLB: cache de traduções ($>99\%$ hit)
- Tabela multinível (economia)
- FIFO, LRU, Clock
- Thrashing quando RAM insuficiente

Competências desenvolvidas:

- Traduzir endereços virtual → físico
- Simular TLB e tabela de páginas
- Aplicar FIFO, LRU, Clock
- Contar page faults
- Implementar simulador
- Analisar impacto no desempenho

Próximo módulo:

Barramento – tipos, arbitração, padrões (PCIe, USB, SATA)!

Barramento: Conexão entre Componentes

Metodologia: Pesquisa Rápida + Infográfico Colaborativo

- Conceito e tipos de barramento
- Barramentos síncronos vs assíncronos
- Arbitração: centralizada vs distribuída
- Padrões: PCI, PCIe, USB, SATA
- Largura de banda e gargalos
- Atividade: infográfico de um padrão de barramento

Tipo	Função	Direção	Largura típica
Dados	Transfere dados	Bidirecional	32, 64, 128 bits
Endereços	Indica posição	CPU → memória	32 ou 64 bits
Controle	Coordena operações	Ambas	4–8 linhas

Linhas de controle típicas:

- **Read/Write:** direção da transferência
- **Memory/IO:** destino é memória ou dispositivo
- **Bus Request / Bus Grant:** arbitração
- **IRQ:** pedido de interrupção
- **Clock:** sincronização (barramentos síncronos)
- **Ready/Wait:** dispositivo lento pede mais tempo

Fórmula

$$BW = \frac{\text{Largura (bits)} \times \text{Frequência (Hz)} \times \text{Transferências/ciclo}}{8}$$

Resultado em **bytes/segundo**.

Exemplos:

Barramento	Largura	Freq.	Transf./ciclo	BW
PCI (clássico)	32 bits	33 MHz	1	133 MB/s
PCI-X	64 bits	133 MHz	1	1066 MB/s
DDR4-3200	64 bits	1600 MHz	2 (DDR)	25.6 GB/s
PCIe 4.0 (x16)	16 lanes	16 GHz	1	32 GB/s

Exercício rápido: Calcule a BW de PCIe 5.0 x16 (32 GT/s por lane, 128b/130b encoding).

Síncrono:

- Operações controladas por **clock global**
- Todos os dispositivos sincronizados
- Mais rápido (previsível)
- Problema: clock limitado pelo dispositivo mais lento
- Cabo curto (clock skew)
- Exemplo: barramento de memória (FSB)

Assíncrono:

- Sem clock global
- Usa protocolo de **handshaking**
- Cada dispositivo opera na sua velocidade
- Mais flexível, mais lento
- Cabos mais longos
- Exemplo: USB, I²C

Handshaking assíncrono

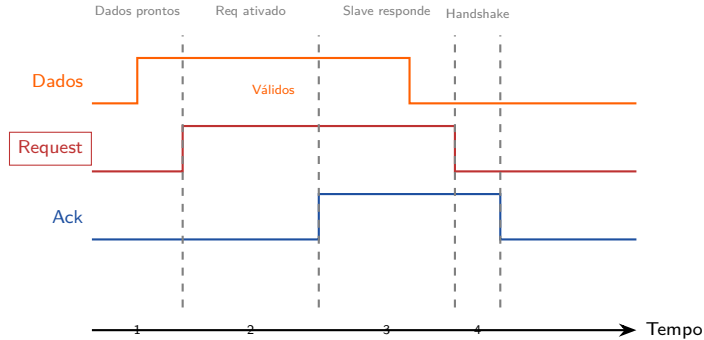
Master: coloca endereço + dados no barramento, ativa Request

Slave: processa, quando pronto ativa Acknowledge

Master: lê resposta, desativa Request

Slave: desativa Acknowledge

Handshaking – Diagrama de Sinais



4 fases: 1) Master coloca dados 2) Master ativa Request 3) Slave ativa Ack 4) Master desativa Request, Slave desativa Ack

Problema

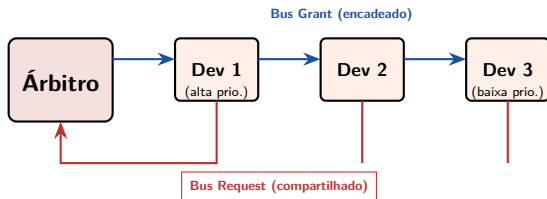
Se múltiplos dispositivos querem usar o barramento ao mesmo tempo, quem tem prioridade? É preciso um mecanismo de **arbitração**.

Centralizada:

- Um **árbitro** (controlador) decide
- Dispositivos enviam Bus Request
- Árbitro concede com Bus Grant
- Simples, ponto único de falha
- Variantes: Daisy Chain, Polling

Distribuída:

- Sem árbitro central
- Dispositivos "negociam" entre si
- Cada um tem prioridade fixa ou usa protocolo
- Mais robusto (sem ponto de falha)
- Mais complexo
- Exemplo: Ethernet (CSMA/CD)



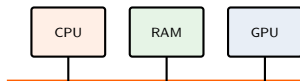
Como funciona:

- 1 Qualquer device ativa Bus Request (OR compartilhado)
- 2 Árbitro envia Bus Grant para Device 1
- 3 Se Dev 1 não precisa, passa Grant para Dev 2 (e assim por diante)
- 4 O primeiro na cadeia que precisa "captura" o Grant

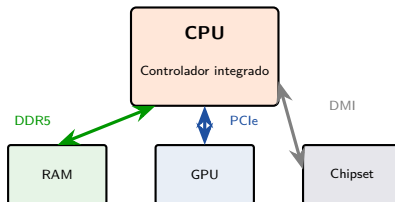
Problema: Dispositivos no final da cadeia podem sofrer **starvation**.

Evolução: Do Barramento Único ao Ponto-a-Ponto

Antigo: Barramento Compartilhado



Moderno: Ponto-a-Ponto



Tendência moderna:

- Barramento compartilhado → **links ponto-a-ponto** (sem contenção)
- Controlador de memória migrou para **dentro da CPU**
- Chipset cuida de periféricos lentos (USB, SATA, áudio)

PCI (1992):

- Barramento **compartilhado** paralelo
- 32/64 bits, 33/66 MHz
- BW máx: 533 MB/s (64 bit, 66 MHz)
- Obsoleto em desktops

PCIe (2003–):

- Links **ponto-a-ponto** seriais
- Lanes: x1, x4, x8, x16
- Cada geração dobra a velocidade
- Dominante em GPUs, SSDs NVMe

PCIe	Taxa/lane	BW x16	Ano
1.0	2.5 GT/s	4 GB/s	2003
2.0	5 GT/s	8 GB/s	2007
3.0	8 GT/s	16 GB/s	2010
4.0	16 GT/s	32 GB/s	2017
5.0	32 GT/s	64 GB/s	2019
6.0	64 GT/s	128 GB/s	2022

Versão	Taxa máxima	Conector típico	Ano
USB 1.1	12 Mbps	Type-A	1998
USB 2.0	480 Mbps	Type-A	2000
USB 3.0	5 Gbps	Type-A (azul)	2008
USB 3.1 Gen 2	10 Gbps	Type-C	2013
USB 3.2 Gen 2x2	20 Gbps	Type-C	2017
USB4	40 Gbps	Type-C	2019
USB4 v2.0	80 Gbps	Type-C	2022

Evolução do USB

- De 12 Mbps para 80 Gbps em 25 anos ($\sim 6700\times$!)
- USB4 é baseado em **Thunderbolt 3** (protocolo PCIe tunneling)
- USB-C: conector **reversível**, carrega dados, vídeo e energia

SATA (Serial ATA):

- Interface para discos (HDD/SSD)
- SATA I: 1.5 Gbps (150 MB/s)
- SATA II: 3 Gbps (300 MB/s)
- SATA III: 6 Gbps (600 MB/s)
- Protocolo AHCI (projetado para HDD)
- **Gargalo** para SSDs modernos!

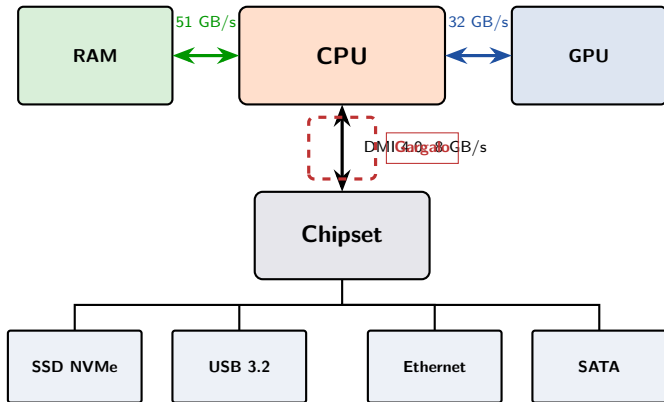
NVMe (sobre PCIe):

- Interface nativa para SSDs
- PCIe 3.0 x4: 3.5 GB/s
- PCIe 4.0 x4: 7 GB/s
- PCIe 5.0 x4: 14 GB/s
- Protocolo projetado para Flash
- Latência: $\sim 10 \mu s$ (vs $\sim 100 \mu s$ SATA)

Por que NVMe é tão mais rápido?

SATA usa protocolo **AHCI** (projetado para HDD com 1 fila de comandos).

NVMe usa **64K filas** com **64K comandos** cada, acesso paralelo aos chips Flash, e latência muito menor. SSD SATA ≤ 600 MB/s. SSD NVMe ≥ 3500 MB/s!



DMI 4.0 (8 GB/s) é compartilhado entre **todos** os periféricos.

Um SSD NVMe PCIe 4.0 pode consumir sozinho grande parte dessa banda, reduzindo a folga para os demais dispositivos.

Exercício (10 min)

Um sistema tem:

- 2 SSDs NVMe (PCIe 4.0 x4 cada = 7 GB/s)
- 1 placa de rede 10 GbE (1.25 GB/s)
- 2 portas USB 3.2 Gen 2 (1.25 GB/s cada)
- DMI 4.0 entre CPU e chipset: 8 GB/s total

Perguntas:

- 1 Qual a demanda total dos periféricos?
- 2 O DMI é gargalo? Se sim, em quanto %?
- 3 Se um SSD estiver conectado diretamente à CPU (via PCIe da CPU), o gargalo muda?

Resposta: Demanda = $2 \times 7 + 1.25 + 2 \times 1.25 = 17.75$ GB/s. DMI = 8 GB/s $\rightarrow$ gargalo de $\frac{17.75-8}{17.75} \approx 55\%$!
Com 1 SSD na CPU: demanda restante = 10.75 GB/s (ainda acima do DMI).

Atividade (50 min)

Cada grupo pesquisa um padrão de barramento e produz um **infográfico** digital (Canva, Google Slides ou PowerPoint):

Grupo	Padrão
1	PCIe (evolução 1.0 a 6.0, lanes, uso)
2	USB (evolução 1.1 a USB4, conectores, aplicações)
3	SATA vs NVMe (protocolo, desempenho, quando usar cada)
4	Thunderbolt (3 e 4, relação com USB4 e PCIe)
5	DDR5 e HBM (memória de alta banda, uso em GPUs)

Cada infográfico deve conter: especificações-chave, evolução histórica (timeline), diagrama de conexão, largura de banda, e 1 exemplo de aplicação real.

Infográfico avaliado por rubrica

Peso: 3% da nota

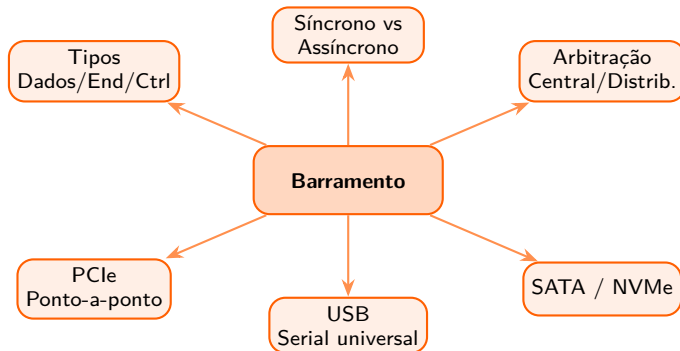
Formato: Grupo

Prazo: Final da aula (envio no Google Classroom)

Critérios do infográfico:

- ❶ Especificações técnicas corretas (velocidade, largura, evolução)
- ❷ Diagrama visual claro da conexão/arquitetura
- ❸ Timeline da evolução do padrão
- ❹ Comparação com alternativas (ex: SATA vs NVMe)
- ❺ Design visual atraente e organizado

Galeria virtual: Infográficos são postados no Classroom para apreciação e comentários mútuos.



Conceitos-chave:

- 3 tipos: dados, endereços, controle
- $BW = largura \times freq \times transf/ciclo$
- Síncrono (clock) vs Assíncrono (handshaking)
- Arbitração: daisy chain, polling, distribuída
- Compartilhado → ponto-a-ponto
- PCIe: 1.0 a 6.0, lanes
- USB: 1.1 a USB4, Type-C
- SATA vs NVMe: AHCI vs 64K filas
- DMI como gargalo moderno

Competências desenvolvidas:

- Calcular largura de banda
- Identificar gargalos
- Comparar padrões
- Pesquisar especificações
- Comunicar tecnicamente (infográfico)

Próximo módulo:

Arquiteturas Superescalares e Tópicos Avançados – o futuro do hardware!

Processadores Superescalares e Tópicos Avançados

Metodologia: Seminário Estudantil

- Limitações do pipeline escalar: IPC máximo de 1
- Processadores superescalares: múltiplas unidades funcionais
- Execução fora de ordem (out-of-order)
- Branch prediction dinâmica avançada (BTB)
- Multicore e multithreading (SMT / Hyper-Threading)
- Seminários dos grupos

Pipeline escalar (Módulo 7)

O pipeline MIPS de 5 estágios pode completar no máximo **1 instrução por ciclo** ($IPC \leq 1$). Na prática, com hazards: $IPC \approx 0.7-0.9$.

Como ir além de $IPC = 1$?

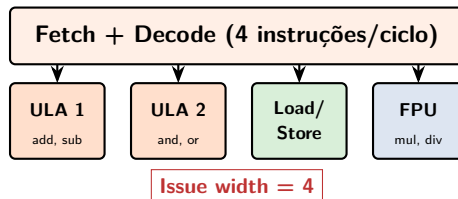
- ❶ **Aumentar o clock:** Limites físicos (potência, calor). Esbarrou no "Power Wall" (~ 2005).
- ❷ **Pipeline mais profundo:** 20+ estágios (Pentium 4: 31 estágios). Penalidade de branch cresce.
- ❸ **Executar múltiplas instruções por ciclo:** → **Superescalar!**

A solução moderna

Processadores superescalares buscam, decodificam e executam **2, 4, 6 ou 8+ instruções por ciclo**, alcançando $IPC > 1$.

Definição

Um processador **superescalar** possui múltiplas unidades funcionais (ULAs, unidades de load/store, FPU's) e pode **emitir** (issue) várias instruções por ciclo para essas unidades.



Issue width = n : até n instruções emitidas por ciclo.

IPC teórico máximo = n . Na prática: $IPC \approx 1.5-3$.

Pergunta-chave

O hardware pode emitir 4 instruções/ciclo, mas o **programa** tem 4 instruções independentes por ciclo?

Exemplo – Pouco ILP:

```
1:  add $t0, $t1, $t2
2:  sub $t3, $t0, $t4  ← depende de 1
3:  and $t5, $t3, $t6  ← depende de 2
4:  or  $t7, $t5, $t8  ← depende de 3
```

Cadeia de dependências → $IPC \approx 1$ (mesmo com issue width = 4).

Exemplo – Muito ILP:

```
1:  add $t0, $t1, $t2
2:  sub $t3, $t4, $t5  ← independente de 1
3:  and $t6, $t7, $t8  ← independente
4:  or  $t9, $s0, $s1  ← independente
```

Todas independentes → $IPC = 4$ possível!

In-Order (escalar simples):

- Instruções executadas na **ordem do programa**
- Se uma instrução está bloqueada (dependência), todas as seguintes esperam
- Simples, previsível
- IPC limitado

Out-of-Order (OoO):

- Instruções podem executar **fora da ordem** do programa
- Se I2 está bloqueada, I3 pode executar antes
- Explora ILP máximo disponível
- Hardware complexo (scoreboard, ROB)
- **Todos** os CPUs desktop/server modernos!

Exemplo

I1: lw \$t0, 0(\$s0) (cache miss: 200 ciclos!)

I2: add \$t1, \$t0, \$t2 (depende de I1)

I3: sub \$t3, \$t4, \$t5 (independente!)

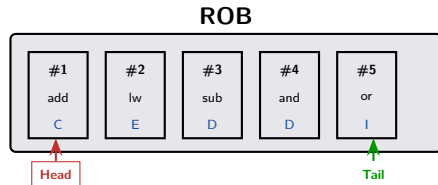
In-order: I3 espera 200 ciclos. **OoO:** I3 executa **imediatamente!**

Problema

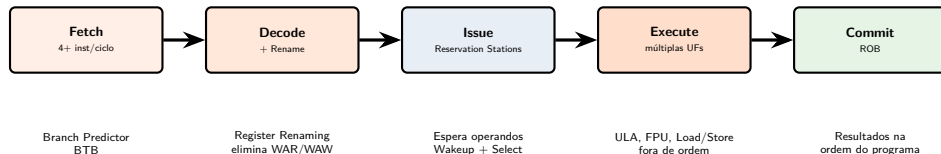
Se instruções executam fora de ordem, como garantir que o **resultado final** respeita a ordem do programa?
(Exceções, interrupções e branch mispredictions precisam de estado consistente!)

Solução: Reorder Buffer

O ROB armazena instruções na **ordem do programa**. Instruções podem executar fora de ordem, mas só "completam" (commit) na ordem original.



Instruções saem (commit) pela **head** na ordem do programa.



Estágios-chave adicionais:

- **Register Renaming:** elimina dependências falsas (WAR, WAW) usando registradores físicos extras
- **Reservation Stations:** filas de espera por operandos, emitem quando prontos
- **ROB:** garante ordem no commit e permite recuperação de mispredictions

Problema – WAR (Write After Read):

```
l1:  add $t0, $t1, $t2  (lê $t1)
l2:  sub $t1, $t3, $t4  (escreve $t1 – conflito com l1!)
```

l2 não pode escrever \$t1 antes que l1 leia. Mas a dependência é **falsa** – l2 não precisa do resultado de l1!

Solução: Renaming

```
l1:  add P7, P2, P3      (lê P2 = $t1 antigo)
l2:  sub P12, P4, P5     (escreve em P12, registrador físico novo!)
```

Agora l1 e l2 são **completamente independentes**!

CPUs modernas têm 100–300 registradores **físicos** para 32 registradores **arquiteturais**.

Em superescalares, predição é ainda mais crítica:

- Pipeline mais profundo → penalidade de misprediction maior (15–20 ciclos)
- Issue width = 4 → 4 instruções especulativas desperdiçadas por misprediction

Técnicas avançadas:

Técnica	Como funciona	Acurácia
BTB	Tabela com endereço-alvo do branch	–
BHT 2-bit	Preditor saturante (Módulo 7)	~87%
Correlating	Usa histórico de branches anteriores	~92%
Tournament	Combina local + global, escolhe melhor	~95%
TAGE	Múltiplas tabelas com históricos variados	~97%
Neural	Perceptrons treinados por branch	~98%+

CPUs modernas (AMD Zen 4, Apple M2) usam preditores tipo **TAGE** com >97% de acerto.

Definição

Especulação é executar instruções **antes de saber** se devem ser executadas (baseado em predições). Se a predição estiver errada, os resultados são descartados.

Tipos de especulação:

- **Control speculation:** Executa instruções após branch predito (flush se misprediction)
- **Data speculation:** Assume que um load não conflita com stores anteriores (reexecuta se errado)
- **Value prediction:** Prediz o resultado de instruções (pesquisa, raro em produção)

O ROB torna especulação segura

Resultados especulativos ficam no ROB sem fazer commit. Se a especulação estava errada: flush do ROB (descarta tudo após o branch). Se estava certa: commit normal.

Dinâmica (70 min)

Cada grupo apresenta em **10 min + 5 min de perguntas**.

Audiência preenche ficha de avaliação por pares.

Grupo	Tema
1	Superescalar: issue width, UFs, IPC real vs ideal
2	Out-of-Order: ROB, Reservation Stations, Register Renaming
3	Multicore: motivação, escalabilidade, Lei de Amdahl
4	GPU Computing: SIMT, CUDA, aplicações em IA
5	RISC-V: ISA aberta, ecossistema, futuro

Cada apresentação deve incluir: conceito, diagrama de blocos, exemplo de processador real, vantagens/desvantagens, e uma pergunta para a turma.

O que aprendemos:

- $IPC > 1$: superescalar
- Múltiplas UFs + issue width
- ILP: paralelismo de instruções
- In-order vs out-of-order
- ROB: commit na ordem
- Register renaming: elimina WAR/WAW
- Branch prediction avançada
- Especulação

Para a próxima Aula:

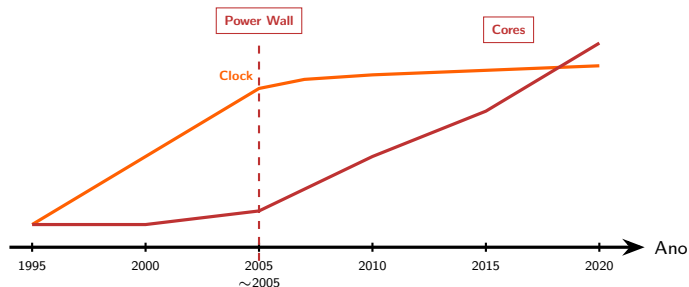
- Preparar argumentos para a mesa redonda
- Tema: "Mais cores, mais especialização, ou ambos?"
- Pesquisar: tendências 2024–2030

Multicore, Tendências e Encerramento

Metodologia: Mesa Redonda + Encerramento

- Multicore e multithreading (SMT)
- VLIW e DSPs
- GPU computing e aceleradores
- Tendências: chiplets, neuromórfico, quântico
- Mesa redonda: o futuro da arquitetura
- Revisão geral para a prova

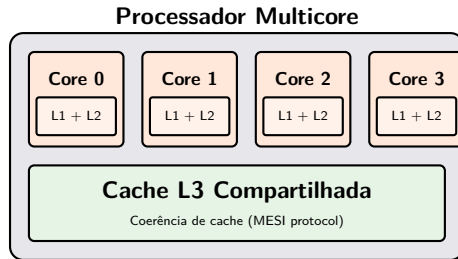
Do Single-Core ao Multicore



Power Wall (~2005)

A frequência do clock parou de crescer porque a **dissipação de potência** se tornou inviável. $P \propto f \times V^2$.

Solução: manter o clock e adicionar **mais cores**.



Desafios do multicore:

- **Lei de Amdahl:** Speedup limitado pela fração sequencial. Com 10% serial: máx $10\times$ com ∞ cores.
- **Coerência de cache:** Se Core 0 modifica um dado que Core 1 tem em cache, como manter consistência?
- **Programação paralela:** O software precisa ser escrito para explorar múltiplos cores (threads, OpenMP, etc.)

Ideia

Um único core executa instruções de **múltiplas threads simultaneamente**, compartilhando as unidades funcionais. Preenche "bolhas" de uma thread com instruções de outra.

Sem SMT



UFs ociosas!

Thread A
Thread B

Com SMT (2 threads)



UFs preenchidas!

Exemplos: Intel Hyper-Threading (2 threads/core), AMD SMT (2 threads/core).
Ganho típico: ~15–30% a mais de throughput sem hardware extra significativo.

Conceito

Em vez do hardware detectar paralelismo (superescalar), o **compilador** agrupa instruções independentes em uma "palavra de instrução longa" que especifica múltiplas operações.

	Superescalar	VLIW
Quem encontra ILP?	Hardware (dinâmico)	Compilador (estático)
Complexidade HW	Alta	Baixa
Complexidade SW	Baixa	Alta
Consumo de energia	Maior	Menor
Exemplos	Intel x86, ARM	Intel Itanium, TI C6000
Uso atual	Dominante em CPUs	DSPs, processadores embarcados

VLIW é eficiente para **DSPs** (processamento de sinais) onde os padrões de acesso são previsíveis e o compilador pode otimizar bem.

GPU = Massive SIMD/SIMT

GPUs têm milhares de cores simples otimizados para executar a **mesma instrução** em milhares de dados simultaneamente (SIMT).

	CPU	GPU
Cores	4–24 (complexos)	1000–16000 (simples)
Clock	3–5 GHz	1–2 GHz
Cache	Grande (L1/L2/L3)	Pequena
Controle	OoO, especulação	In-order, SIMT
Ideal para	Código sequencial	Paralelismo massivo
TFLOPS (FP32)	~0.5–1	~20–80

Aplicações: IA/Deep Learning, gráficos, mineração, simulação científica, criptografia.

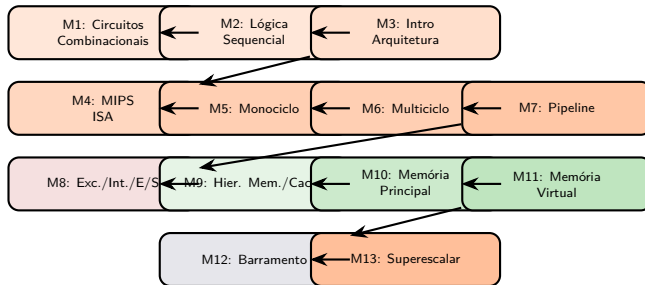
Programação: CUDA (NVIDIA), OpenCL, ROCm (AMD).

- ❶ **Chiplets e 3D Stacking:** múltiplos chips pequenos em um pacote (AMD Zen 4, Apple M-series). Melhor yield, mix de tecnologias.
- ❷ **Aceleradores especializados:** TPUs (Google, para IA), NPUs (Neural Processing Units em smartphones), FPGAs para computação reconfigurável.
- ❸ **RISC-V:** ISA aberta e livre. Crescimento exponencial em IoT, embarcados, e até servidores. SiFive, Alibaba T-Head.
- ❹ **Computação neuromórfica:** Chips que imitam neurônios (Intel Loihi 2). Ultra-baixo consumo para IA edge.
- ❺ **Computação quântica:** Qubits, superposição, entrelaçamento. IBM Eagle (1121 qubits). Ainda experimental.
- ❻ **Processing-in-Memory (PIM):** Computação dentro dos chips de memória. Reduz tráfego CPU↔RAM (Samsung HBM-PIM).
- ❼ **CXL (Compute Express Link):** Novo protocolo sobre PCIe para memória compartilhada entre CPU, GPU e aceleradores.

Regras (40 min)

Tema: "O futuro da arquitetura de computadores: mais cores, mais especialização, ou ambos?"

- 3 posições (sorteadas):
 - 1 "Mais cores homogêneos" (multicore escalável)
 - 2 "Mais especialização" (GPUs, TPUs, NPUs para cada tarefa)
 - 3 "Heterogêneo é o futuro" (big.LITTLE, CPU+GPU+NPU integrados)
- 10 min de preparação em grupo
- 20 min de debate (5 min cada posição + réplicas)
- 10 min de síntese coletiva



13 módulos, do transistor ao processador superescalar. Cada módulo construiu sobre os anteriores!

Desempenho:

$$T_{exec} = N_{inst} \times CPI \times T_{clock}$$

$$\text{Speedup} = \frac{T_{antigo}}{T_{novo}}$$

$$\text{Lei de Amdahl: } S = \frac{1}{(1 - f) + f/k}$$

Memória:

$$AMAT = t_{hit} + m \times t_{penalty}$$

$$AMAT_{multi} = t_{L1} + m_{L1}(t_{L2} + m_{L2}(\dots))$$

Pipeline:

$$CPI_{efetivo} = 1 + stalls_{dados} + stalls_{controle}$$

$$S_{pipeline} \approx \frac{k}{1 + \text{stall cycles per inst}}$$

Cache: Offset = $\log_2(\text{bloco})$, Índice = $\log_2(\text{conjuntos})$, Tag = 32 – ind – off

Seminário (avaliação por rubrica + pares)

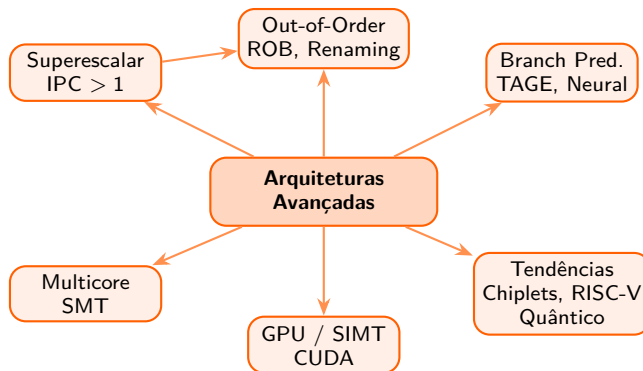
Peso: 7% da nota

Formato: Grupo

Avaliação:

- ❶ **Rubrica do professor (70%):** corretude técnica, clareza, diagramas, profundidade, respostas a perguntas
- ❷ **Avaliação por pares (30%):** cada aluno avalia os outros seminários em: clareza da explicação, qualidade dos slides, capacidade de responder perguntas

Cada grupo entrega: Slides do seminário (PDF) + referências usadas.



Conceitos-chave:

- Superescalar: issue width > 1
- ILP: paralelismo de instruções
- Out-of-order + ROB
- Register renaming (WAR/WAW)
- Branch prediction avançada
- Especulação
- Multicore + SMT
- VLIW (compilador encontra ILP)
- GPU: SIMT, milhares de cores
- Tendências: chiplets, RISC-V, PIM

Competências desenvolvidas:

- Analisar ILP e gargalos
- Entender OoO execution
- Comparar paradigmas (superescalar, VLIW, GPU)
- Aplicar Lei de Amdahl a multicore
- Pesquisar e apresentar temas avançados
- Debater sobre o futuro

Próxima parada:

PROVA 2! Módulos 9–13.

Boa revisão e bons estudos!

Parabéns por chegarem até aqui!

Em 13 módulos, vocês viajaram do **transistor** ao **processador superescalar**:

- Construíram somadores e flip-flops no Logisim
- Programaram em assembly MIPS no MARS
- Projetaram um processador monociclo completo
- Analisaram pipeline, hazards e forwarding
- Entenderam cache, memória virtual e TLB
- Exploraram o futuro: multicore, GPU, RISC-V

O hardware é a base de tudo que o software faz.

Agora vocês entendem o que acontece **por baixo dos panos**.

Bons estudos e sucesso na carreira!